📖

Redis란? / Redis의 장점

✅ Redis란?

Redis의 의미를 인터넷에 검색해보면 아래와 같이 나온다.

> 레디스(Redis)는 Remote Dictionary Server의 약자로서, “키-값” 구조의 비정형 데이터를 저장하고 관리하기 위한 오픈 소스 기반의 비관계형 데이터베이스 관리 시스템(DBMS)이다.
> - 위키백과 -

너무 어렵게 적혀져있다. First Word 법칙에 따라 쉽게 바꿔서 이해해보자.

> **Redis는 데이터 처리 속도가 엄청 빠른 NoSQL 데이터베이스**이다.

이렇게 기억하고 있어도 충분하다. NoSQL 데이터베이스를 풀어서 얘기하자면 Key-Value의 형태로 저장하는 데이터베이스라고 생각하면 된다.

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

출처 : https://redis.io/nosql/key-value-databases/

✅ Redis의 장점

Redis는 다양한 장점을 가지고 있다. 여러 특징 중 딱 1가지만 확실하게 기억해라.

> **레디스(Redis)는 인메모리(in-memory)에 모든 데이터를 저장한다.**
> **그래서 데이터의 처리 성능이 굉장히 빠르다.**

MySQL과 같은 RDBMS의 데이터베이스는 대부분 디스크(Disk)에 데이터를 저장한다. 하지만 Redis는 메모리(RAM)에 데이터를 저장한다. 디스크(Disk)보다 메모리(RAM)에서의 데이터 처리속도가 월등하게 빠르다. 이 때문에 Redis의 데이터 처리 속도가 RDBMS에 비해 훨씬 빠르다.



Redis 주요 사용 사례

Redis의 사용 사례를 검색해보면 아주 다양하다. 검색했을 때 아래와 같은 사용 사례를 확인할 수 있다.

- 캐싱 (Caching)
- 세션 관리 (Session Management)
- 실시간 분석 및 통계 (Real-time Analystics)
- 메시지 큐 (Message Queue)
- 지리공간 인덱싱 (Geospatial Indexing)
- 속도 제한 (Rate Limiting)
- 실시간 채팅 및 메시징 (Real-time Chat And Messaging)

레디스(Redis)에 내장된 기능이 다양하다보니 여러 용도로 사용된다. 레디스를 처음 배우는 입장에서 이 모든 사례를 익히려고 하면 막막할 수 밖에 없다. 따라서 파레토의 법칙에 의해 현업에서 많이 사용되는 '**캐싱(데이터 조회 성능 향상)**'만 집중적으로 살펴볼 것이다.

'캐싱(데이터 조회 성능 향상)'의 사례를 학습하다보면 자연스럽게 레디스의 기본 기능들이 익혀진다. 이렇게 배우면 레디스를 정말 빨리 익힐 수 있다. 그런 뒤에 레디스의 다른 사례에 대해서도 조금씩 학습하면서 살을 붙여나가면 된다.

kha302570@gmail.com

백엔드 채용 공고에 종종 등장하는 ‘대용량 트래픽 처리 경험’, ‘Redis 사용 경험’

✅ 최근 백엔드 개발자 채용 공고에 자주 등장하는 스펙

백엔드 개발자 채용 공고를 살펴보면 ‘**대용량 트래픽**’에 관한 얘기와 ‘**NoSQL**’에 관한 얘기가 자주 나온다. 이 역량이 서비스가 점점 고도화 될 수록 빼놓을 수 없는 역량이기 때문이다. 이 2가지 역량을 채울 수 있는 방법 중 하나가 Redis이다.

대용량 트래픽을 처리하기 위해서 필수적으로 사용되는 기능이 Redis의 캐싱(Caching) 기능이다. 또한 Redis는 NoSQL의 일종이다. 따라서 Redis를 학습함으로써 최근 회사에서 요구하는 자격 요건을 맞출 수 있도록 노력해보자.

자격요건

- 경력 3년 이상 선호
- Spring framework + Spring Boot
- Spring + Mybatis or Spring + JPA
- AWS(Amazon Web Services) 운영/개발 경험을 보유하신분 (필수)

우대사항

- MSA(Microservices Architecture) 개발 경험 보유
- 높은 AWS 활용 역량을 갖추신 분
- 대용량 트래픽 서비스 설계/개발/운영 경험 보유
- 핀테크 / 금융 관련 서비스 설계/개발/운영 경험 보유
- 웹 프론트엔드 개발 경험 보유
- Git-flow를 사용한 소스관리 경험 보유

자격요건

- Java 언어에 능숙하신 분
- Spring Framework, Spring Boot 개발 역량을 보유하신 분
- MySql, Redis 개발 역량을 보유하신 분
- git 등의 SCM을 통한 협업 경험이 있는 분
- 강한 자율성과 책임감을 보유하신 분
- 함께 성장하고 배우는 것에 가치를 두는 진취적인 자세를 보유하신분
- 문제를 스스로 정의하고 해결할 수 있는 분

우대사항

- MSA 환경에서의 개발 경험을 보유하신 분
- Redis 개발 경험을 보유하신 분
- AWS 사용 경험자
- ELK 사용 경험자

주요업무

- A/B 테스트 플랫폼 백엔드 서버 개발
- 데이터분석 플랫폼 백엔드 개발
- 인앱/푸시 메시지 서비스 백엔드 개발
- 신규 서비스 설계 및 개발
- 대용량 트래픽을 고려한 서버 설계
- SDK 개발

자격요건

- Java/Kotlin 기반 서버 개발 경험
- Spring Framework 사용 경험
- RDBMS, ORM 사용 경험

우대사항

- Reactive Streams, Coroutine에 대한 이해
- NoSQL (Redis, Elasticsearch, MongoDB) 사용 경험
- Messaging Service (Kafka, SQS, Kinesis) 사용 경험
- MSA 개발 경험
- 고가용성 아키텍처 설계, 개발 경험
- AWS, Docker, Kubernetes 사용 경험



로컬(Windows, MacOS)에서 Redis 설치하기

✅ MacOS에서 Redis 설치하기

> Install Redis on macOS
> Use Homebrew to install and start Redis on macOS
> https://redis.io/docs/latest/operate/oss_and_stack/install/install-redis/install-redis-on-mac-os/

Redis 설치

```
# HomeBrew 설치 (이미 설치되어 있다면 재설치 필요 X)
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# HomeBrew 설치 여부 확인
$ brew --version

# redis 설치
$ brew install redis

# redis 실행
$ brew services start redis

# redis 실행 여부 확인
$ brew services info redis

# reids 중지
$ brew services stop redis
```

Redis에 접속

Redis에 접속할 수 있는 프로그램이 `redis-cli`이다. `redis-cli`로 Redis에 명령을 내릴 수도 있다.

```
$ redis-cli # Redis에 접속

127.0.0.1:6379> ping
PONG # 이렇게 출력되면 정상적으로 Redis가 실행되고 있다는 뜻
```

✅ Windows에서 Redis 설치하기

Windows를 쓰고 있다면 아래 링크를 참고해 Redis를 설치하자.

> [Redis] 윈도우10 환경에서 레디스 설치하기
> Redis 설치 프로그램 다운로드 아래 링크에 접속하여 msi 확장자의 Redis 설치 프로그램을 다운로드한다. https://github.com/microsoftarchive/redis/releases Releases · microsoftarchive/redis Redis is an in-memory database that persists on disk. The data model is key-value, but many different kind of values
> https://ittrue.tistory.com/318



[REDIS] 📚 Window10 환경에 Redis 설치 & 설정

Redis 윈도우 설치 Redis 다운로드 페이지로 이동하여 설치 프로그램을 다운로드하고 설치를 진행한다. Releases · microsoftarchive/redis Redis is an in-memory database that persists on disk. The data model is key-value, but many different kind of values are supported: Strings, Lists, Sets, Sorted Sets, Hashes -

https://inpa.tistory.com/entry/REDIS-📚-Window10-환경에-Redis-설치하기



kkb3025@gmail.com



Redis 기본 명령어 익히기



대부분의 Redis 강의나 책을 보시면 알겠지만 자료 구조(string, list, hash, Set, sorted Set 등)를 설명하고, 수없이 많은 명령어(SET, GET, EXISTS, DEL, HSET 등)를 설명하고 있다. 하지만 레디스를 빨리 익히려면 저기 있는 모든 개념을 다 익힌 뒤에 실습을 하면 늦다. 머릿 속에 한 가지 개념만 들어와도 벅찬데 수십개의 개념을 머리에 넣으려고 하다보니, 막상 실습 때 아무 것도 기억나지 않는 것이다.

따라서 오리엔테이션 때 얘기했던 '파레토의 법칙'에 입각해서 딱 필요한 개념만 배우고 바로 실습을 통해 레디스를 써먹어 볼 것이다. 장담컨데 이렇게 레디스를 학습하면 정말 빠르게 익힐 수 있다.

이 강의에서는 딱 7가지의 명령어만 배울 것이다. 레디스를 조회 성능을 개선할 때는 이 7가지 명령어만 알아도 충분하다. 그러다 레디스를 쓰면서 추가적으로 필요한 기능이 있으면 그때그때 찾아서 사용하면 된다.

✅ 데이터(Key, Value) 저장하기

```
# set [key 이름] [value]
$ set jaeseong:name "jaeseong park" # 띄워쓰기 해서 저장하려면 쌍따옴표로 묶어주면 됨
$ set jaeseong:hobby soccer
```

✅ 데이터 조회하기 (Key로 Value 값 조회하기)

```
# get [key 이름]
$ get jaeseong:name
$ get jaeseong:hobby

$ get pjs:name # 없는 데이터를 조회할 경우 (nil)이라고 출력됨
```

✅ 저장된 모든 key 조회하기

```
$ keys *
```

✅ 데이터 삭제하기 (Key로 데이터 삭제하기)

```
# del [key 이름]
$ del jaeseong:hobby

$ get jaeseong:hobby # 삭제됐는 지 확인
```

✅ 데이터 저장 시 만료시간(TTL) 정하기

레디스는 RDBMS와는 다르게 데이터 저장 시 만료시간을 설정할 수 있다. 즉, 영구적으로 데이터를 저장하지 않고 일정 시간이 되면 데이터가 삭제되도록 셋팅할 수 있다.

레디스의 특성상 메모리 공간이 한정 되어 있기 때문에 모든 데이터를 레디스에 저장할 수 없다. 따라서 만료시간(TTL)을 활용해 자주 사용하는 데이터만 레디스에 저장해놓고 쓰는 식으로 활용한다.

```
# set [key 이름] [value] ex [만료 시간(초)]
$ set jaeseong:pet dog ex 30
```

✅ 만료시간(TTL) 확인하기

```
# ttl [key 이름]
# 만료 시간이 몇 초 남았는 지 반환
# 키가 없는 경우 -2를 반환
# 키는 존재하지만 만료 시간이 설정돼 있지 않은 경우에는 -1을 반환
$ ttl jaeseong:pet
$ ttl jaeseong:name
$ ttl pjs:name
```

✅ 모든 데이터 삭제하기

```
$ flushall
```

khb3025@gmail.com

📖

Redis에서 Key 네이밍 컨벤션 익히기

> 💡 Redis의 Key 이름을 잘 짓는 건 굉장히 중요하다. 따라서 현업에서 자주 사용하는 Key 네이밍 컨벤션을 배워보자. (꼭 이렇게 적어야만 하는 건 아니니 필요에 따라 응용해서 사용해도 된다.)

✅ 현업에서 자주 활용하는 네이밍 컨벤션

회사마다 여러가지 컨벤션이 존재하겠지만 딱 한가지만 기억하자.

> 콜론(`:`)을 활용해 계층적으로 의미를 구분해서 사용

예시를 살펴보면 바로 이해가 될 것이다.

- `users:100:profile` : 사용자들(users) 중에서 PK가 100인 사용자(user)의 프로필(profile)
- `products:123:details` : 상품들(products) 중에서 PK가 123인 상품(product)의 세부사항(details)

이런 식으로 컨벤션을 정했을 때의 장점이 있다.

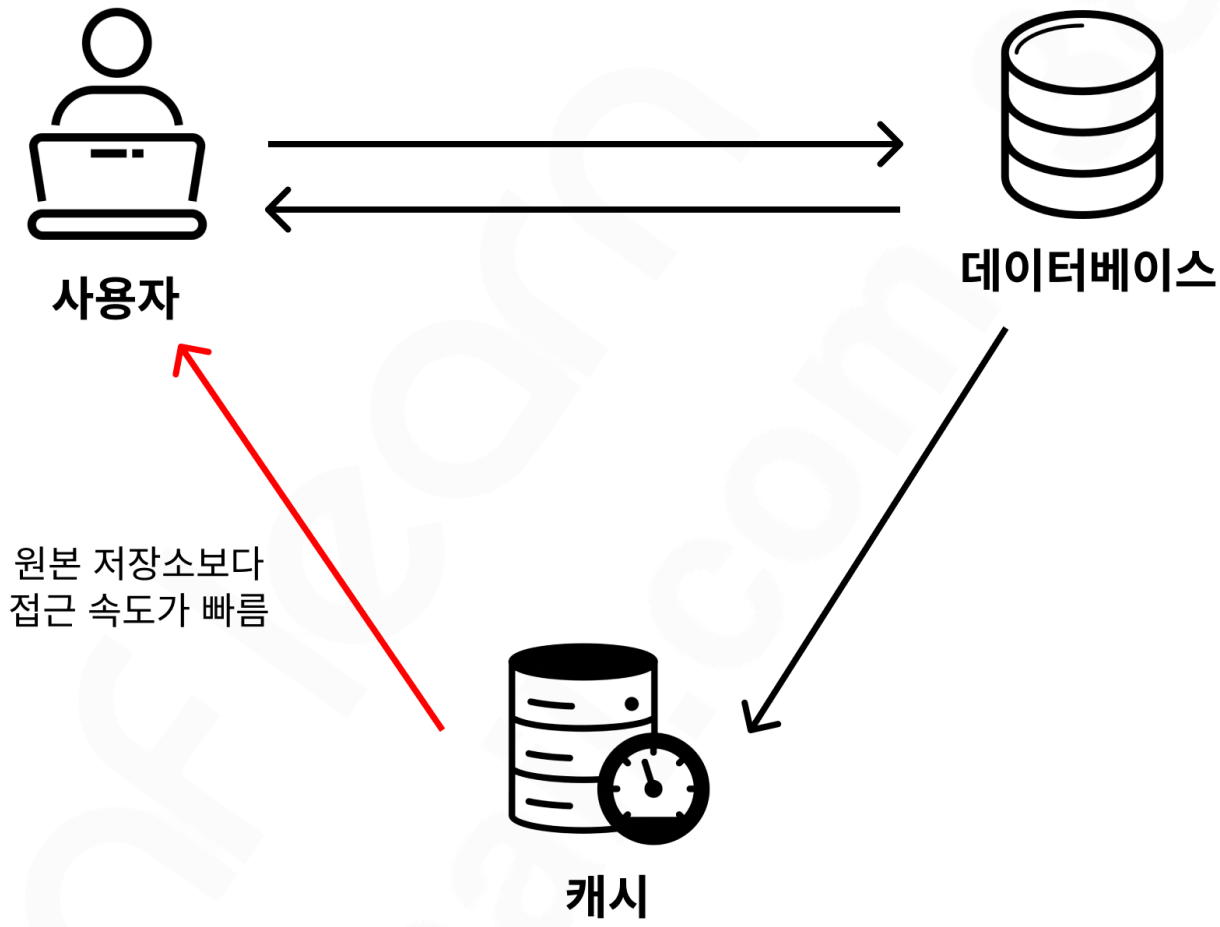
1. **가독성** : 데이터의 의미와 용도를 쉽게 파악할 수 있다.
2. **일관성** : 컨벤션을 따름으로써 코드의 일관성이 높아지고 유지보수가 쉬워진다.
3. **검색 및 필터링 용이성** : 패턴 매칭을 사용해 특정 유형의 Key를 쉽게 찾을 수 있다 .
4. **확장성** : 서로 다른 Key와 이름이 겹쳐 충돌할 일이 적어진다.

kb302571@gmail.com

캐시(Cache), 캐싱(Caching)이란?

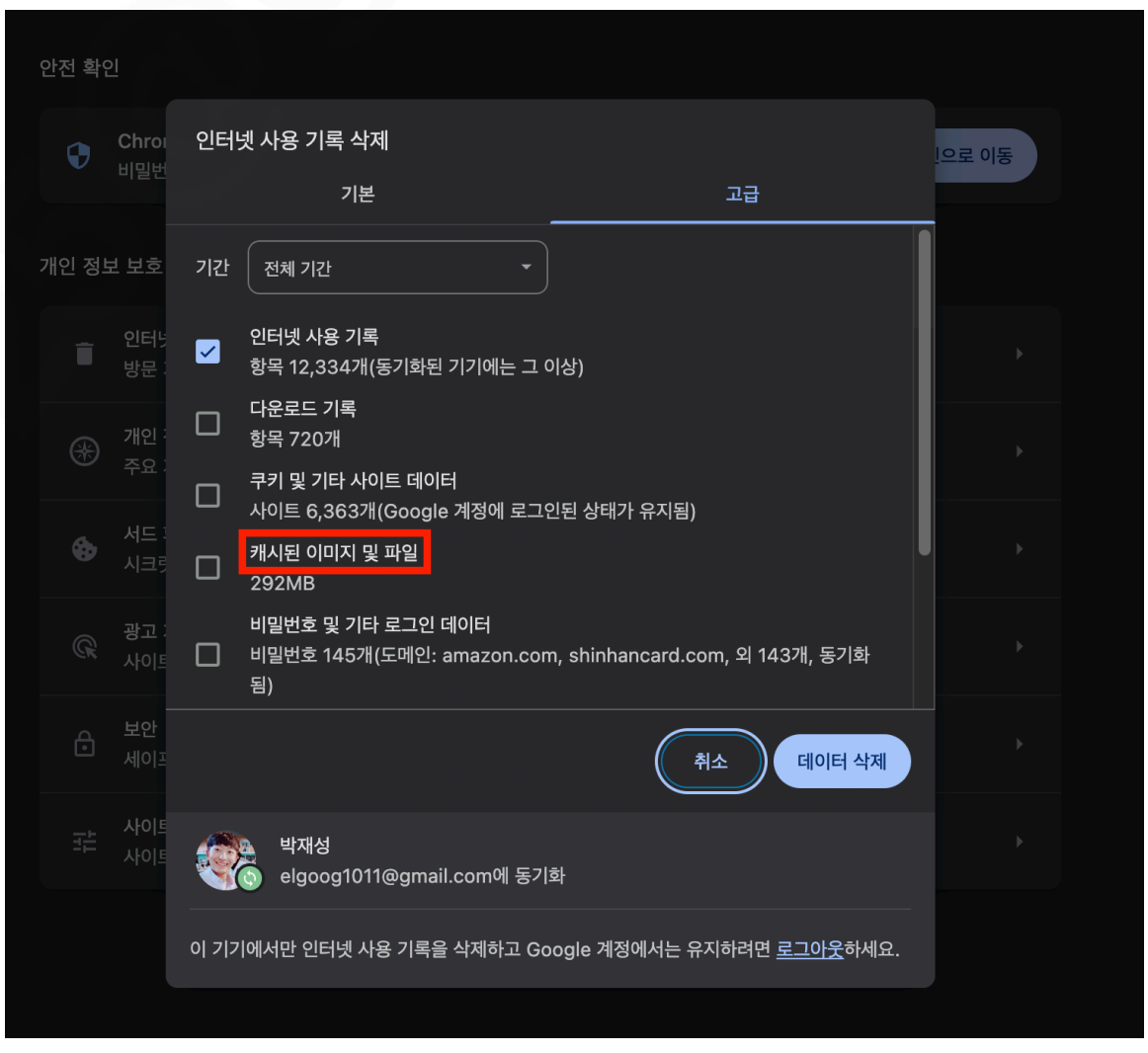
✅ 캐시(Cache)란?

캐시(Cache)란, **원본 저장소보다 빠르게 가져올 수 있는 임시 데이터 저장소**를 의미한다.



참고로 캐시(Cache)라는 단어는 Redis에서만 쓰이는 용어는 아니고 전반적인 개발 분야에서 통용돼서 쓰인다.

[예시]



✅ 캐싱(Caching)이란?

캐싱(Caching)이란 **캐시(Cache, 임시 데이터 저장소)에 접근해서 데이터를 빠르게 가져오는 방식**을 의미한다.

현업에서는 아래와 같이 얘기하는 편이다.

> “이 API는 응답 속도가 너무 느린데? 이 응답 데이터는 **캐싱(Cahing)** 해두고 쓰는 게 어때?’

이 말을 풀어서 설명하자면 **‘API 응답 결과를 원본 저장소보다 빠르게 가져올 수 있는 임시 데이터 저장소에 저장해두고, 빠르게 조회할 수 있게 만드는 게 어때?’**라는 의미이다.

kkb3025@gmail.com

📖 데이터를 캐싱할 때 사용하는 전략 (Cache Aside, Write Around)

👩🏻 레디스를 캐시로 쓸 때 어떤 방식으로 사용할 지 전략이 다양하다. 처음에는 그 중에서 딱 2가지의 전략만 배워도 충분하다. 이 전략이 현업에서 가장 많이 사용되고 있는 전략이다. 나머지의 전략들은 이 2가지 전략이 익숙해진 뒤에 차근차근 추가로 학습하면 된다. *(파레토의 법칙)*

✅ Cache Aside (= Look Aside, Lazy Loading) 전략

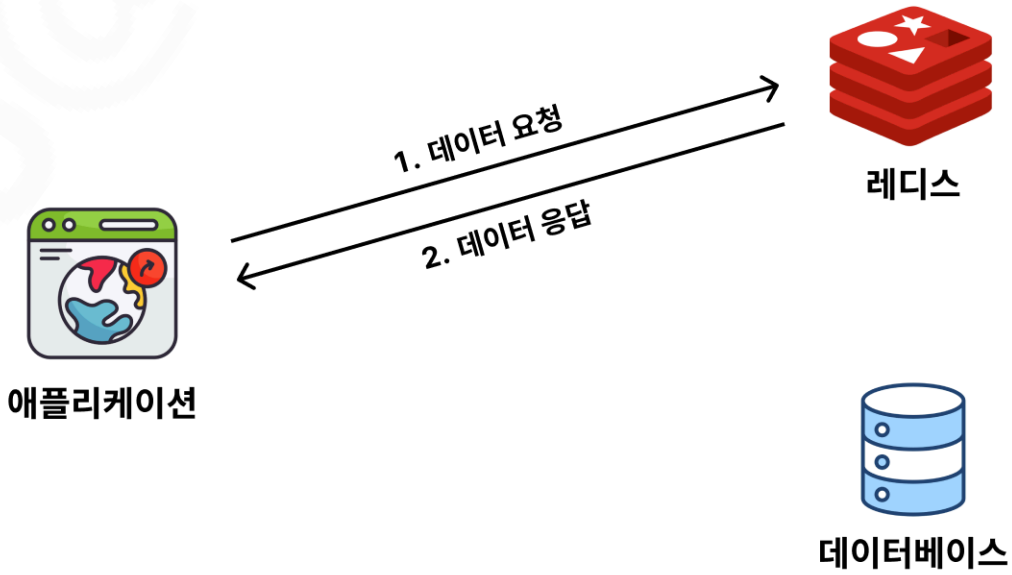
데이터를 조회할 때 주로 사용하는 전략이 **Cache Aside 전략**이다. **Look Aside 전략** 또는 **Lazy Loading 전략**이라고 부른다. 이 전략은 어떤 식으로 작동하는 지 하나씩 알아보자.

처음으로 게시판 서비스를 배포했다고 가정하고 Cache Aside 작동 방식을 이해해보자.

1. 처음 게시판 서비스를 배포했기 때문에 데이터베이스와 레디스에는 아무런 데이터도 저장이 안 되어 있다.
2. 일부 사용자가 들어와 게시글 작성을 함으로써 데이터를 저장한다. 이 데이터는 데이터베이스에 저장된다. (레디스에는 저장되지 않는다.)
3. 사용자가 데이터를 조회하려고 요청한다. 이 때, 데이터베이스로부터 바로 데이터 조회를 하기 전에 레디스에 있는 지 먼저 확인한다.
4. 레디스에 데이터가 없는 걸 확인한 뒤에 데이터베이스로부터 데이터를 조회해서 응답한다.
5. 데이터베이스로부터 조회한 데이터를 응답한 뒤에 레디스에도 데이터를 저장해둔다.
6. 다시 한 번 사용자가 데이터를 조회하려고 요청한다.
7. 레디스에 조회하고자 하는 데이터가 있는 지 확인했더니, 데이터가 존재해서 레디스로부터 데이터를 바로 가져와버린다.

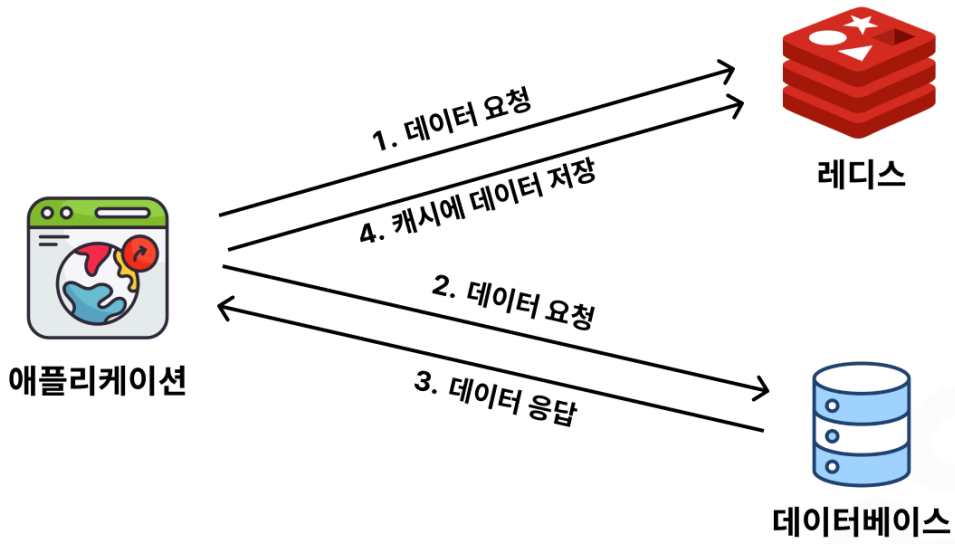
이게 Cache Aside 전략을 적용시킨 형태이다. 실제 예시에서 흐름을 봤으니 기억하기 좋게 다시 한 번 정리해보자.

1. **캐시에 데이터가 있을 경우 (= Cache Hit)**



데이터를 요청했을 때 캐시에 데이터가 있는 경우를 보고 **Cache Hit**이라고 한다.

2. **캐시에 데이터가 없을 경우 (= Cache Miss)**



데이터를 요청했을 때 캐시에 데이터가 없는 경우를 보고 **Cache Miss**라고 한다.

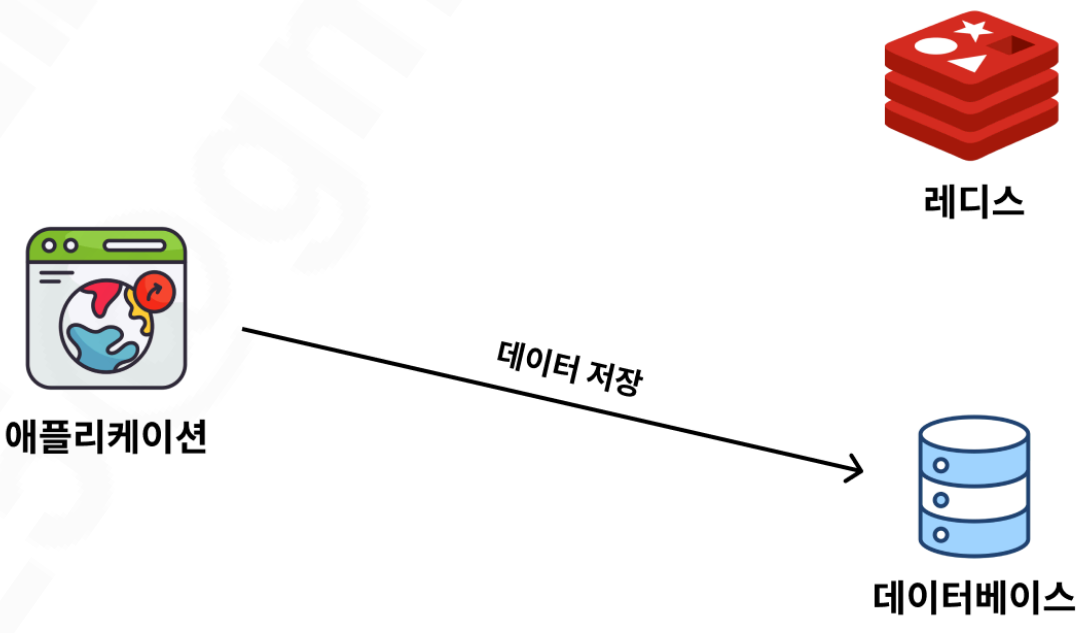
이번에는 한 문장으로 요약해보자.

> **Cache Aside 전략**은 **캐시(Cache)에서 데이터를 확인하고, 없다면 DB를 통해 조회해오는 방식**이다.

(전략 이름을 외우는 게 중요한 게 아니다. 과정을 기억하는 게 중요하다.)

✅ Write Around 전략

Cache Aside 전략이 데이터를 어떻게 조회할 지에 대한 전략이었다면, Write Around 전략은 데이터를 어떻게 쓸 지(저장, 수정, 삭제)에 대한 전략이다. Write Around 전략은 Cache Aside 전략과 같이 자주 활용되는 전략이다. Write Around 전략이 어떤 방식인지 '저장'의 예시로 알아보자.



Write Around 전략은 생각보다 너무 간단하다. 데이터를 저장할 때는 레디스에 저장하지 않고 데이터베이스에만 저장하는 방식이다. 그러다 데이터를 조회할 때 레디스에 데이터가 없으면 데이터베이스로부터 데이터를 조회해와서 레디스에 저장시켜주는 방식이다.

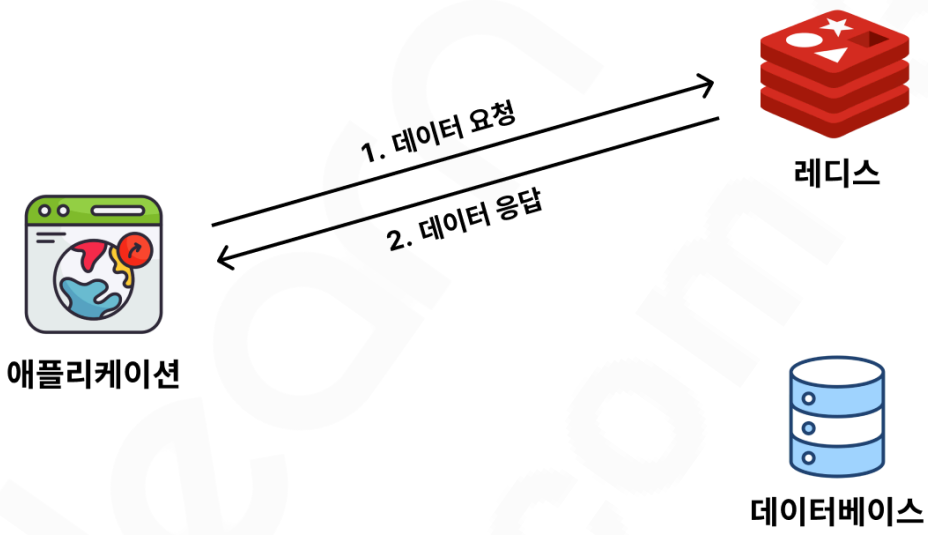
한 문장으로 요약해보자.

> **Write Around 전략**은 **쓰기 작업(저장, 수정, 삭제)을 캐시에는 반영하지 않고, DB에만 반영하는 방식**을 뜻한다.

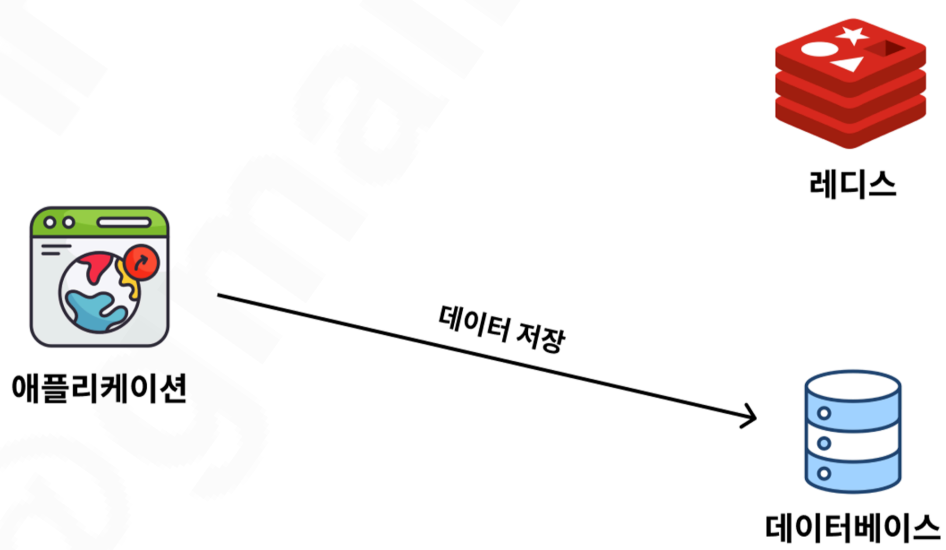
Cache Aside, Write Around 전략의 한계점 / 해결 방법

✅ Cache Aside, Write Around 전략의 한계점

1. **캐시된 데이터와 DB 데이터가 일치하지 않을 수 있다.**



Cache Aside 전략 (Cache Hit)



Write Around 전략

Cache Asdie와 Write Around 전략을 같이 썼을 때의 한계점 중 하나는 **캐시된 데이터와 DB 데이터가 일치하지 않을 수 있다**는 점이다. 조금 어렵게 표현하자면 **데이터의 일관성을 보장할 수 없다는 뜻**이다.

Write Around 전략에 따르면 데이터를 수정할 때 DB만 업데이트를 시키기 때문에 기존에 저장된 레디스의 데이터 값과 DB의 데이터 값은 다를 수 밖에 없다.

예를 들어, 분명 프로필 수정에서 내 이름을 '박재성'에서 '박선재'로 바꿨는데, 내 프로필 조회를 해봤더니 내 이름이 여전히 '박재성'이라고 표시된다는 뜻이다.

2. **캐시에 저장할 수 있는 공간이 비교적 작다.**

   DB는 디스크(Disk)에 저장해서 많은 양을 저장하기 용이하지만, 캐시는 메모리(RAM)에 저장하기 때문에 DB에 비해 많은 양의 데이터를 저장할 수가 없다.

✅ 이 한계를 어떻게 극복할까?

1. **캐시된 데이터와 DB 데이터가 일치하지 않을 수 있다.**

   캐시와 DB의 데이터를 일치시키기 위해, 데이터를 수정할 때마다 동시에 업데이트 시키면 성능적으로 느려진다. 그렇다고 성능 향상을 위해 DB의 데이터만 업데이트 시키면 캐시와 DB의 데이터가 일치하지 않게 된다.

   하지만 어쩔 수 없다. 어떤 선택을 하든 기회 비용(Trade Off)이 발생한다. 무언가를 얻으면 무언가를 포기해야 한다. 대부분의 개발 기술들이 장점이 있으면 단점이 있다. 따라서 데이터 조회 성능 개선 목적으로 레디스를 쓰는 경우에는 데이터의 일관성을 포기하고 성능 향상을 택한 것이다.

   이러한 이유로 인해 캐시를 적용시키기에 적절한 데이터는 다음과 같다.

   - 자주 조회되는 데이터
   - 잘 변하지 않는 데이터
   - 실시간으로 정확하게 일치하지 않아도 되는 데이터

   하지만 장기간 데이터가 일치하지 않는 건 문제가 될 수 있다. 따라서 적절한 주기로 데이터를 동기화시켜주어야 한다. 이 때 활용하는 기능이 레디스의 **TTL 기능(만료 시간 설정 기능)**이다.

   일정 시간이 지나면 데이터가 캐시에서 삭제된다. 그럼 특정 사용자가 조회를 하는 순간 Cache Miss가 발생한다. DB의 데이터를 새로 조회해와서 캐시에 데이터를 넣게 된다. 즉, 데이터가 새롭게 갱신되는 효과가 있는 것이다.

2. **캐시에 저장할 수 있는 공간이 비교적 작다.**

   위에서 활용했던 **TTL 기능(만료 시간 설정 기능)**을 활용하면 캐시의 공간을 효율적으로 쓸 수 있다. 왜냐면 자주 조회하지 않는 데이터는 만료 시간에 의해 데이터가 삭제되기 때문이다.

✅ 요약

Cache Aside, Write Around 전략을 사용할 때 주로 TTL을 같이 활용한다.

khb302500@gmail.com

📖

캐싱으로 조회 성능 개선을 하기 전 OOO을 항상 먼저해야 한다!

✅ 데이터 조회 성능을 개선하는 방법

데이터 조회 성능을 개선하는 방법은 다양하다.
(아래 용어가 뭔지 몰라도 상관없다. '여러가지 방식으로 데이터 조회 성능을 개선할 수 있구나' 정도로 알고 넘어가자.)

- SQL 튜닝
- 캐싱 서버 활용 (Redis 등)
- 레플리케이션 (Master/Slave 구조)
- 샤딩
- DB 스케일업 (CPU, Memory, SSD 등 하드웨어 업그레이드)

✅ 많은 성능 개선 방법 중 'SQL 튜닝'을 왜 먼저 고려해야 할까?

1. SQL 튜닝을 제외한 나머지 방법은 추가적인 시스템을 구축해야 한다. 따라서 **금전적, 시간적 비용이 추가적으로 발생**한다. 조금 더 복잡해진 시스템 구조로 인해 **관리 비용이 늘어난다.** 그에 비해 **SQL 튜닝은 기존의 시스템 변경 없이 성능을 개선할 수 있다.**

2. **근본적인 문제를 해결하는 방법**이 SQL 튜닝일 가능성이 높다. **SQL 자체가 비효율적으로 작성됐다면 아무리 시스템적으로 성능을 개선한다고 하더라도 한계가 있다.** 하지만 SQL 튜닝을 통해 기본적으로 성능을 향상시킨다면, 시스템적인 성능 개선이 필요없거나 훨씬 간단한 개선으로 큰 성능 개선 효과를 얻을 수 있다.

요약하자면, DB 성능 개선 방법들 중 **가장 가성비가 좋은 방법이 SQL 튜닝**인 것이다.

✅ 추천

혹시나 **SQL 튜닝**에 대한 기본기를 다지고 싶다면 아래 강의를 추천한다.

비전공자도 이해할 수 있는 MySQL 성능 최적화 입문/실전 (SQL 튜닝편) 강의 | JSCODE 박재성 - 인프런
JSCODE 박재성 | 비전공자 입장에서도 쉽게 이해할 수 있고, 실전에서 바로 적용 가능한 'MySQL 성능 최적화 입문/실전 (SQL 튜닝편)' 강의를 만들어봤습니다!, 😡 에라이, 못 해먹겠네!비전공자로 개발을 시작해 여러 회사에서 CTO로 활동하다가, 현재는 교육
https://inf.run/7QHc5



kb3025

📖 기본적인 Spring Boot 프로젝트 셋팅하기

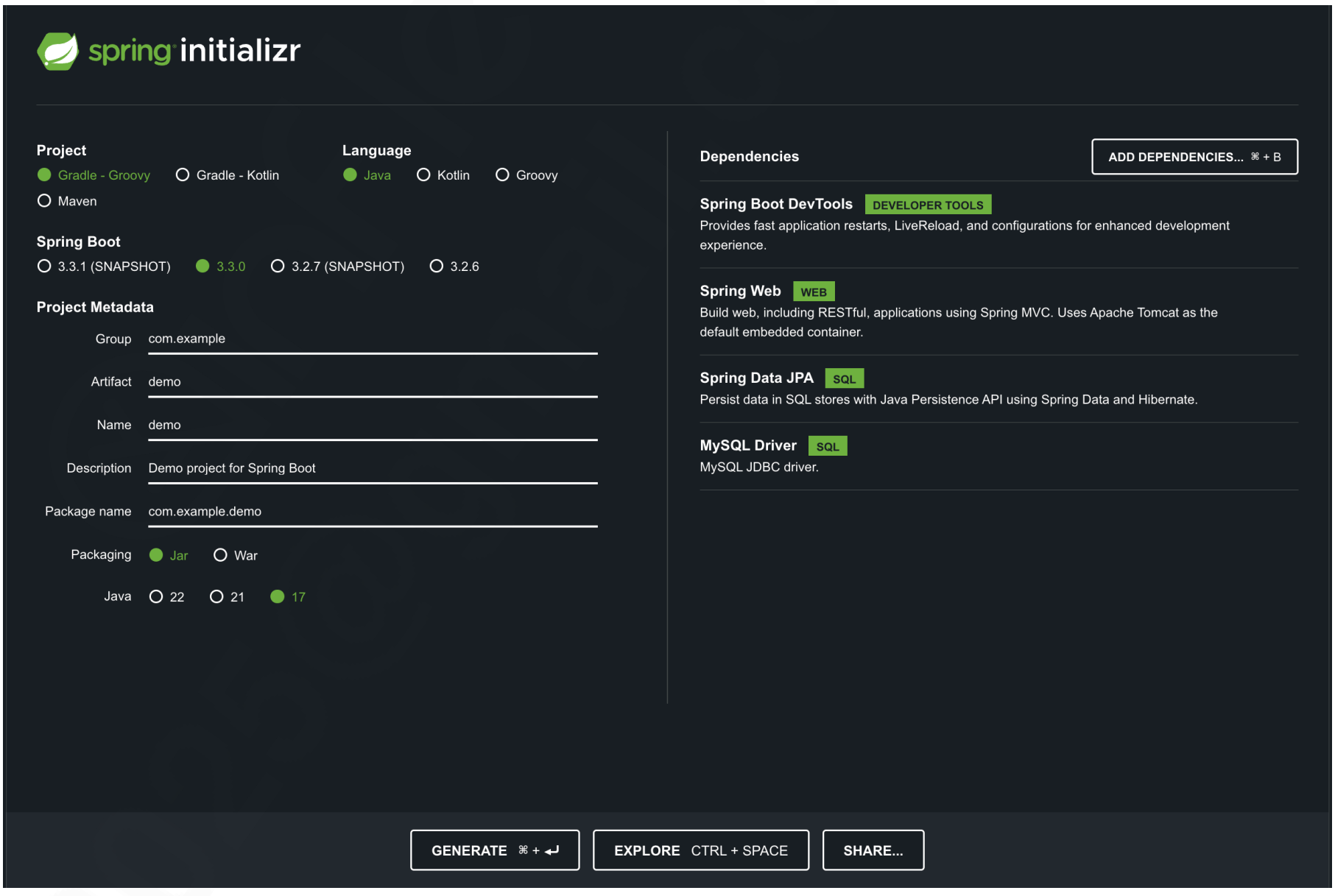
✅ 기본 Spring Boot 프로젝트 셋팅하기

> 동일한 환경에서 실습을 진행하기 위해 아래 버전을 사용할 것을 권장한다.
> - Spring Boot는 3.x.x 버전
> - MySQL 8.x 버전
> - JDK 17

1. **Spring Boot 프로젝트 셋팅**

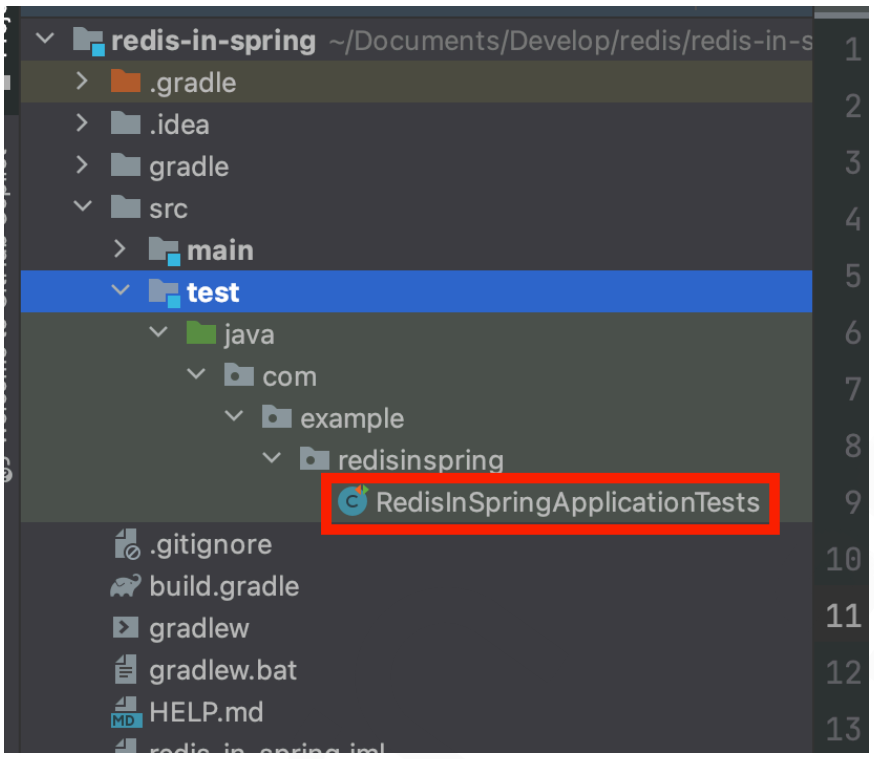
start.spring.io

https://start.spring.io/



- Java 17 버전을 선택하자. 아래 과정을 Java 17 버전을 기준으로 진행할 예정이다.
- Dependencies는 `Spring Boot DevTools`, `Spring Web`, `Spring Data JPA`, `MySQL Driver`를 선택해라.

2. **불필요한 테스트 코드 삭제**



3. **application.yml에 DB 연결을 위한 정보 작성하기**

> 이 예제에서는 기존에 있는 `application.properties`를 지우고 `application.yml`을 생성했다.

application.yml

```
# local 환경
spring:
  profiles:
    default: local
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

3. **Board 엔티티 만들기**

Board

```
@Entity
@Table(name = "boards")
public class Board {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String title;

  private String content;

  @CreatedDate
  @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")
  @JsonSerialize(using = LocalDateTimeSerializer.class)
  @JsonDeserialize(using = LocalDateTimeDeserializer.class)
```

```
  private LocalDateTime createdAt;

  public Long getId() {
    return id;
  }

  public String getTitle() {
    return title;
  }

  public String getContent() {
    return content;
  }

  public LocalDateTime getCreatedAt() {
    return createdAt;
  }
}
```

4. **기본 Controller, Service, Repository 만들기**

   BoardController

```
@RestController
@RequestMapping("boards")
public class BoardController {

  private BoardService boardService;

  public BoardController(BoardService boardService) {
    this.boardService = boardService;
  }

  @GetMapping()
  public List<Board> getBoards(
      @RequestParam(defaultValue = "1") int page, @RequestParam(defaultValue = "10") int size
  ) {
    return boardService.getBoards(page, size);
  }

}
```

BoardService

```
@Service
public class BoardService {
  private BoardRepository boardRepository;

  public BoardService(BoardRepository boardRepository) {
    this.boardRepository = boardRepository;
  }

  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
```

```
  }
 }
```

BoardRepository

```
public interface BoardRepository extends JpaRepository<Board, Long> {
  Page<Board> findAllByOrderByCreatedAtDesc(Pageable pageable);
}
```

✅ 더미 데이터 넣기

```
-- 높은 재귀(반복) 횟수를 허용하도록 설정
-- (아래에서 생성할 더미 데이터의 개수와 맞춰서 작성하면 된다.)
SET SESSION cte_max_recursion_depth = 1000000;

-- boards 테이블에 더미 데이터 삽입
INSERT INTO boards (title, content, created_at)
WITH RECURSIVE cte (n) AS
(
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM cte WHERE n < 1000000 -- 생성하고 싶은 더미 데이터의 개수
)
SELECT
    CONCAT('Title', LPAD(n, 7, '0')) AS title,  -- 'Title' 다음에 7자리 숫자로 구성된 제목 생성
    CONCAT('Content', LPAD(n, 7, '0')) AS content,  -- 'Content' 다음에 7자리 숫자로 구성된 내용 생성
    TIMESTAMP(DATE_SUB(NOW(), INTERVAL FLOOR(RAND() * 3650 + 1) DAY) + INTERVAL FLOOR(RAND() * 86400) SEC
FROM cte;
```

- 위 SQL문은 MySQL 8.0부터 사용이 가능하다.

khb3025@gmail.com

📖

Spring Boot 프로젝트에 Redis 셋팅 추가하기

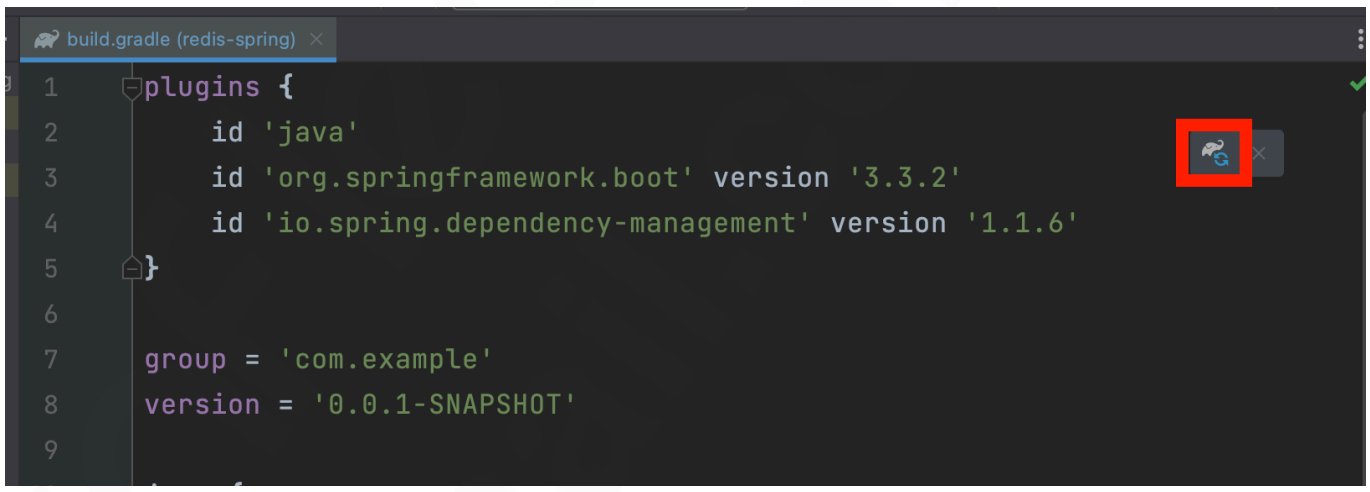
✅ Spring Boot 프로젝트에 Redis 셋팅 추가하기

1. **Redis 의존성 추가하기**

   build.gradle

```
...

dependencies {
  ...
  implementation 'org.springframework.boot:spring-boot-starter-data-redis'
}
```



IDE : IntelliJ

의존성을 새로 추가해준 뒤에 반드시 위 버튼을 눌러주어야 반영이 된다.

2. **application.yml 수정하기**

   application.yml

```
# local 환경
spring:
  profiles:
    default: local
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  data:
    redis:
      host: localhost
      port: 6379

logging:
```

```
  level:
    org.springframework.cache: trace # Redis 사용에 대한 로그가 조회되도록 설정
```

3. **Redis 설정 추가하기**

   config/RedisConfig

```
@Configuration
public class RedisConfig {
  @Value("${spring.data.redis.host}")
  private String host;

  @Value("${spring.data.redis.port}")
  private int port;

  @Bean
  public LettuceConnectionFactory redisConnectionFactory() {
    // Lettuce라는 라이브러리를 활용해 Redis 연결을 관리하는 객체를 생성하고
    // Redis 서버에 대한 정보(host, port)를 설정한다.
    return new LettuceConnectionFactory(new RedisStandaloneConfiguration(host, port));
  }
}
```

config/RedisCacheConfig

```
@Configuration
@EnableCaching // Spring Boot의 캐싱 설정을 활성화
public class RedisCacheConfig {
  @Bean
  public CacheManager boardCacheManager(RedisConnectionFactory redisConnectionFactory) {
    RedisCacheConfiguration redisCacheConfiguration = RedisCacheConfiguration
        .defaultCacheConfig()
        // Redis에 Key를 저장할 때 String으로 직렬화(변환)해서 저장
        .serializeKeysWith(
            RedisSerializationContext.SerializationPair.fromSerializer(
                new StringRedisSerializer()))
        // Redis에 Value를 저장할 때 Json으로 직렬화(변환)해서 저장
        .serializeValuesWith(
            RedisSerializationContext.SerializationPair.fromSerializer(
                new Jackson2JsonRedisSerializer<Object>(Object.class)
            )
        )
        // 데이터의 만료기간(TTL) 설정
        .entryTtl(Duration.ofMinutes(1L));

    return RedisCacheManager
        .RedisCacheManagerBuilder
        .fromConnectionFactory(redisConnectionFactory)
        .cacheDefaults(redisCacheConfiguration)
        .build();
  }
}
```

4. **BoardService에 캐싱 로직 추가하기**

   BoardService

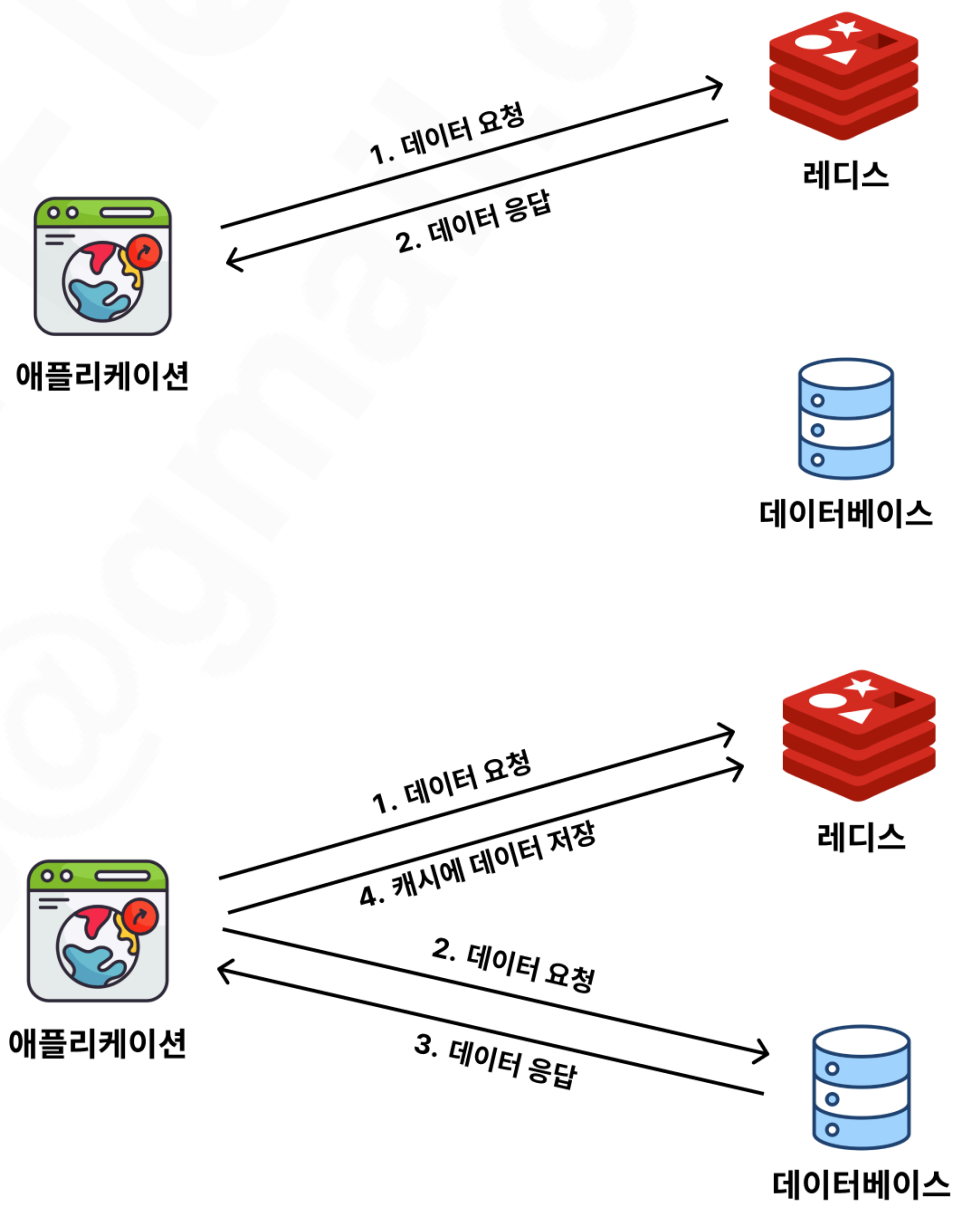
```
@Service
public class BoardService {

  private BoardRepository boardRepository;
  private Pageable pageable;

  public BoardService(BoardRepository boardRepository) {
    this.boardRepository = boardRepository;
  }

  @Cacheable(cacheNames = "getBoards", key = "'boards:page:' + #page + ':size:' + #size", cacheManager = "board
  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
  }
}
```

`@Cacheable` 어노테이션을 붙이면 Cache Aside 전략으로 캐싱이 적용된다. 즉, 해당 메서드로 요청이 들어오면 레디스를 확인한 후에 데이터가 있다면 레디스의 데이터를 조회해서 바로 응답한다. 만약 데이터가 없다면 메서드 내부의 로직을 실행시킨 뒤에 return 값으로 응답한다. 그리고 그 return 값을 레디스에 저장한다.



(복습 : 📖 데이터를 캐싱할 때 사용하는 전략 (Cache Aside, Write Around))

[속성 값 설명]

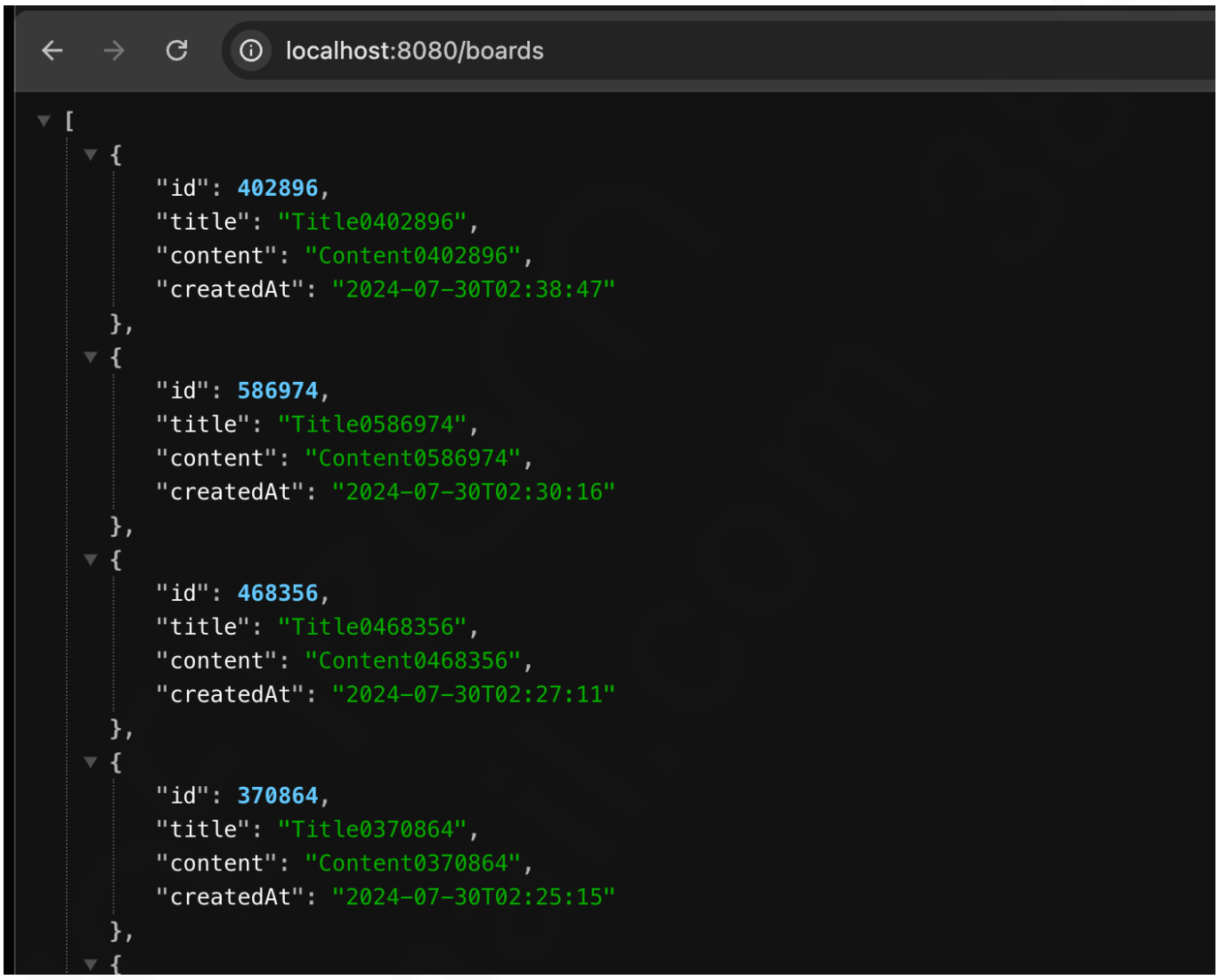
- `cacheNames` : 캐시 이름을 설정
- `key` : Redis에 저장할 Key의 이름을 설정

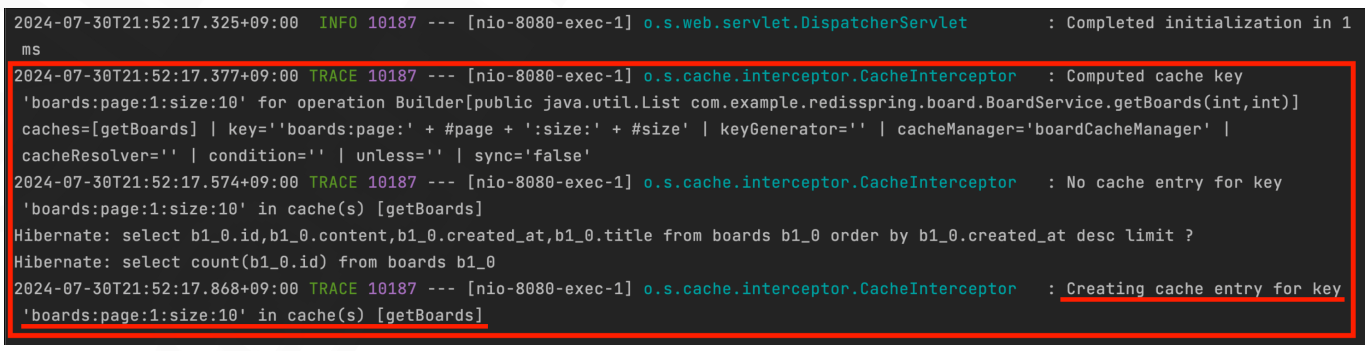
  📖 Redis에서 Key 네이밍 컨벤션 익히기

- `cacheManager` : 사용할 `cacheManager`의 Bean 이름을 지정

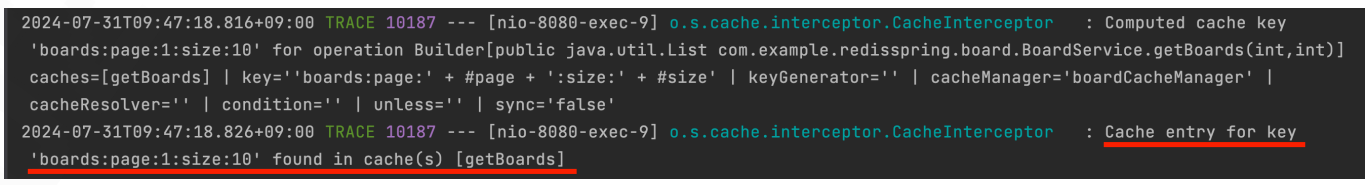
✅ 테스트 해보기

1. **Spring Boot 서버를 실행시켜서 API 실행시켜보기**





Cache가 존재하지 않아서 DB로부터 데이터를 조회한 뒤 Cache를 생성했다고 로그가 찍혀있다.



한 번 더 새로고침을 해보면 Cache가 생성(Creating)되지 않고 기존 Cache를 조회해왔음을 알 수 있다.

2. **Redis-cli를 활용해 정상적으로 캐싱이 됐는 지 확인하기**

```
$ redis-cli

$ keys * # Redis에 저장되어 있는 모든 key 조회
$ get getBoards::boards:page:1:size:10 # 특정 key의 Value 조회
$ ttl getBoards::boards:page:1:size:10 # 특정 key의 TTL 조회
```

Redis를 적용하기 전후 성능 비교해보기 (Postman)

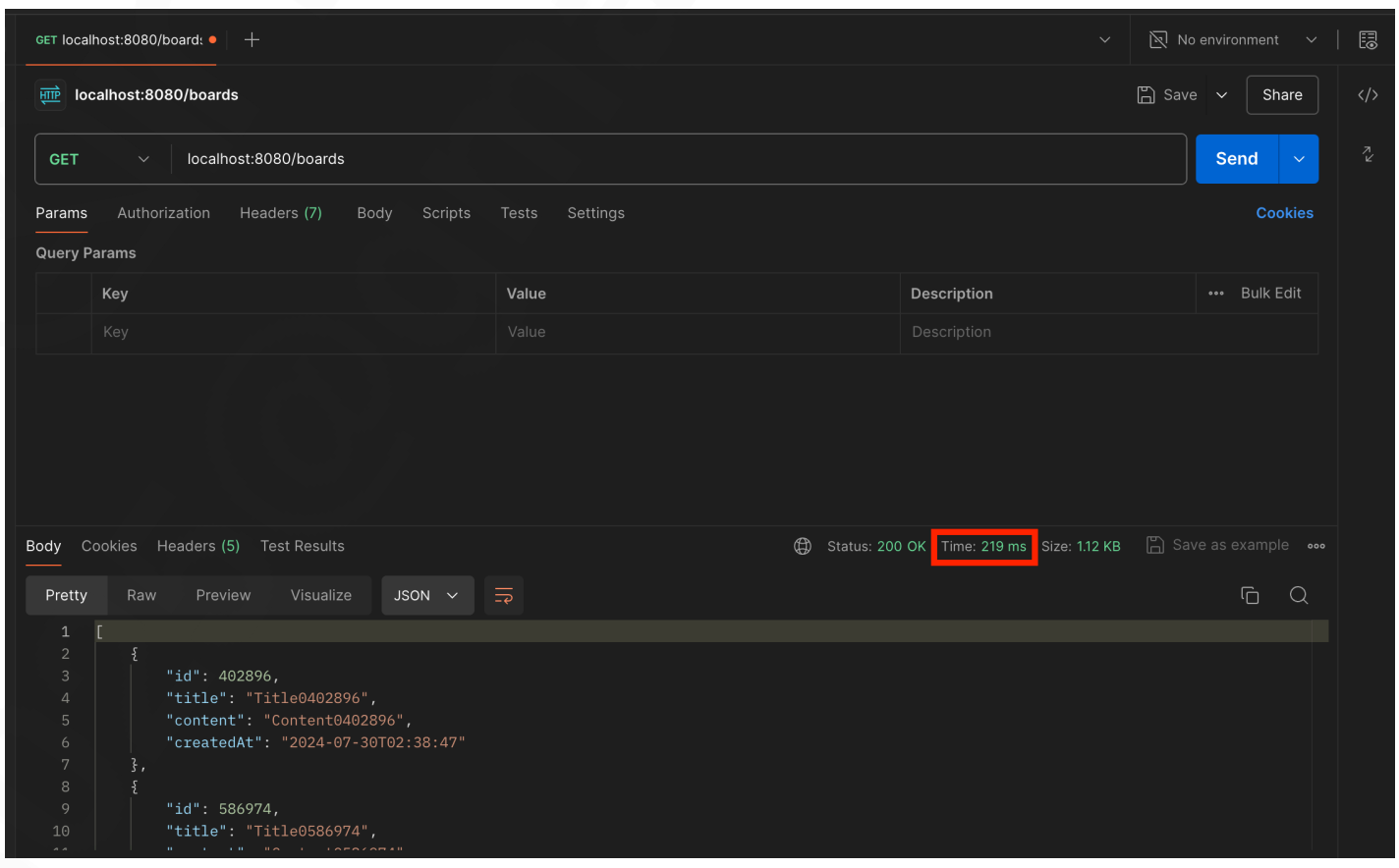
👩🏻 성능 개선을 할 땐 반드시 **수치를 측정하면서 비교**해야 한다. 성능 개선을 할 때 '느린 것 같아요', '빨라진 것 같아요'라고 얘기하는 건 잘못된 습관이다. 성능 개선을 할 때는 실제로 얼마나 느린지를 측정해야 하고, 개선을 한 뒤에 얼마나 빨라졌는 지를 측정해야 한다. 즉, 정확한 수치로 얘기할 수 있어야 한다.

✅ Redis 적용 전

BoardService

```
@Service
public class BoardService {
  ...
  //  @Cacheable(cacheNames = "getBoards", key = "'boards:page:' + #page + ':size:' + #size", cacheManager = "boardC
  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
  }
}
```

캐싱을 적용시키는 어노테이션을 주석 처리한 뒤 Postman으로 테스트를 해보자.



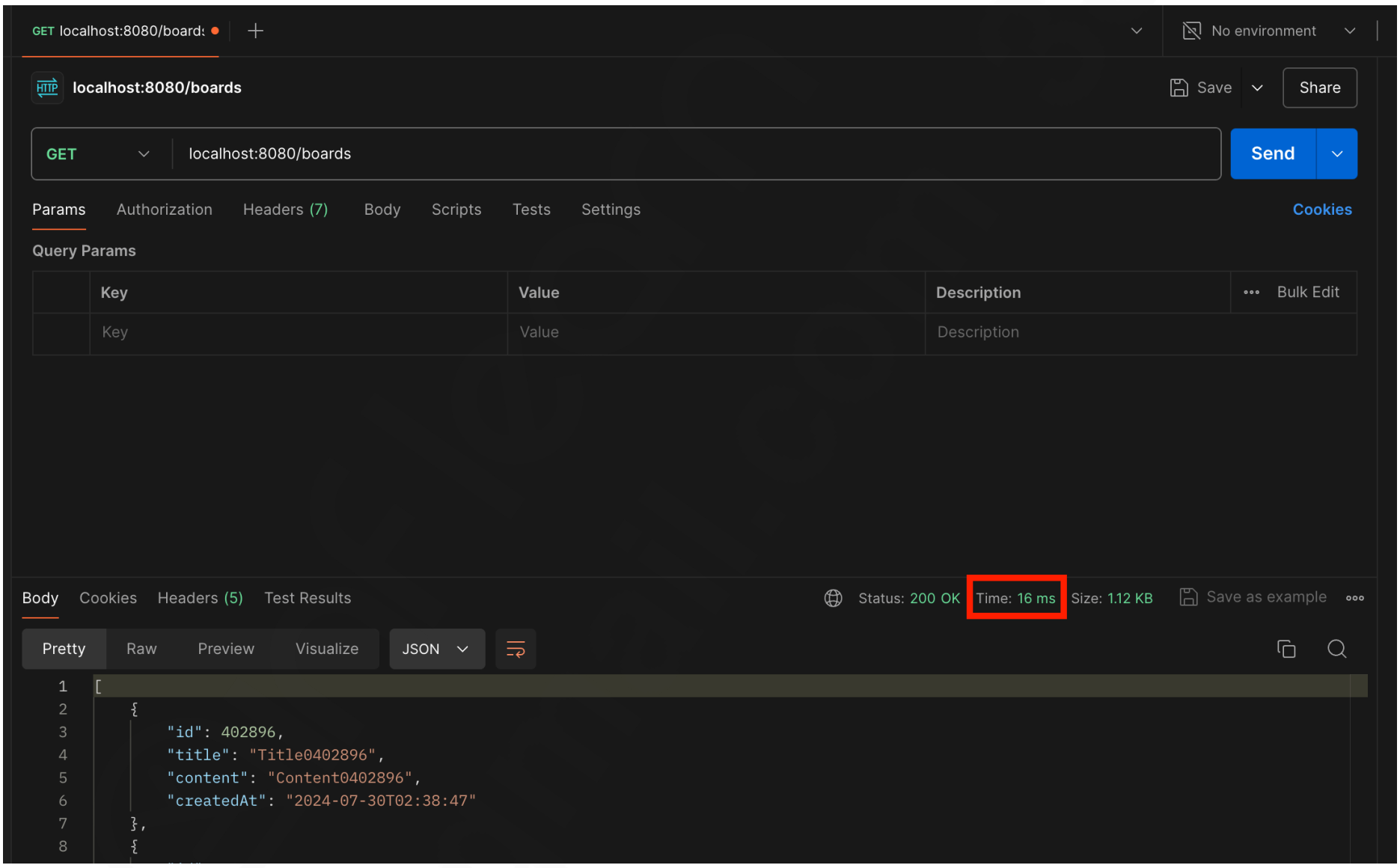
여러번 요청을 보내보니 평균적으로 200ms 정도의 속도가 나오는 걸 확인할 수 있다.
(개인이 가지고 있는 컴퓨터의 성능에 따라 수치는 다를 수 있다.)

✅ Redis 적용 후

```
@Service
public class BoardService {
  ...
```

```
  @Cacheable(cacheNames = "getBoards", key = "'boards:page:' + #page + ':size:' + #size", cacheManager = "boardCac
  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
  }
}
```

주석 처리를 해제한 뒤에 다시 한 번 테스트를 해보자.



여러번 요청을 보내보니 평균적으로 20ms 정도의 속도가 나오는 걸 확인할 수 있다. 10배 정도 속도가 향상됐다.

khb302

📖

기본적인 Nest.js 프로젝트 셋팅하기

❗ Nest.js를 주로 사용하는 수강생이더라도 **이전 섹션(로컬 환경에서 Spring Boot + Redis로 구현하기)**의 강의를 듣고오는 걸 추천한다.

✅ 기본적인 Nest.js 프로젝트 셋팅하기

1. **Nest.js 프로젝트 생성**

```
# nest new {프로젝트명}
$ nest new instagram-server
```

2. **TypeORM 설치하기**

```
$ npm i @nestjs/typeorm typeorm mysql2
```

3. **DB 정보 셋팅하기**

app.module.ts

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'password',
      database: 'mydb',
      autoLoadEntities: true,
      synchronize: true,
    }),
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

4. **Board 엔티티 만들기**

board.etnity.ts

```
@Entity('boards')
export class Board {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;
```

```
  @Column()
  content: string;

  @CreateDateColumn()
  createdAt: Date;
}
```

5. **기본 Controller, Service, Reposiotry 만들기**

   board.controller.ts

```
@Controller()
export class BoardController {
  constructor(private readonly boardService: BoardService) {}

  @Get('boards')
  getBoards(@Query('page') page: number = 1, @Query('size') size: number = 10) {
    return this.boardService.getBoards(page, size);
  }
}
```

board.service.ts

```
@Injectable()
export class BoardService {
  constructor(
    @InjectRepository(Board)
    private boardRepository: Repository<Board>,
  ) {}

  async getBoards(page: number, size: number): Promise<Board[]> {
    const skip = (page - 1) * size;
    const boards = await this.boardRepository.find({
      order: { createdAt: 'DESC' },
      skip: skip,
      take: size,
    });
    return boards;
  }
}
```

app.module.ts

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'password',
      database: 'mydb',
      autoLoadEntities: true,
      synchronize: true,
      logging: true,
    }),
```

```
    TypeOrmModule.forFeature([Board]),
  ],
  controllers: [BoardController],
  providers: [BoardService],
})
export class AppModule {}
```

✅ 더미 데이터 넣기

```
-- 기존 테이블 데이터 삭제
TRUNCATE TABLE boards;

-- 높은 재귀(반복) 횟수를 허용하도록 설정
-- (아래에서 생성할 더미 데이터의 개수와 맞춰서 작성하면 된다.)
SET SESSION cte_max_recursion_depth = 1000000;

-- boards 테이블에 더미 데이터 삽입
INSERT INTO boards (title, content, createdAt)
WITH RECURSIVE cte (n) AS
(
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM cte WHERE n < 1000000 -- 생성하고 싶은 더미 데이터의 개수
)
SELECT
    CONCAT('Title', LPAD(n, 7, '0')) AS title,  -- 'Title' 다음에 7자리 숫자로 구성된 제목 생성
    CONCAT('Content', LPAD(n, 7, '0')) AS content,  -- 'Content' 다음에 7자리 숫자로 구성된 내용 생성
    TIMESTAMP(DATE_SUB(NOW(), INTERVAL FLOOR(RAND() * 3650 + 1) DAY) + INTERVAL FLOOR(RAND() * 86400) SEC
FROM cte;
```

- 위 SQL문은 MySQL 8.0부터 사용이 가능하다.

knb3025@gmail.com

📖

Nest.js 프로젝트에 Redis 셋팅 추가하기

✅ Nest.js 프로젝트에 Redis 셋팅 추가하기

1. **라이브러리 설치하기**

```
$ npm i @nestjs/cache-manager cache-manager cache-manager-ioredis
$ npm i -D @types/cache-manager @types/cache-manager-ioredis
```

2. **AppModule 코드 수정하기**

   app.module.ts

```
import { BoardController } from './board.controller';
import { BoardService } from './board.service';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Board } from './board.entity';
import { CacheModule } from '@nestjs/cache-manager';
import * as redisStore from 'cache-manager-ioredis';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: 'password',
      database: 'mydb',
      autoLoadEntities: true,
      synchronize: true,
      logging: true,
    }),
    TypeOrmModule.forFeature([Board]),
    CacheModule.register({
      store: redisStore,
      host: 'localhost',
      port: 6379,
      ttl: 60,
    }),
  ],
  controllers: [BoardController],
  providers: [BoardService],
})
export class AppModule {}
```

3. **BoardService 코드 수정하기**

   board.service.ts

```
import { Inject, Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
```

```
import { Board } from './board.entity';
import { Repository } from 'typeorm';
import { CACHE_MANAGER, Cache } from '@nestjs/cache-manager';

@Injectable()
export class BoardService {
  constructor(
    @InjectRepository(Board)
    private boardRepository: Repository<Board>,
    @Inject(CACHE_MANAGER)
    private cacheManager: Cache,
  ) {}

  async getBoards(page: number, size: number): Promise<Board[]> {
    const cacheKey = `boards:page:${page}:size:${size}`;
    const cachedData = await this.cacheManager.get<Board[]>(cacheKey);
    if (cachedData) {
      return cachedData;
    }

    const skip = (page - 1) * size;
    const boards = await this.boardRepository.find({
      order: {
        createdAt: 'desc',
      },
      skip: skip,
      take: size,
    });

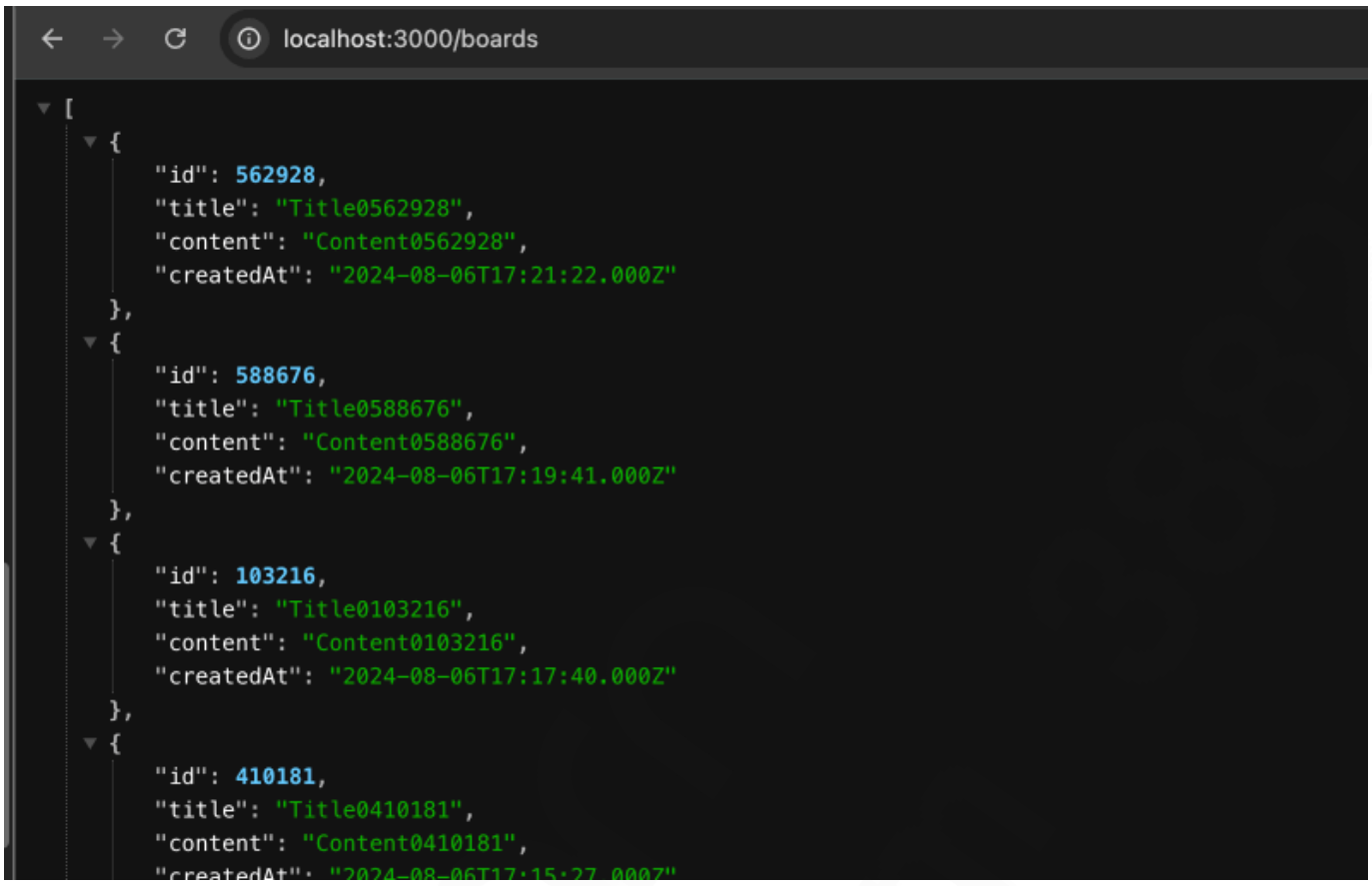
    await this.cacheManager.set(cacheKey, boards);

    return boards;
  }
}
```

참고) Spring에서는 어노테이션만 가지고 처리할 수 있는 부분이 굉장히 많지만 Nest.js는 한계점이 존재한다. 이 때문에 캐싱에 대한 처리를 일일이 로직으로 짜주어야 하는 경우가 많다.

✅ 테스트 해보기

1. **Nest.js 서버 실행시켜서 API 실행시켜보기**



2. **Redis-cli를 활용해 정상적으로 캐싱이 됐는 지 확인하기**

```
$ redis-cli

$ keys * # Redis에 저장되어 있는 모든 key 조회
$ get getBoards::boards:page:1:size:10 # 특정 key의 Value 조회
$ ttl getBoards::boards:page:1:size:10 # 특정 key의 TTL 조회
```

khb3025@gmail.com

📖

[보충 설명] Nest.js가 v11일 경우 Redis 셋팅 방법

✅ Nest.js 버전 확인 방법

Nest.js 프로젝트의 package.json 파일을 열어서 @nestjs/core가 몇 버전인지 확인하면 된다. 만약 아래와 같이 `11.1.2` 라고 기재되어 있다면 Nest.js는 11 버전인 것이다.

package.json

```
{
  ...
  "dependencies": {
    "@keyv/redis": "^4.4.0",
    "@nestjs/cache-manager": "^3.0.1",
    "@nestjs/common": "^11.1.2",
    "@nestjs/core": "^11.1.2",
    "@nestjs/platform-express": "^11.1.2",
    "@nestjs/typeorm": "^11.0.0",
    "cache-manager": "^6.4.3",
    "cache-manager-ioredis": "^2.1.0",
    "mysql2": "^3.14.1",
    "reflect-metadata": "^0.2.2",
    "rxjs": "^7.8.2",
    "typeorm": "^0.3.24"
  },
    ...
}
```

✅ Nest.js가 v11일 경우 Redis 셋팅 방법

1. **의존성 설치**

```
$ npm install @keyv/redis
```

2. **app.module.ts 셋팅 변경**

```
import { Module } from "@nestjs/common";
import { TypeOrmModule } from "@nestjs/typeorm";
import { Board } from "./board.entity";
import { BoardController } from "./board.controller";
import { BoardService } from "./board.service";
import { CacheModule } from "@nestjs/cache-manager";
import KeyvRedis from "@keyv/redis";

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mysql',
      host: 'localhost',
      port: 3306,
      username: 'root',
```

```
    password: 'password123',
    database: 'mydb',
    autoLoadEntities: true,
    synchronize: true,
  }),
  TypeOrmModule.forFeature([Board]),
  CacheModule.registerAsync({
    imports: undefined, useClass: undefined, useExisting: undefined,
    useFactory: async () => {
      return {
        stores: [
          new KeyvRedis('redis://localhost:6379'),
        ]
      }
    }
  }),
],
controllers: [BoardController],
  providers: [BoardService],
})
export class AppModule {}
```

3. **Service 로직 수정**

```
@Injectable()
export class BoardService {
  constructor(
    @InjectRepository(Board)
    private boardRepository: Repository<Board>,
    @Inject(CACHE_MANAGER)
    private cacheManager: Cache,
  ) {}

  async getBoards(page: number, size: number): Promise<Board[]> {
    const cacheKey = `boards:page:${page}:size:${size}`;
    const cachedData = await this.cacheManager.get<Board[]>(cacheKey);
    if (cachedData) {
      return cachedData;
    }

    const skip = (page - 1) * size;
    const boards = await this.boardRepository.find({
      order: { createdAt: 'DESC' },
      skip: skip,
      take: size,
    });

    // ttl 값 추가 (단위 : milliseconds)
    await this.cacheManager.set(cacheKey, boards, 60 * 1000);

    return boards;
  }
}
```

4. **Redis-cli를 활용해 정상적으로 캐싱이 됐는 지 확인하기**

```
$ redis-cli

$ keys * # Redis에 저장되어 있는 모든 key 조회
$ get boards:page:1:size:10 # 특정 key의 Value 조회
$ ttl boards:page:1:size:10 # 특정 key의 TTL 조회
```

kkb3025@gmail.com

📖

대략적으로 AWS 비용 얼마나 나오는 지

✅ EC2

- EC2 인스턴스 (t3a.small) : 시간당 0.026 USD (24시간당 약 800원)

  (비용이 걱정되시는 분들은 학습이 끝나자마자 인스턴스를 종료하시길 권장드립니다.)
- 데이터 전송 비용 : 1 GB당 0.1368 USD (1GB당 약 200원)

  (실습 과정 동안 1GB 이하의 데이터만 전송합니다.)
- Public IPv4 비용 : 시간당 0.005 USD (24시간당 약 200원)

✅ RDS

- RDS 인스턴스 (t4g.micro) : 시간당 0.026 USD (24시간당 약 800원)

  (프리티어일 경우 월 750시간까지 무료)
- 스토리지 비용 : GB-월당 0.131 USD (20GB-24시간당 약 200원)

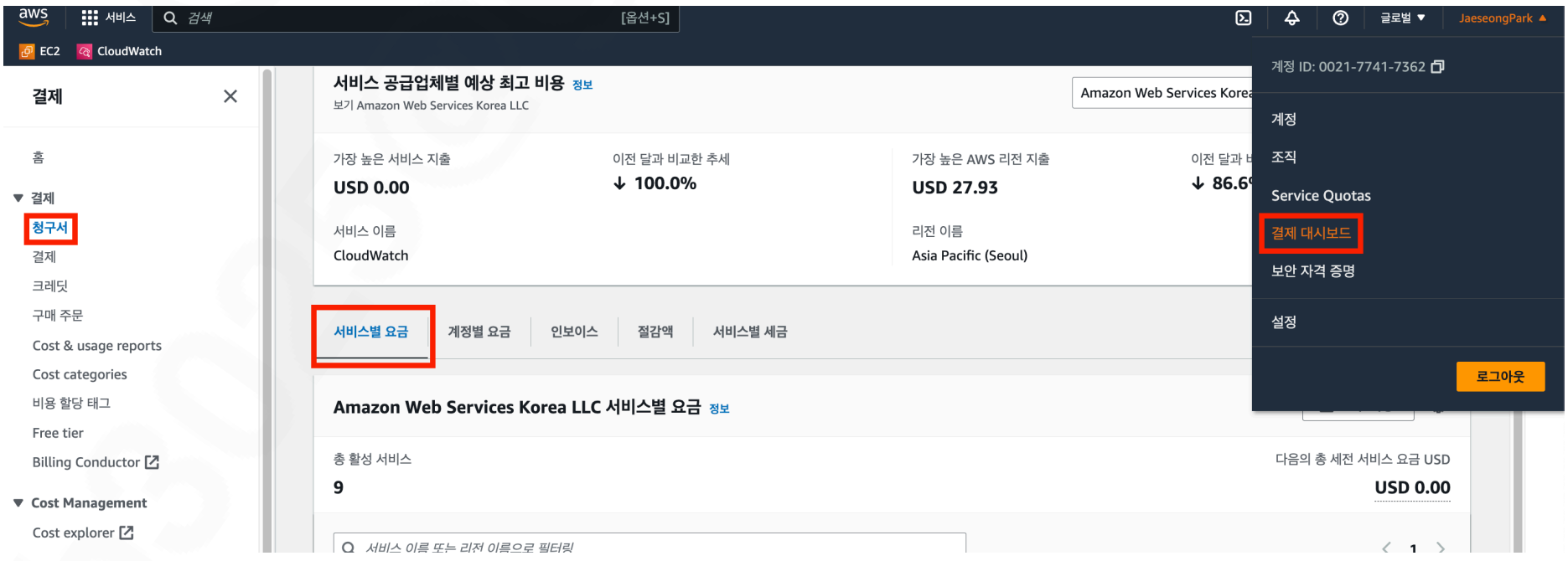
  (프리티어일 경우 20GB까지 무료)
- Public IPv4 비용 : 시간당 0.005 USD (24시간당 약 200원)

✅ ElasitCache

- 캐시 (cache.t3micro) : 시간당 0.025 USD (24시간당 약 800원)

  (프리티어일 경우 월 750시간까지 무료)

✅ 혹시나 비용이 많이 나가고 있는 건 아닌 지 체크하는 방법



Amazon Web Services Korea LLC 서비스별 요금 정보

모두 확장

총 활성 서비스

9

다음의 총 세전 서비스 요금 USD

USD 0.00

서비스 이름 또는 리전 이름으로 필터링

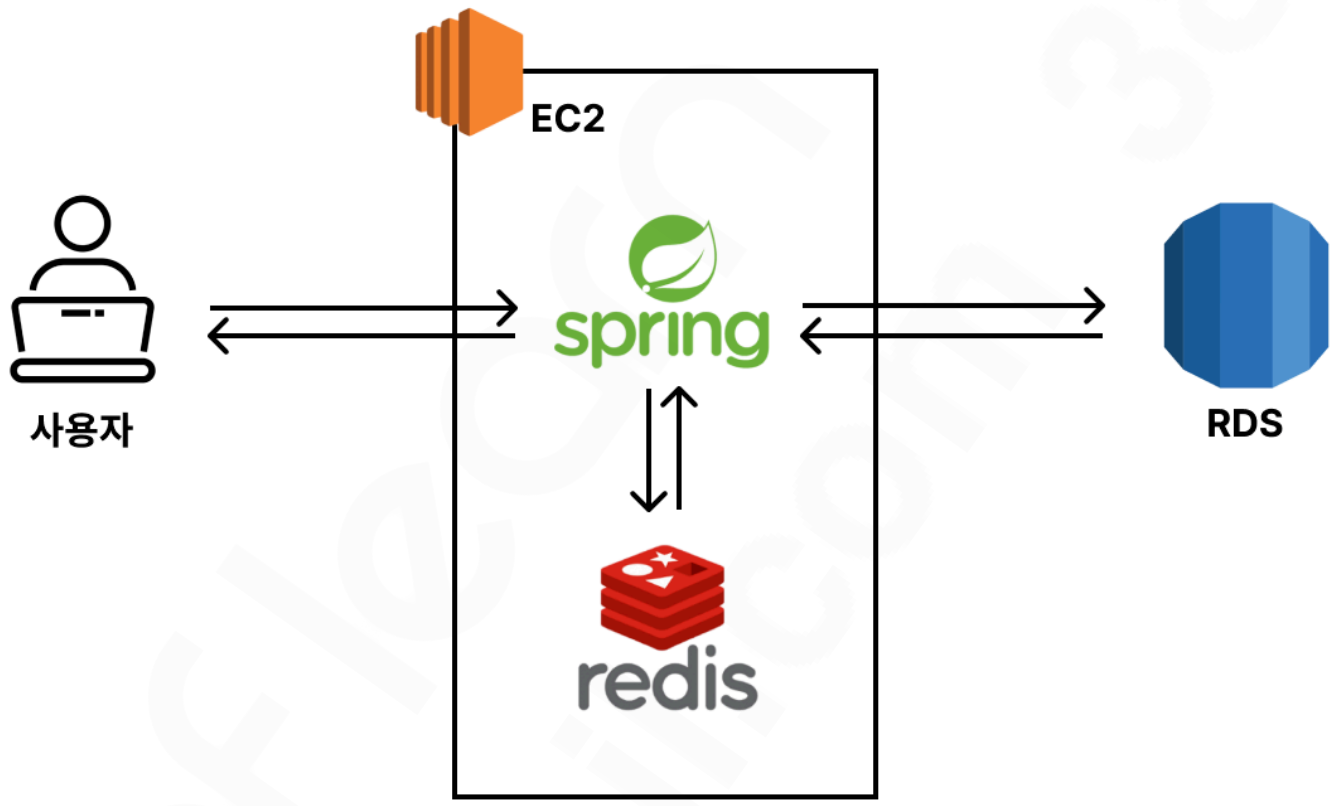
1

설명	사용량 수량	의 금액 USD
CloudWatch		USD 0.00
Data Transfer		USD 0.00
EC2 Container Registry (ECR)		USD 0.00
Elastic Compute Cloud		USD 0.00
Elastic Load Balancing		USD 0.00
Key Management Service		USD 0.00
Relational Database Service		USD 0.00
Simple Storage Service		USD 0.00
Virtual Private Cloud		USD 0.00
총 세금		**USD 0.00**

kb302500@gmail.com

EC2, RDS, Spring Boot, Redis를 활용한 아키텍처 구성

✅ 아키텍처 구성



kb302507@gmail.com

📖

EC2, RDS, Spring Boot, Redis 셋팅

👩🏻 AWS에 초점을 맞춘 강의가 아니기 때문에 EC2, RDS에 대한 디테일한 설명은 생략할 예정이다. 혹시 AWS에 대한 기본기를 다지고 싶다면 아래 강의를 추천한다.

비전공자도 이해할 수 있는 AWS 입문/실전 (https://inf.run/73afg)

✅ EC2 생성

[고려 사항]

- EC2 인스턴스를 생성할 때 **t3a.small 이상으로 생성할 것을 권장**한다. t2.micro에서 Spring Boot, Redis를 다 돌리기에는 성능이 모자라다. 성능이 모자라서 중간에 EC2가 멈춰버리기도 한다.
- 보안 그룹 설정에서 8080번 포트 열어주는 것을 깜빡하지 말자.

✅ RDS 생성

[고려 사항]

- 실습을 원활하게 하기 위해 RDS 생성할 때 초기 DB 이름을 `mydb` 라고 붙여서 생성하자.
- 보안 그룹 설정에서 3306번 포트 열어주는 것 깜빡하지 말자.

✅ EC2에 Redis 설치

1. **Redis 설치하기**

```
$ sudo apt update
$ sudo apt install redis
```

2. **Redis 잘 설치됐는 지 확인**

```
$ redis-cli

127.0.0.1:6379> ping
PONG
```

✅ EC2에 Spring Boot 프로젝트 셋팅하기

1. **JDK 설치하기**

Spring Boot는 3.x.x 버전을 사용할 예정이고, JDK는 17버전을 사용할 예정이다. 그에 맞게 환경을 설치해보자.

```
$ sudo apt install openjdk-17-jdk
```

2. **잘 설치됐는 지 확인하기**

```
$ java -version
```



3. **Spring Boot 프로젝트에서 application.yml 정보 수정하기**

   application.yml

```
# local 환경
spring:
  profiles:
    default: local
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  data:
    redis:
      host: localhost
      port: 6379

logging:
  level:
    org.springframework.cache: trace

---
# prod 환경
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: jdbc:mysql://{rds 주소}:3306/mydb
    username: admin
    password: password
```

4. **Github Repository에 올리기**

   참고) 실습에서는 편의를 위해 `application.yml` 정보를 `.gitignore` 처리를 하지 않았다. 실제 프로젝트에서는 보안을 위해 `application.yml` 정보를 `.gitignore` 처리를 해서 버전 관리가 되지 않게 만들어야 한다.

5. **EC2로부터 Github Clone 받기**

```
$ git clone {Github Repository 주소}
$ cd {프로젝트 경로}
```

4. **서버 실행시키기**

```
# 스프링 프로젝트 경로로 들어가서 아래 명령어 실행
$ ./gradlew clean build -x test
$ cd build/libs
$ java -jar -Dspring.profiles.active=prod {빌드된 jar 파일명}
```

정상적으로 서버가 실행되는 걸 확인할 수 있다. 그리고 JPA의 ddl 옵션으로 인해 테이블도 생성이 된다.

✅RDS에 더미데이터 넣기

1. **DB GUI 툴로 RDS에 접속하기**

   (실습에서는 Datagrip을 활용)

2. **더미 데이터 넣어주는 SQL문 실행시키기**

```
-- 높은 재귀(반복) 횟수를 허용하도록 설정
-- (아래에서 생성할 더미 데이터의 개수와 맞춰서 작성하면 된다.)
SET SESSION cte_max_recursion_depth = 1000000;

-- boards 테이블에 더미 데이터 삽입
INSERT INTO boards (title, content, created_at)
WITH RECURSIVE cte (n) AS
(
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM cte WHERE n < 1000000 -- 생성하고 싶은 더미 데이터의 개수
)
SELECT
    CONCAT('Title', LPAD(n, 7, '0')) AS title,  -- 'Title' 다음에 7자리 숫자로 구성된 제목 생성
    CONCAT('Content', LPAD(n, 7, '0')) AS content,  -- 'Content' 다음에 7자리 숫자로 구성된 내용 생성
    TIMESTAMP(DATE_SUB(NOW(), INTERVAL FLOOR(RAND() * 3650 + 1) DAY) + INTERVAL FLOOR(RAND() * 86400) S
FROM cte;
```

- 위 SQL문은 MySQL 8.0부터 사용이 가능하다.

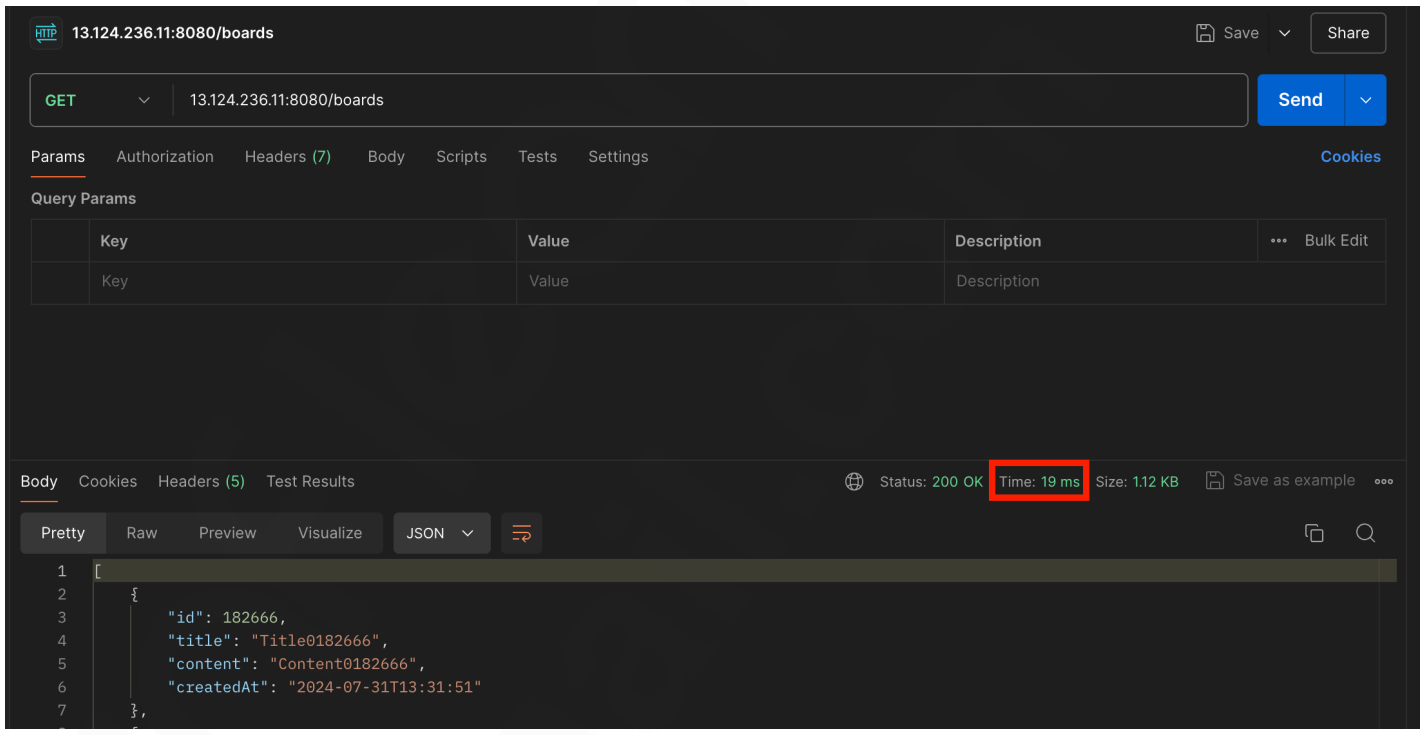
khb302500@gmail.com

📖

Redis를 적용하기 전후 성능 비교해보기 (Postman)

✅ Redis를 적용했을 때의 성능 측정

```
# 스프링 프로젝트 경로로 들어가서 아래 명령어 실행
$ ./gradlew clean build -x test
$ cd build/libs
$ java -jar -Dspring.profiles.active=prod {빌드된 jar 파일명}
```



여러 번 요청을 보내보니 평균적으로 20ms 정도의 속도가 나오는 걸 확인할 수 있다.

✅ Redis를 적용하지 않았을 때의 성능 측정

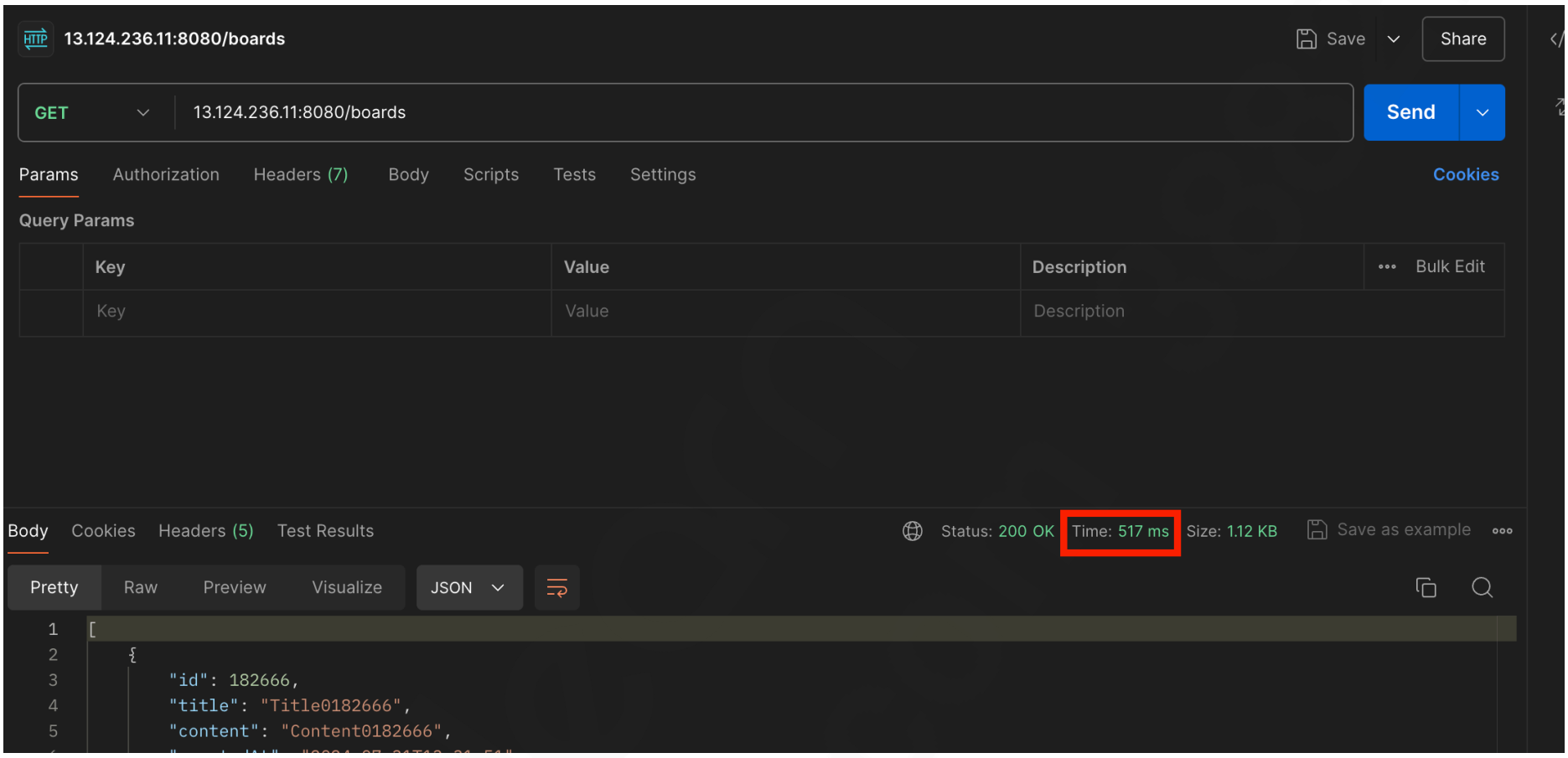
BoardService

```
@Service
public class BoardService {
  ...
  //  @Cacheable(cacheNames = "getBoards", key = "'boards:page:' + #page + ':size:' + #size", cacheManager = "boardC
  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
  }
}
```

캐싱을 적용시키는 어노테이션을 주석 처리한 뒤 Postman으로 테스트를 해보자.

```
# 스프링 프로젝트 경로로 들어가서 아래 명령어 실행
$ ./gradlew clean build -x test
```

```
$ cd build/libs
$ java -jar -Dspring.profiles.active=prod {빌드된 jar 파일명}
```



여러 번 요청을 보내보니 평균적으로 500ms 정도의 속도가 나오는 걸 확인할 수 있다. 레디스를 적용하지 않으니 확연하게 성능이 느려진 걸 확인할 수 있다.

kb302500@gmail.com

📖

내가 구성한 백엔드 서버는 1초당 몇 개의 요청을 견뎌낼 수 있을까?

✅ 부하 테스트의 기본 개념

백엔드 서버를 구현하고 나서 배포를 하게 된다. 실제 서비스에 배포를 하기 전에 문득 이런 생각이 들 수도 있다.

"혹시 요청이 몰려서 서버가 터지면 어떡하지?"

"내 서버는 어느 정도 사용자 요청을 견딜 수 있는 거지?"

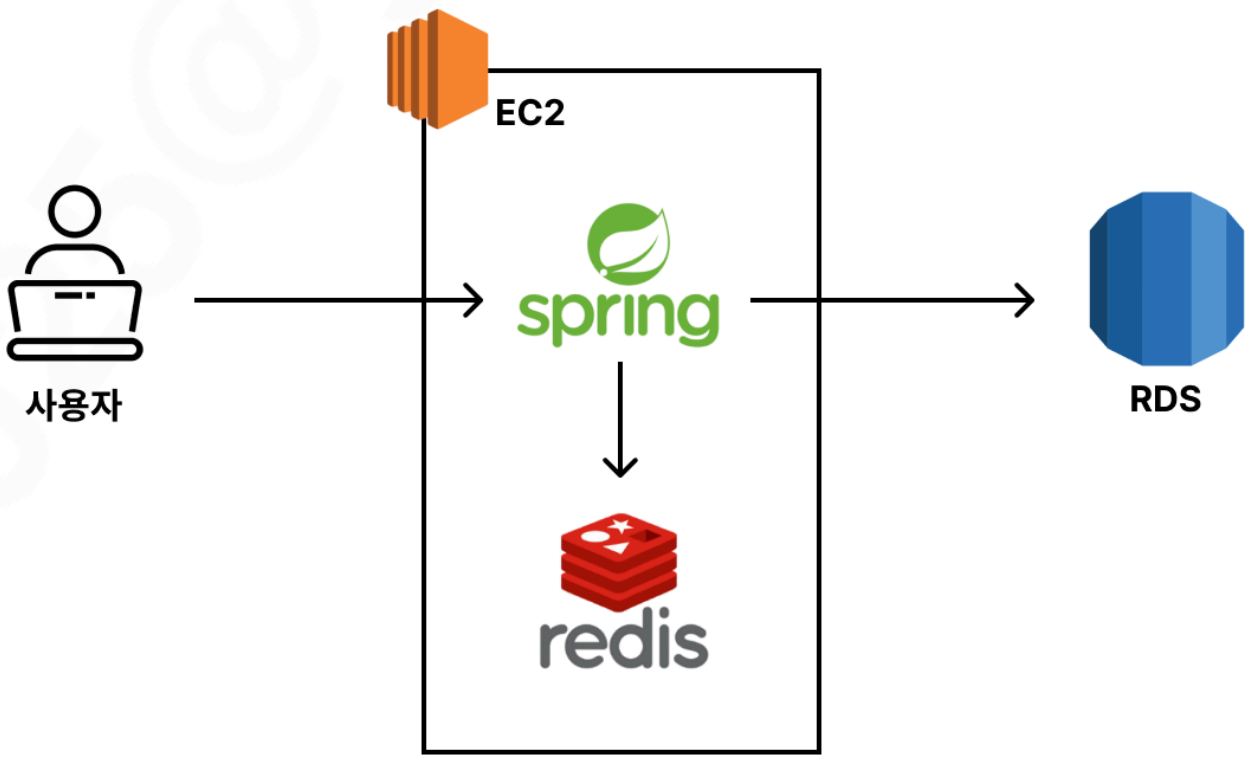
이런 불안감을 없애기 위해서 서비스를 배포하기 전에 백엔드 서버가 어느 정도의 요청을 견딜 수 있는 지 **부하 테스트**를 해봐야 한다. 부하 테스트를 본격적으로 시작하기 전에 자주 사용하는 용어부터 정리하고 넘어가자.

부하 테스트에서 **서비스가 1초당 처리할 수 있는 작업량**을 보고 **Throughput**이라고 부른다. 단위는 **TPS(Transaction Per Seconds, 1초당 처리한 트랜잭션의 수)**를 많이 활용한다. 만약 내가 만든 서비스가 1초에 최대 100개의 API 요청을 처리할 수 있다면, 이 서비스의 **Throughput**은 **100 TPS**라고 얘기한다.

이 정도의 용어만 알아둬도 충분하다. 이제 우리가 구축한 서비스의 Throughput을 측정해보자. 우리는 아래와 같이 인프라를 구축했다.

- EC2 - t3a.small
  - Spring Boot
  - Redis
- RDS - t3.micro

위와 같은 사양으로 인프라를 구성했을 때 어느 정도의 요청을 견딜 수 있는 지 테스트 해볼 것이다. 그리고 Redis가 포함되어 있고 없고의 성능 차이도 비교해볼 것이다.



📖

부하 테스트를 위한 환경 셋팅 (k6)

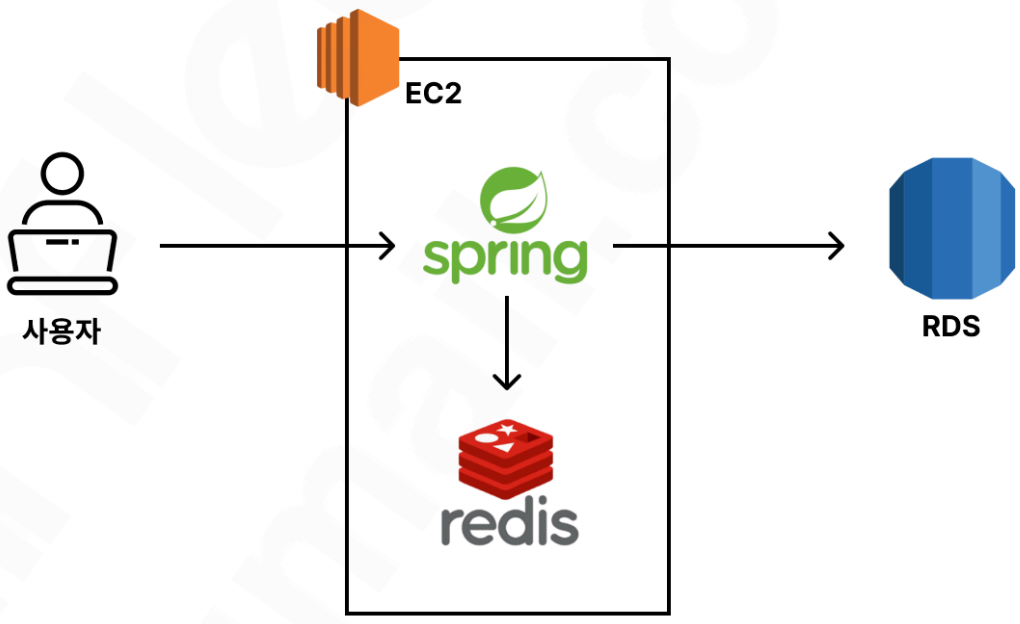
✅ k6란?

성능 비교를 위해 k6라는 부하테스트 툴을 사용할 것이다. 부하테스트 툴에는 k6 이외에도 ngrinder, jmeter, ab, locust 등 다양한 툴이 있다. 하지만 그 중에서 간단하고 빠르게 테스트 해볼 수 있는 툴인 **k6**를 활용하고자 한다.

간단하게 빠르게 테스트할 수 있다고 해서 테스트 결과가 부정확한 건 아니다. k6도 높은 정확도와 고부하를 발생시킬 수 있는 부하테스트 툴이다. k6를 설치해보자. 현업에서도 많이 쓰니까 걱정하지 말고 사용해도 된다.

(부하 테스트에 대한 강의가 아니기 때문에 자세한 k6의 사용법에 초점을 맞추진 않겠다. 성능 지표 위주로 참고할 예정이다.)

✅ k6는 사용자인척 요청을 보내는 툴이다.



원래라면 사용자가 직접 서비스에 요청을 보내야 하는데, k6는 여러 명의 사용자를 대신해서 요청을 보내는 툴이다.

✅ k6 설치 방법

> **공식문서 : https://k6.io/docs/get-started/installation/**

Windows - 설치 방법 1

아래 영상을 참고해서 설치해보도록 하자. k6 프로그램을 직접 다운받아서 설치하는 방법이다.

#2 - Install K6 on Windows OS | Run First Script
#2 - Install K6 on Windows OS | Run First Script
Schedule a meeting in case of any queries/guidance/counselling:
https://youtu.be/eVmcDt5C8io



Windows - 설치 방법 2

Package Manager를 활용해서 설치하는 방법이다.

K6 성능 테스트 툴

K6이란? k6은 엔지니어링 팀의 성능 테스트를 쉽고 생산적으로 만드는 오픈 소스 부하 테스트 도구이다. k6은 무료이며 개발자 중심적이며 확장 가능하다. 주요특징 테스트 스크립트를 Javascript (ES6) 로 작성. jmeter (jmx) 대비 개발자 친화적이고, 시나리오와 함께 테스트 설정까지 코드로 구현 가능, 재사용을 위한 모듈화 가능 Jmeter 대비 더 작은 리소

https://yscho03.tistory.com/101#google_vignette



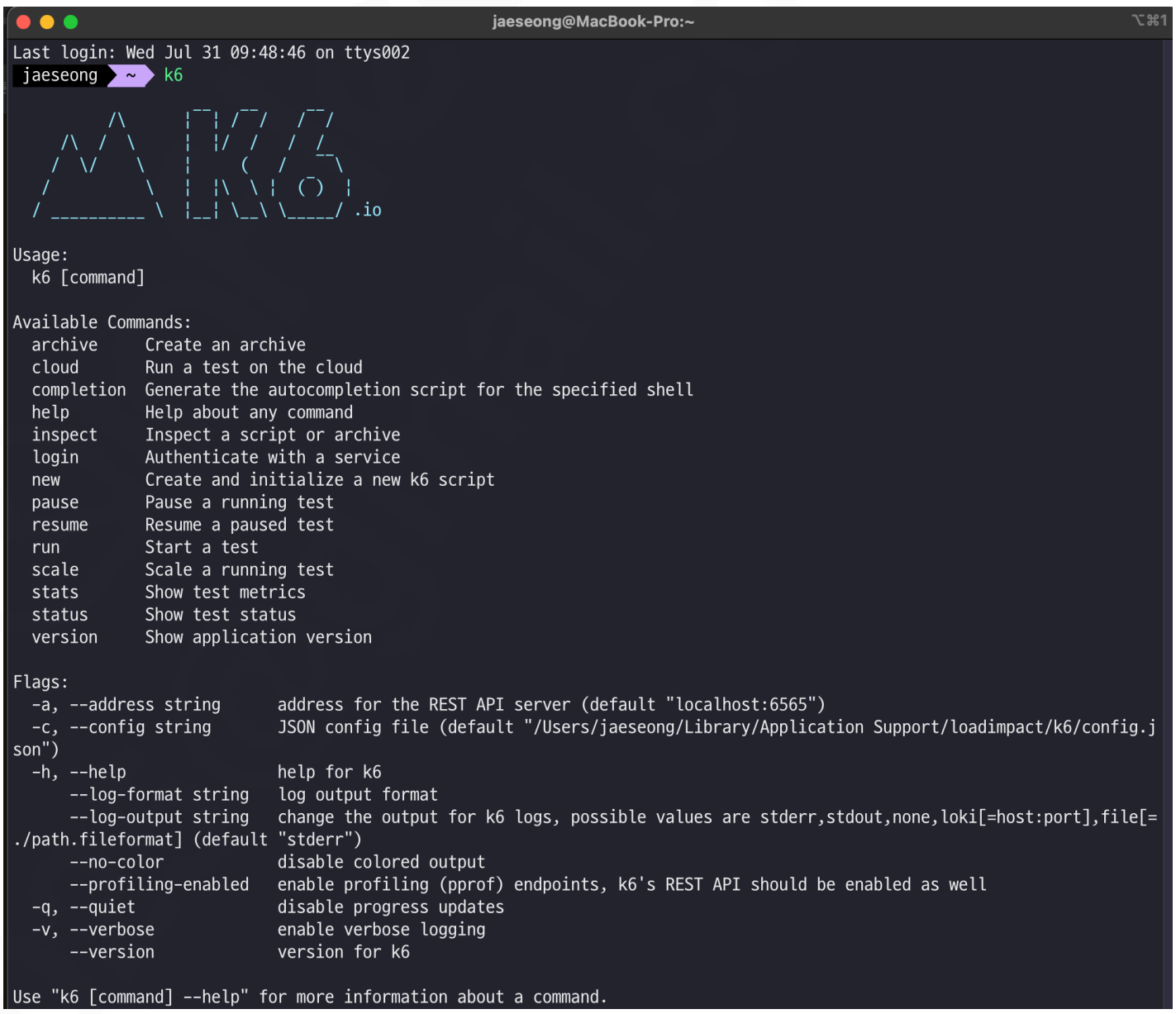
MacOS

MacOS에서는 brew를 활용해 쉽게 설치할 수 있다.

```
$ brew install k6
```

✅ k6 잘 설치됐는 지 확인하기

```
$ k6
```



터미널 창에 `k6` 라고 입력했을 때 위와 같은 화면이 뜬다면 잘 설치된 것이다.

✅ API에 부하를 주기 위해 k6 스크립트 작성

script.js

```
import http from 'k6/http';
import { sleep } from 'k6';
```

```
export default function () {
  http.get('http://{EC2 IP 주소}:8080/boards');
}
```

> 부하 테스트를 하기 위한 기본적인 셋팅은 끝이 났다. 다음 강의에 본격적으로 부하 테스트를 해보자.

kb302500@gmail.com

📖

Redis를 적용하기 전·후 Throughput(처리량) 비교해보기

✅ 캐싱을 적용시키기 전의 Throughput 측정

1. **캐싱 코드 주석처리하기**

   BoardService

```
@Service
public class BoardService {
  ...
  //  @Cacheable(cacheNames = "getBoards", key = "'boards:page:' + #page + ':size:' + #size", cacheManager = "boa
  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
  }
}
```

2. **Spring Boot 서버 빌드 및 백그라운드 실행**

```
# 스프링 프로젝트 경로로 들어가서 아래 명령어 실행
$ ./gradlew clean build -x test

# 정확한 테스트를 위해 Spring Boot 서버를 백그라운드에서 실행시키자.
$ cd build/libs
$ nohup java -jar -Dspring.profiles.active=prod {빌드된 jar 파일명} &

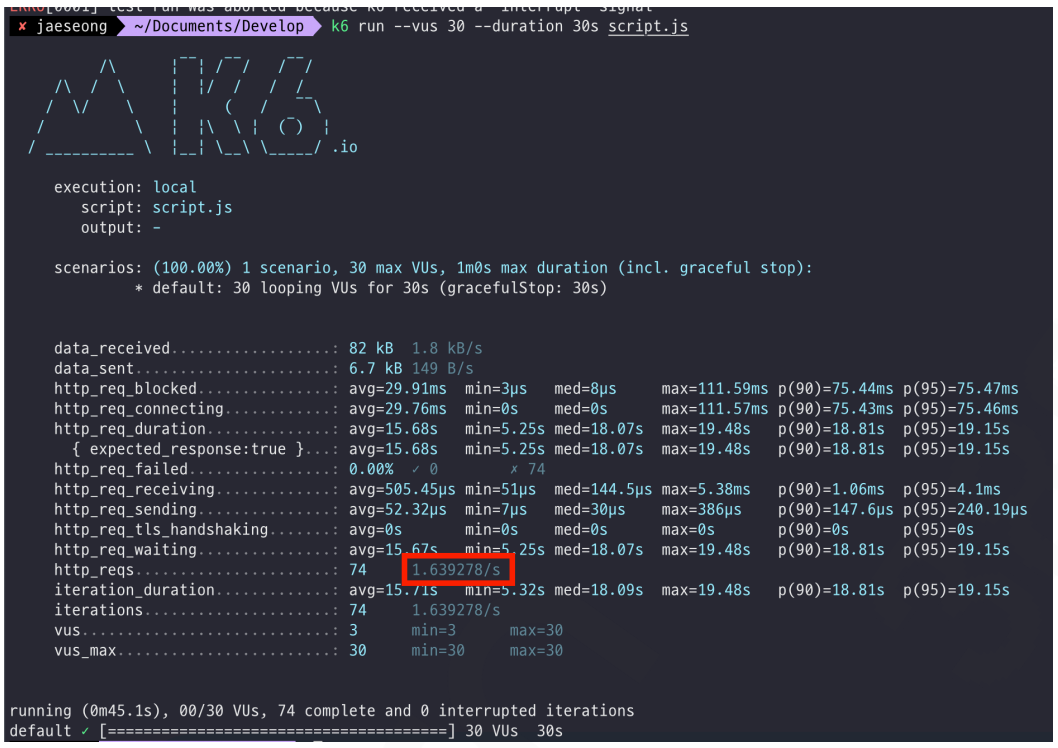
# 8080번 포트에 Spring Boot 서버가 잘 실행되고 있는 지 확인
$ lsof -i:8080
```

3. **로컬 환경에서 K6로 성능 테스트 해보기**

> 더 정확한 성능 측정 방법이 있지만 성능 테스트에 초점을 맞춘 강의가 아니기 때문에 간편한 방법으로 측정하고자 한다.

```
# K6의 스크립트 파일이 위치한 경로에서 아래 명령어 실행시키기
$ k6 run --vus 30 --duration 10s script.js
```

- `--vus 30` : 가상 유저(Virtual Users)를 30명으로 셋팅
  (API 요청을 보내는 사용자가 30명인 것처럼 부하 생성)
- `--duration 30s` : 30초 동안 테스트를 유지



평균적으로 1초에 1.6개의 요청을 처리했다는 뜻이다. 즉, 이 서비스는 1초에 최대 처리할 수 있는 요청의 처리 개수가 1.6개라는 뜻이다.
개발자답게 표현하자면 '**현재 구축한 서비스에서 게시글 조회 API의 Throughput이 1.6 TPS다**'라고 얘기할 수 있다.

✅ 캐싱을 적용시킨 후에 Throughput 측정

1. **캐싱 코드 주석 해제하기**

   BoardService

```
@Service
public class BoardService {
  ...
  @Cacheable(cacheNames = "getBoards", key = "'boards:page:' + #page + ':size:' + #size", cacheManager = "board
  public List<Board> getBoards(int page, int size) {
    Pageable pageable = PageRequest.of(page - 1, size);
    Page<Board> pageOfBoards = boardRepository.findAllByOrderByCreatedAtDesc(pageable);
    return pageOfBoards.getContent();
  }
}
```

2. **Spring Boot 서버 빌드 및 백그라운드 실행**

```
# 스프링 프로젝트 경로로 들어가서 아래 명령어 실행
$ ./gradlew clean build -x test

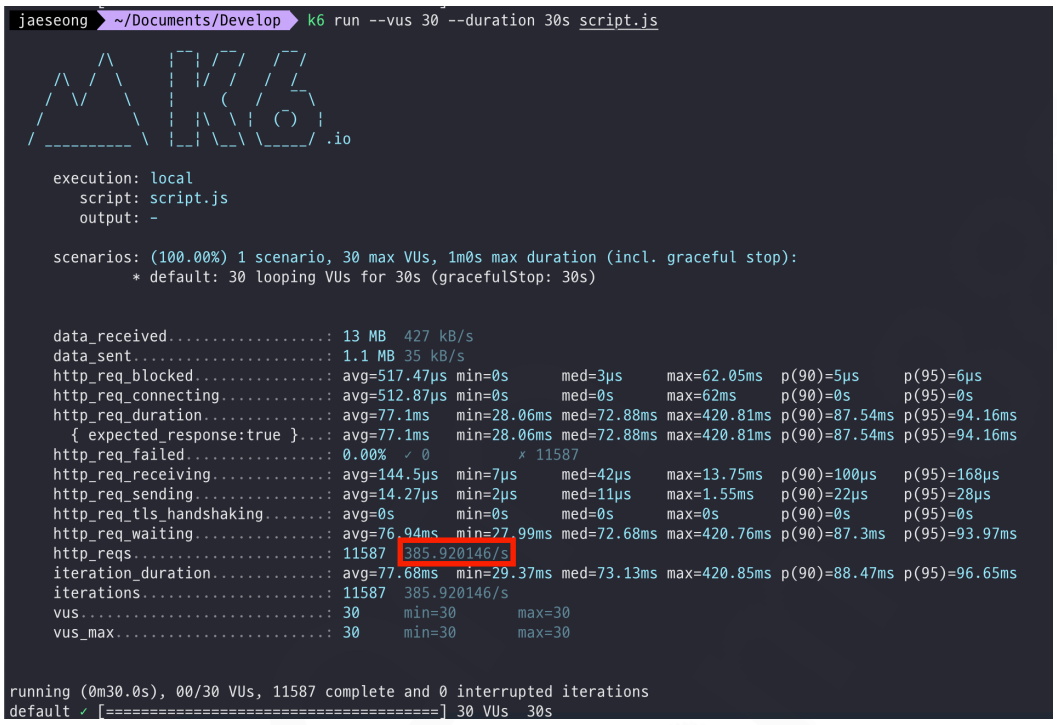
# 기존 서버 종료
$ lsof -i:8080
$ kill {PID 값}

# 정확한 테스트를 위해 Spring Boot 서버를 백그라운드에서 실행시키자.
$ cd build/libs
$ nohup java -jar -Dspring.profiles.active=prod {빌드된 jar 파일명} &

# 8080번 포트에 Spring Boot 서버가 잘 실행되고 있는 지 확인
$ lsof -i:8080
```

3. **로컬 환경에서 K6로 성능 테스트 해보기**

$ k6 run --vus 30 --duration 10s script.js



평균적으로 1초에 385개의 요청을 처리했다는 뜻이다. 즉, 이 서비스는 1초에 최대 처리할 수 있는 요청의 처리 개수가 385개라는 뜻이다. 개발자답게 표현하자면 '**현재 구축한 서비스에서 게시글 조회 API의 Throughput이 385이다**'라고 얘기할 수 있다. Throughput이 385라는 뜻은 1초에 385개 이하의 요청까지는 견딜 수 있는 서비스라고도 해석할 수 있다.

✅ 성능 비교

Redis의 캐싱을 활용하니 성능이 약 240배(= 385 / 1.6) 향상되었다.

khb302@naver.com

📖

Docker Compose로 Redis, Spring Boot 한 번에 띄울 수 있게 구성하기

👩🏻 Docker에 초점을 맞춘 강의가 아니기 때문에 Docker에 대한 디테일한 설명은 생략할 예정이다. 혹시 Docker에 대한 기본기를 다지고 싶다면 아래 강의를 추천한다.

비전공자도 이해할 수 있는 Docker 입문/실전
(
https://inf.run/UvNen)

✅ Docker Compose로 Redis, Spring Boot 한 번에 띄울 수 있게 구성하기

1. **Dockerfile 만들기**

   Dockerfile

```
FROM openjdk:17-jdk

COPY build/libs/*SNAPSHOT.jar app.jar

ENTRYPOINT ["java", "-jar", "/app.jar"]
```

2. **compose.yml 만들기**

   compose.yml

```
services:
  api-server:
    build: .
    ports:
      - 8080:8080
    depends_on:
      cache-server:
        condition: service_healthy
  cache-server:
    image: redis
    ports:
      - 6379:6379
    healthcheck:
      test: [ "CMD", "redis-cli", "ping" ]
      interval: 5s
      retries: 10
```

3. **기존에 실행되고 있는 Redis, Spring Boot 종료하기**

   (Mac 기준)

```
# Redis 중지
$ brew services stop redis
$ brew services info redis # 잘 종료됐는 지 확인
```

```
# Spring Boot 종료
$ lsof -i:8080 # 8080번 포트 실행되고 있는 프로세스 확인
$ kill {Spring Boot의 PID} # 프로세스 종료
$ lsof -i:8080 # 잘 종료됐는 지 확인
```

4. **application.yml 수정하기**

   application.yml

```
# local 환경
spring:
  profiles:
    default: local
  datasource:
    url: jdbc:mysql://host.docker.internal:3306/mydb
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  data:
    redis:
      host: cache-server
      port: 6379

logging:
  level:
    org.springframework.cache: trace

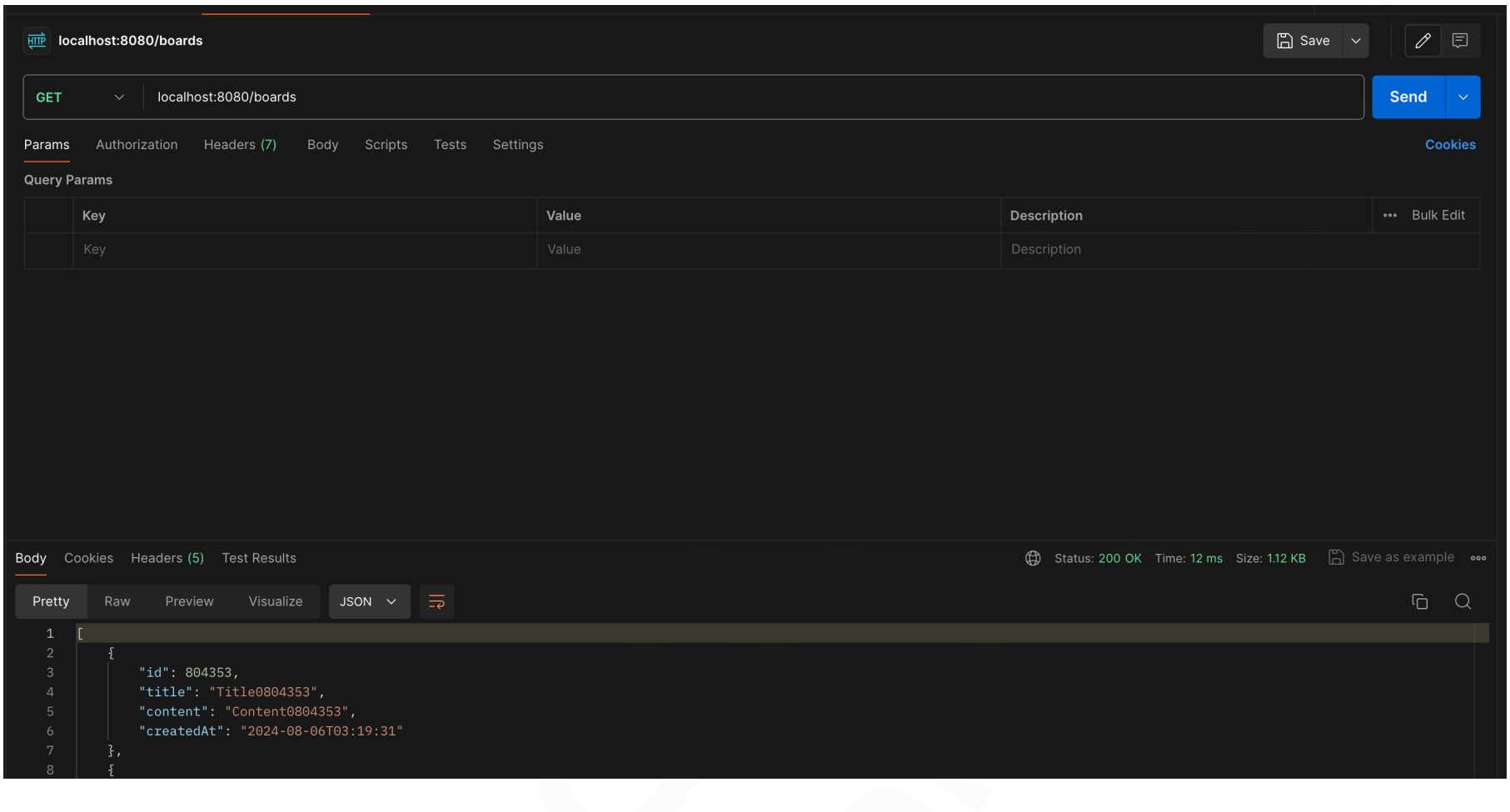

---
# prod 환경
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: jdbc:mysql://instagram-db.coseefawhrzc.ap-northeast-2.rds.amazonaws.com:3306/mydb
    username: admin
    password: password
```

5. **Docker 컨테이너로 띄워보기**

```
$ ./gradlew clean build -x test
$ docker compose up --build -d

$ docker ps # 잘 띄워졌는 지 확인
$ docker compose logs -f # 실시간 로그 확인하기
```

6. **Postman으로 테스트해보기**



@naver.com
kb302

📖

AWS EC2에서 Docker Compose를 활용해 Redis, Spring Boot 띄워보기

✅ AWS EC2에서 Docker Compose를 활용해 Redis, Spring Boot 띄워보기

1. **EC2 내에서 사용할 Dockerfile, Docker compose 파일 만들기**

   Dockerfile-prod

```
FROM openjdk:17-jdk

COPY build/libs/*SNAPSHOT.jar app.jar

ENTRYPOINT ["java", "-jar", "-Dspring.profiles.active=prod", "/app.jar"]
```

   compose-prod.yml

```
services:
  api-server:
    build:
      context: .
      dockerfile: ./Dockerfile-prod
    ports:
      - 8080:8080
    depends_on:
      cache-server:
        condition: service_healthy
  cache-server:
    image: redis
    ports:
      - 6379:6379
    healthcheck:
      test: [ "CMD", "redis-cli", "ping" ]
      interval: 5s
      retries: 10
```

2. **로컬에서 구현한 내용을 Github에 Push하기**

```
$ git commit -m "{커밋 메시지}"
$ git push
```

3. **EC2에서 Git Pull 받기**

```
$ cd {프로젝트 경로}
$ git pull origin main
```

2. **EC2에 Docker 설치하기**

```
$ sudo apt-get update && \
    sudo apt-get install -y apt-transport-https ca-certificates curl software-properties-common && \
    curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add - && \
    sudo apt-key fingerprint 0EBFCD88 && \
    sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
    sudo apt-get update && \
    sudo apt-get install -y docker-ce && \
    sudo usermod -aG docker ubuntu && \
    newgrp docker && \
    sudo curl -L "https://github.com/docker/compose/releases/download/2.27.1/docker-compose-$(uname -s)-$(unam
    sudo chmod +x /usr/local/bin/docker-compose && \
    sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose

$ docker -v # Docker 버전 확인
$ docker compose version # Docker Compose 버전 확인
```

3. **기존에 실행되고 있는 Redis, Spring Boot 종료하기**

```
# Redis 중지
$ sudo systemctl stop redis
$ sudo systemctl status redis # 잘 종료됐는 지 확인

# Spring Boot 종료
$ sudo lsof -i:8080 # 8080번 포트 실행되고 있는 프로세스 확인
$ kill {Spring Boot의 PID} # 프로세스 종료
$ sudo lsof -i:8080 # 잘 종료됐는 지 확인
```

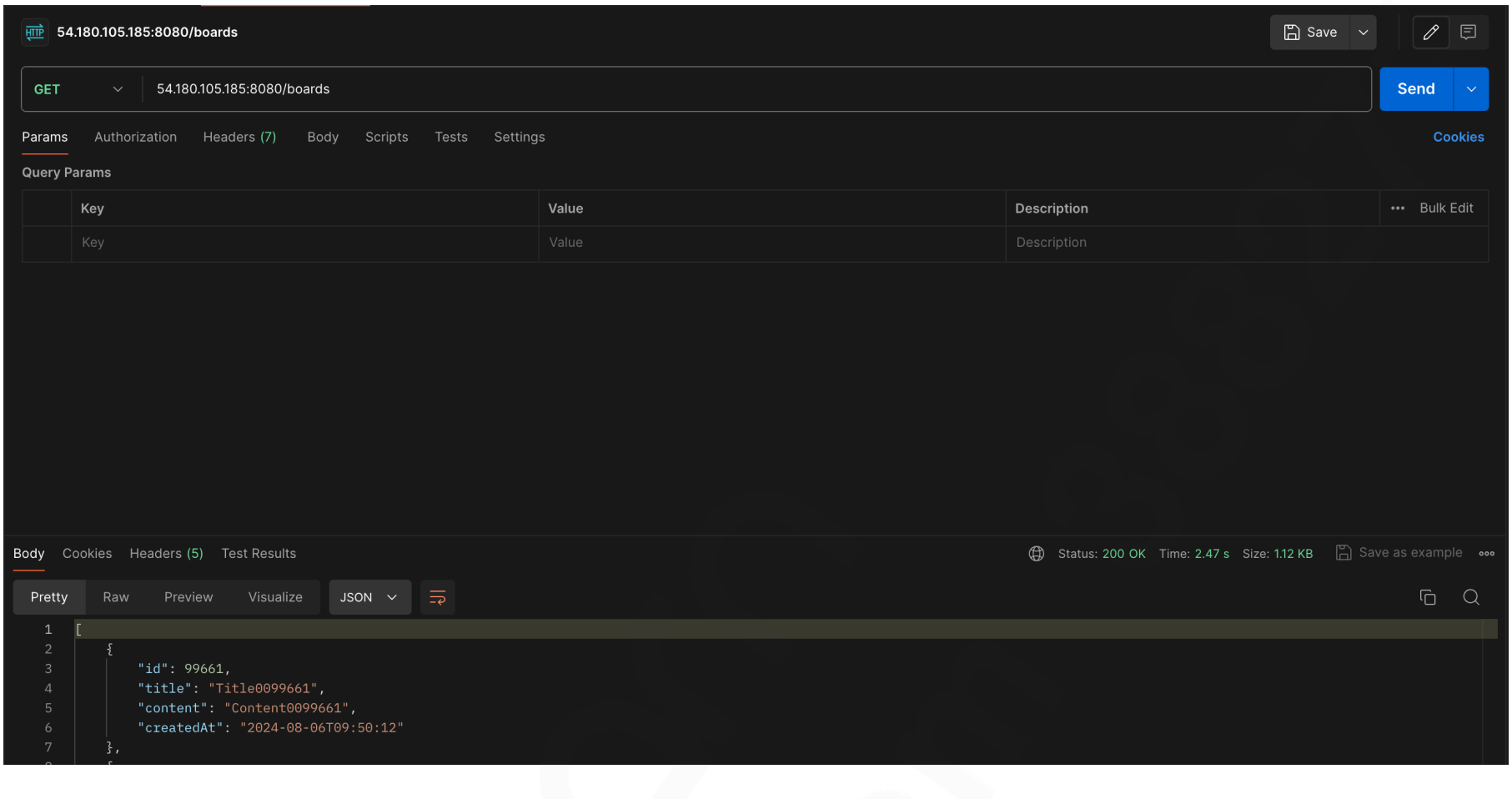
5. **Docker 컨테이너로 띄워보기**

```
$ ./gradlew clean build -x test
$ docker compose -f compose-prod.yml up --build -d

$ docker ps # 잘 띄워졌는 지 확인
$ docker compose logs -f # 실시간 로그 확인하기
```

6. **Postman으로 테스트해보기**

khb3023.com



kb302500@gmail.com



현업에서 EC2에 Redis를 설치해서 쓰지 않고 ElastiCache를 쓰는 이유

✅ 현업에서 EC2에 Redis를 설치해서 쓰지 않고 ElastiCache를 쓰는 이유

현업에서 EC2에 Redis를 직접 설치해서 사용하는 경우는 드물다. 일일이 Redis를 설치하고 셋팅하고 관리하면서 확장까지 하려면 신경쓸 게 생각보다 많다. 하지만 ElastiCache를 사용하면 셋팅도 쉽게할 수 있고, 확장도 쉽게 할 수 있고, 기본적인 모니터링 기능도 제공해주고, 장애가 날 가능성도 훨씬 적다. 이런 이유로 인해 현업에서는 ElasitiCache를 많이 활용한다.

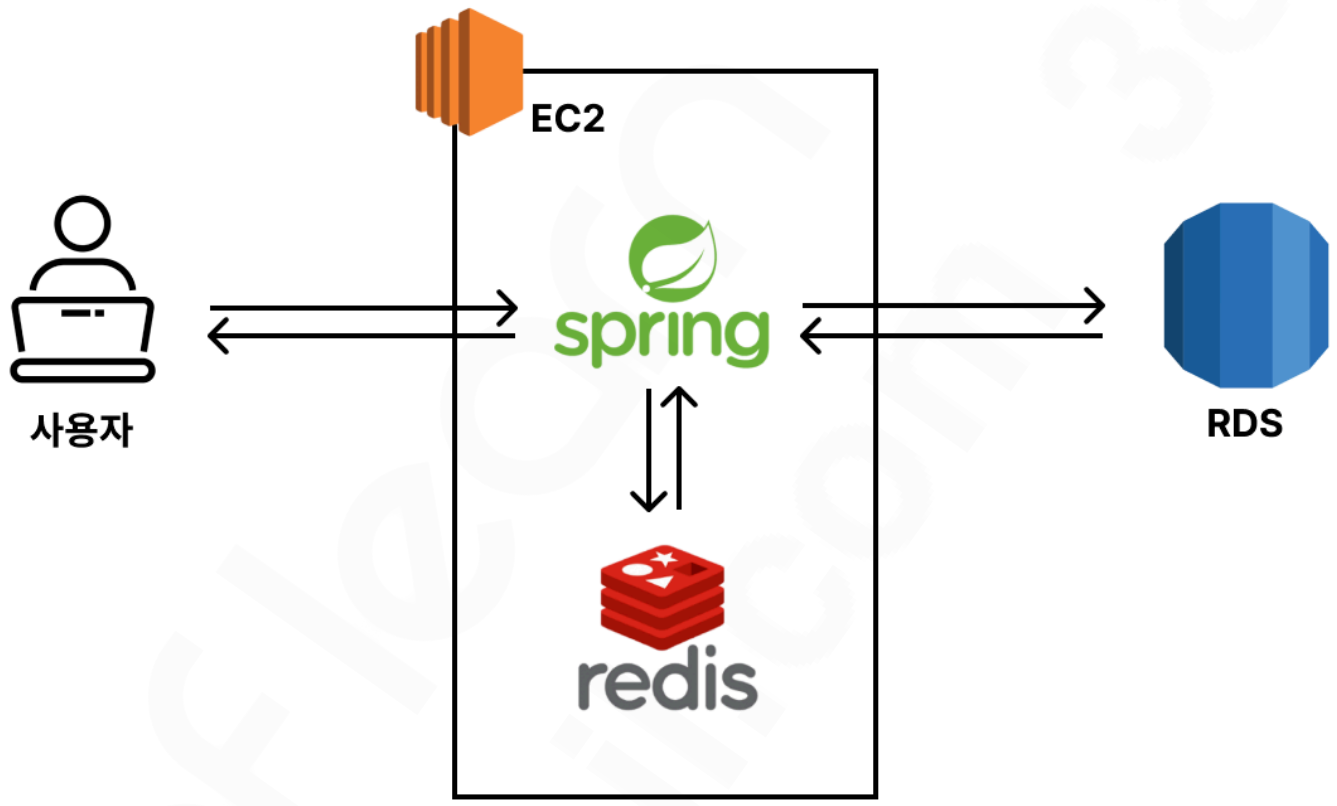
> **“선생님, 그러면 우린 왜 EC2에 Redis를 설치해서 사용하는 건 왜 배운 건가요?”**

EC2에서 Redis를 직접 손수 만져보지 않고 바로 ElastiCache를 사용하게 되면 어떤 식으로 작동하는 지 이해하기가 굉장히 어렵다. 학습을 하는 입장에서는 다 만들어져있는 서비스를 쓰는 것보다 처음부터 구축해보는 게 훨씬 도움이 된다.

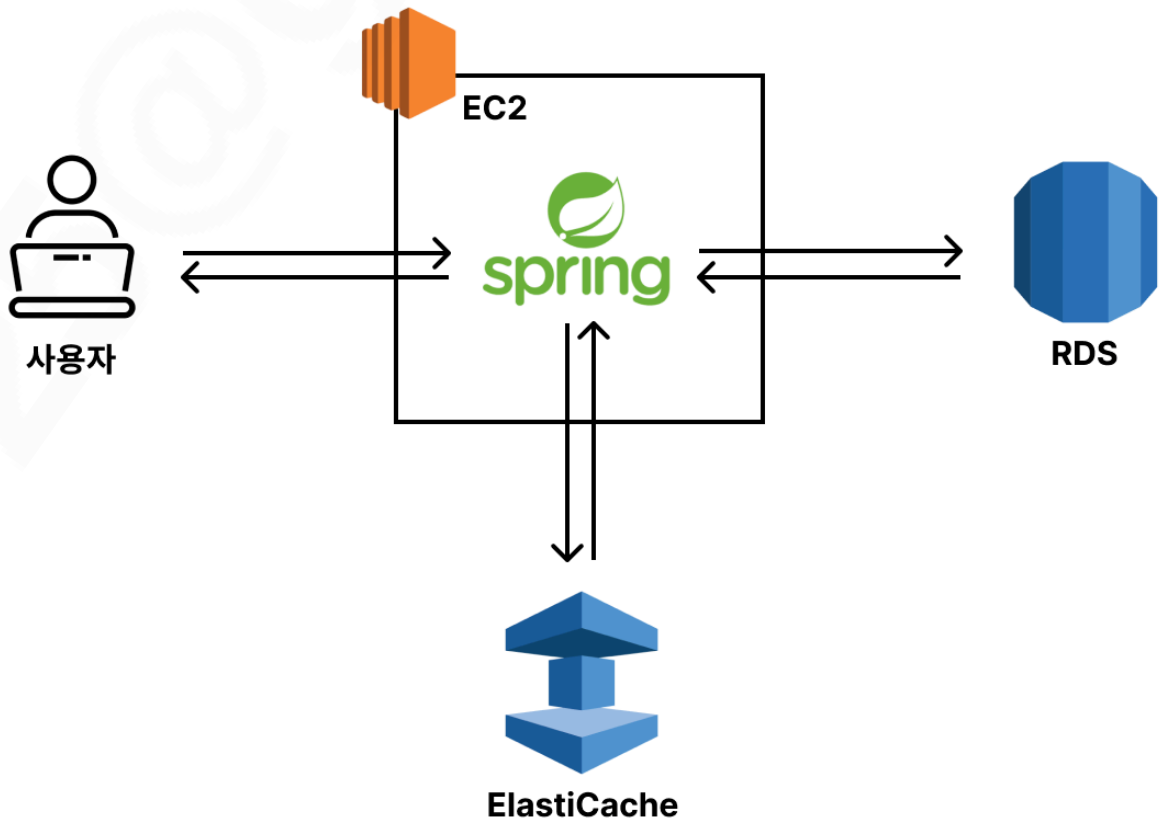
kkb3025@gmail.com

EC2, RDS, Spring Boot, ElastiCache를 활용한 아키텍처 구성

✅ 이전 아키텍처 구성



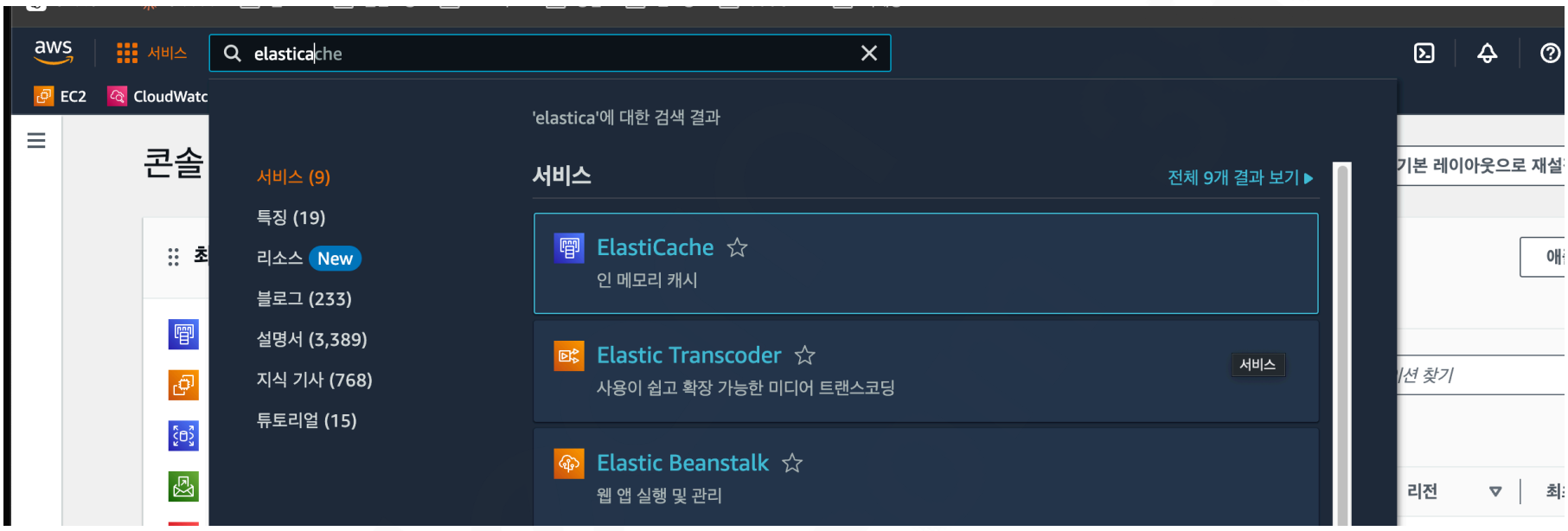
✅ ElastiCache를 도입했을 때의 아키텍처 구성



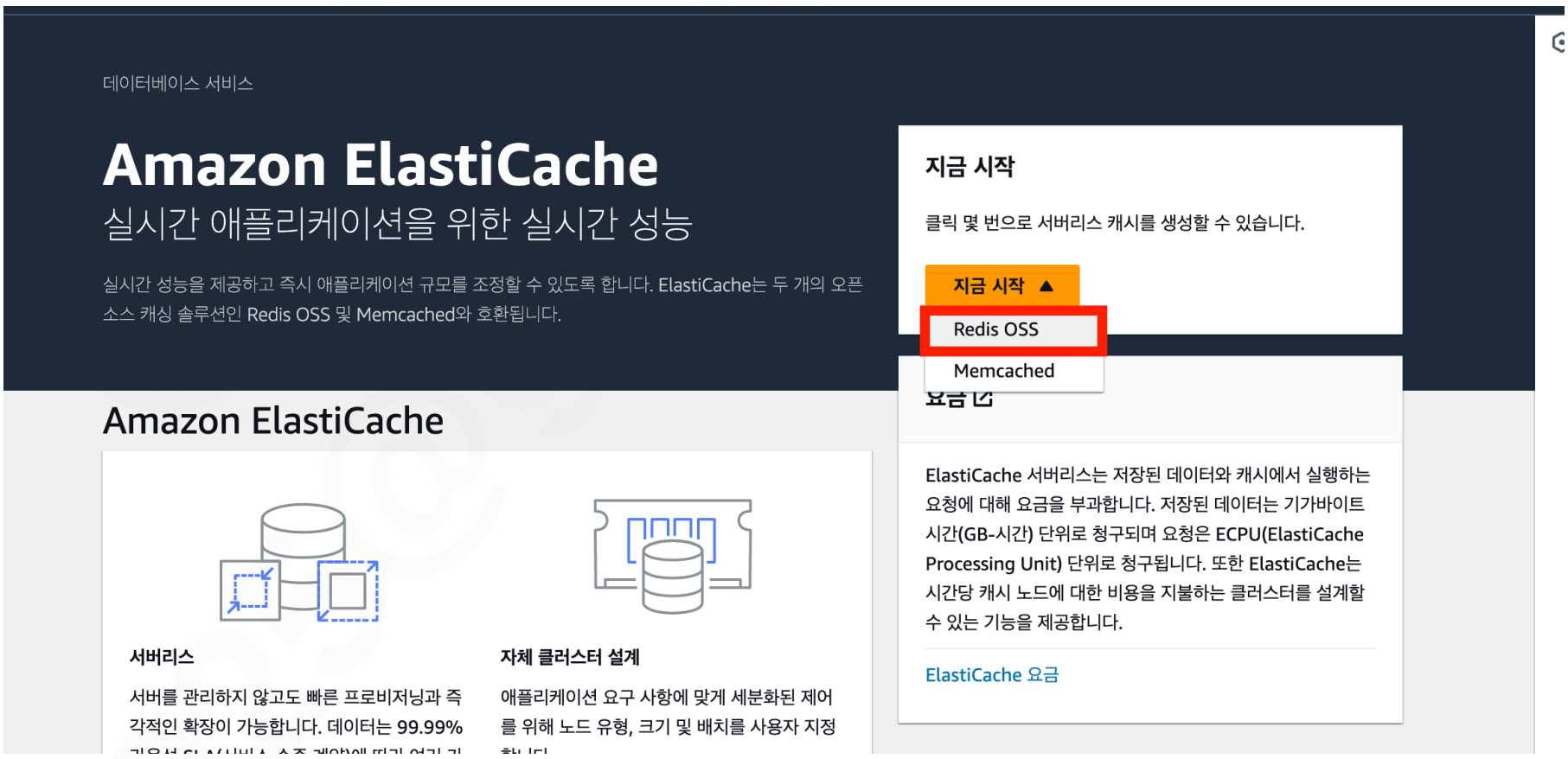
AWS ElastiCache 셋팅하기

✅ AWS ElastiCache 셋팅하기

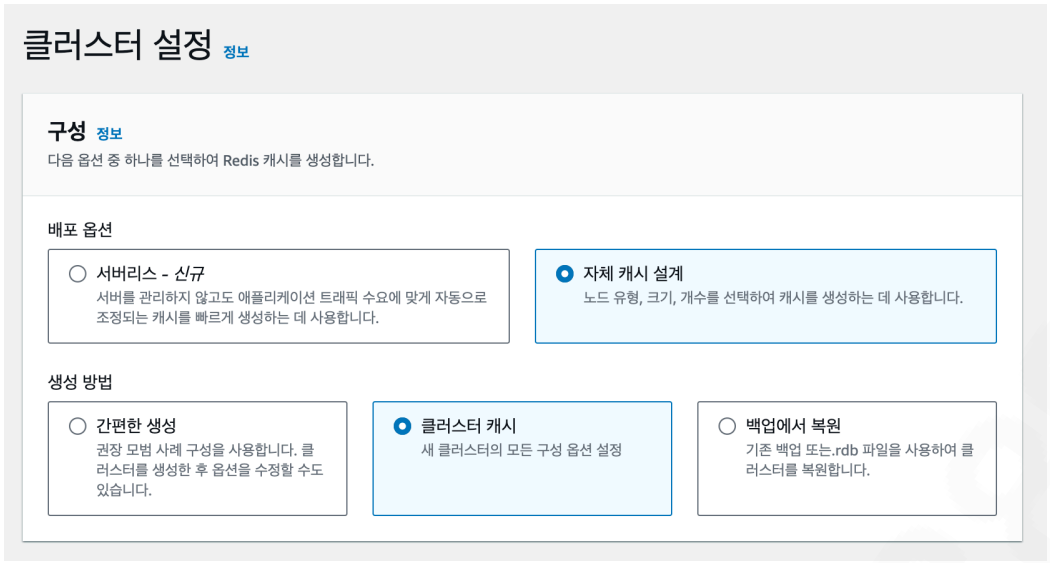
1. **ElastiCache 서비스로 들어가기**



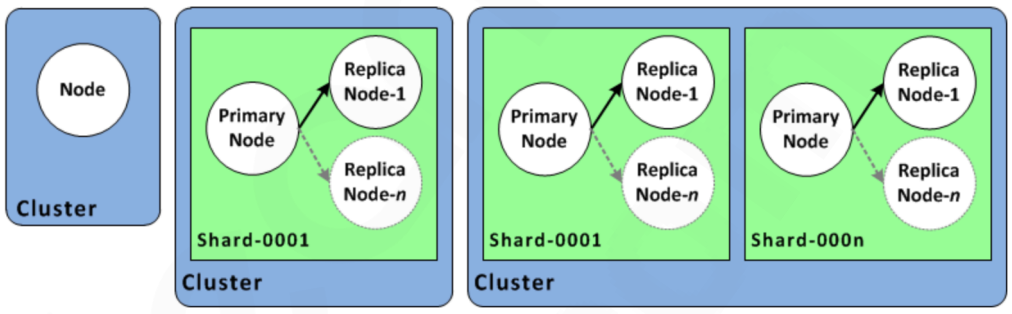
2. **캐시 생성을 위해 '지금 시작' 버튼 누르기**



3. **클러스터 설정에서 '구성' 선택하기**



여기서 얘기하는 **클러스터(cluster)**란 **여러 캐시 서버를 이루는 한 단위의 그룹**을 뜻한다. 여기서 **하나의 캐시 서버**를 **노드(Node)**라고 얘기한다.



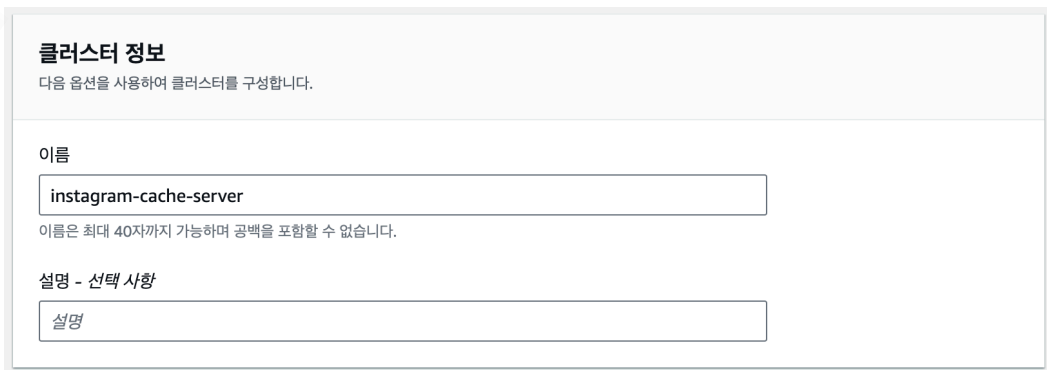
출처 : AWS 공식 문서

4. ‘클러스터 모드’ 설정하기



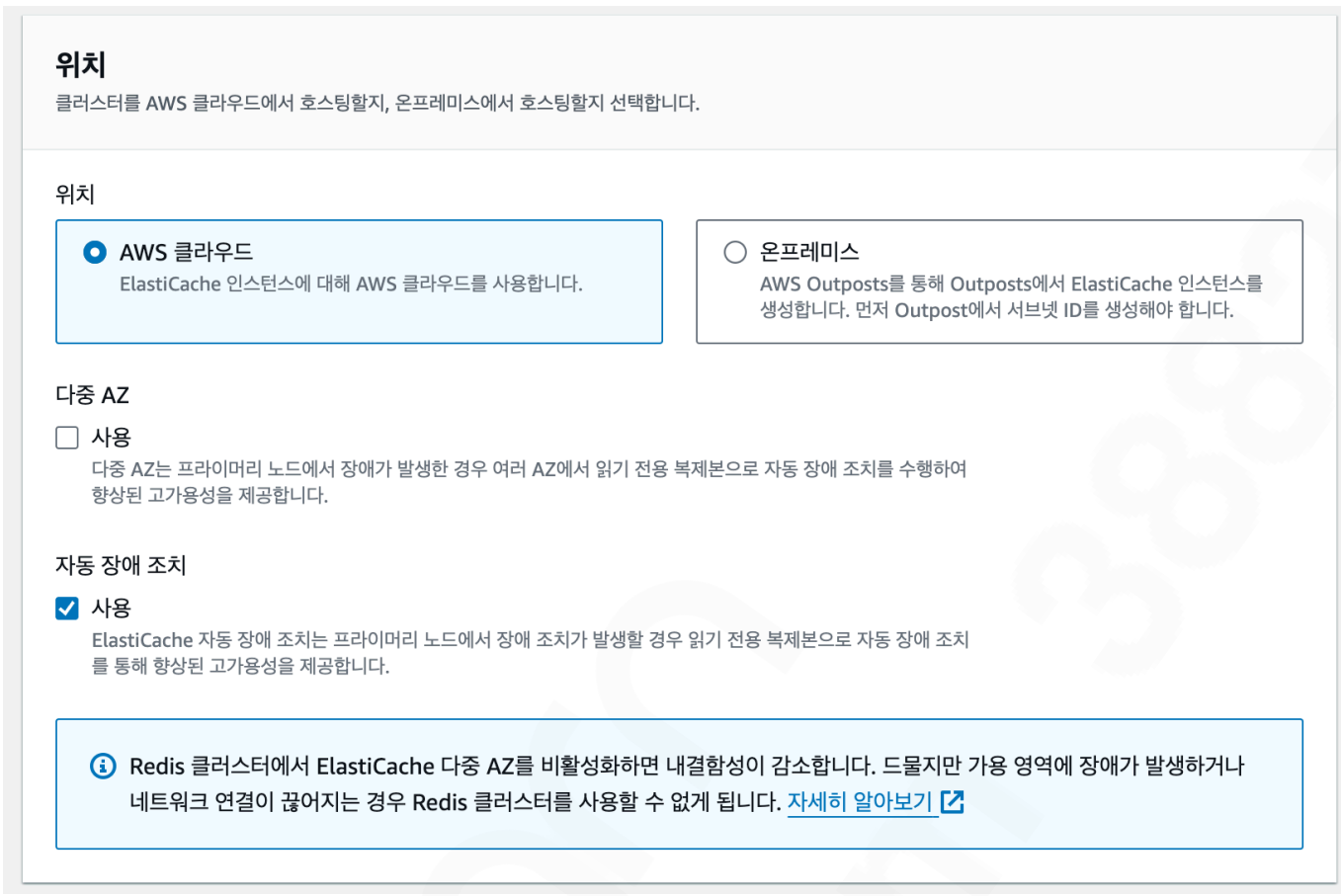
대규모 트래픽 처리를 위한 ‘클러스터 모드’라는 게 있다. 하지만 어지간한 규모의 트래픽이 발생하는 서비스가 아니라면 ‘클러스터 모드’를 쓸 일은 없다. 나중에 관심이 있으면 따로 공부하도록 하자. 레디스를 입문할 때 우선순위가 높은 기능은 아니다.

5. ‘클러스터 정보’ 입력하기

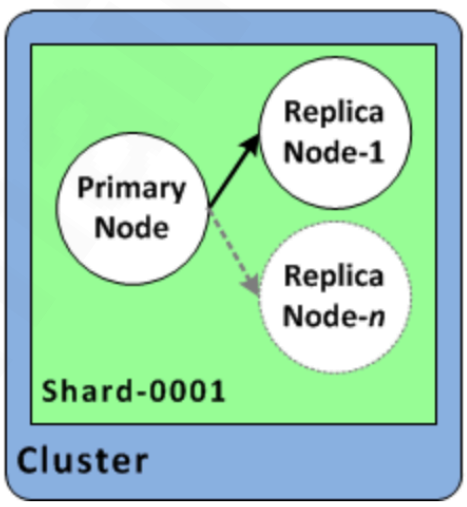


여러 클러스터를 생성했을 때 구별하기 위해 이름을 붙인다. 이름은 바꿀 수 있으니 편하게 적어도 된다.

6. ‘위치’ 설정하기

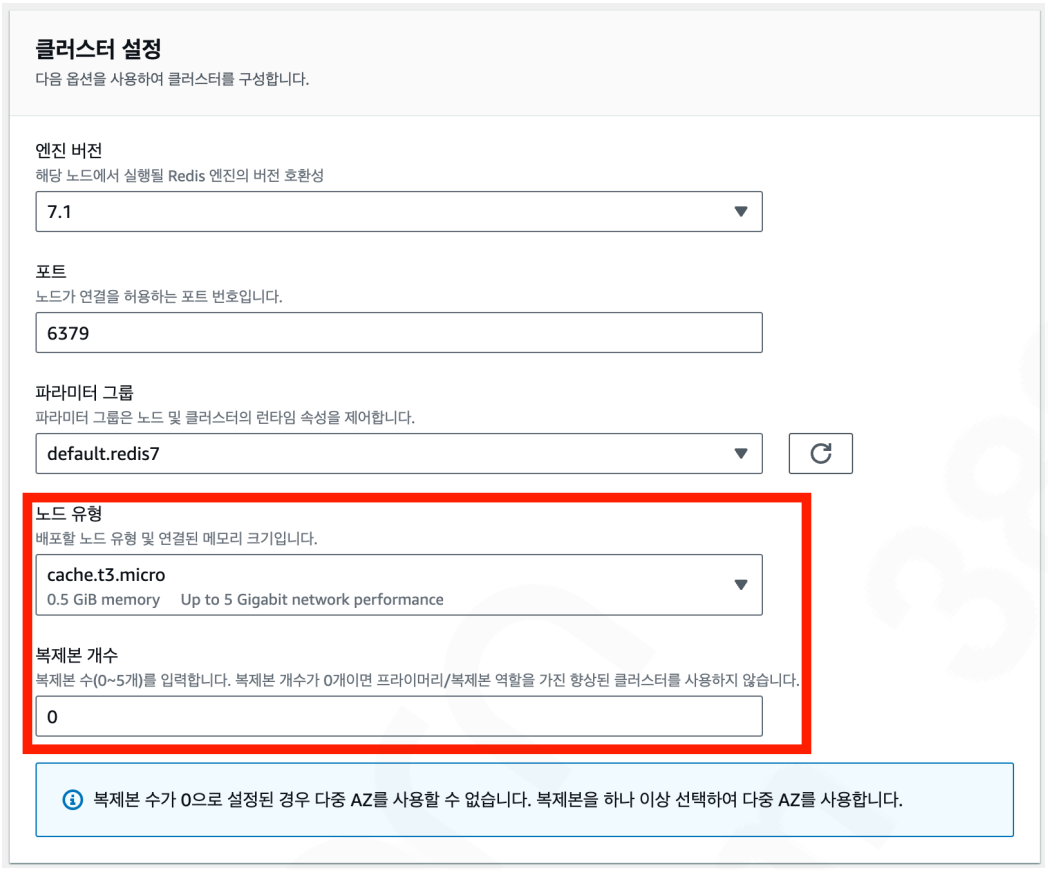


- **다중 AZ(Multi AZ)**는 여러 Region에 캐시 서버를 나눠서 셋팅해두는 것이다. 특정 Region에서 재난이 발생해 서비스가 중단될 수도 있는 걸 방지하는 기능이다. 하지만 재난이 발생할 가능성은 아주 적은데 비해서 비용은 추가로 발생하기 때문에, 재난으로 인해 서비스가 중단되는 게 치명적인 경우가 아니면 사용하지 말자.
- **자동 장애 조치(Failover)**는 클러스터 내부에 특정 노드(Node)가 장애가 났을 때 정상 노드(Node)로 교체하는 기능이다. 쉽게 얘기해서 내부에 장애가 일어나면 스스로 고치는 기능이다.



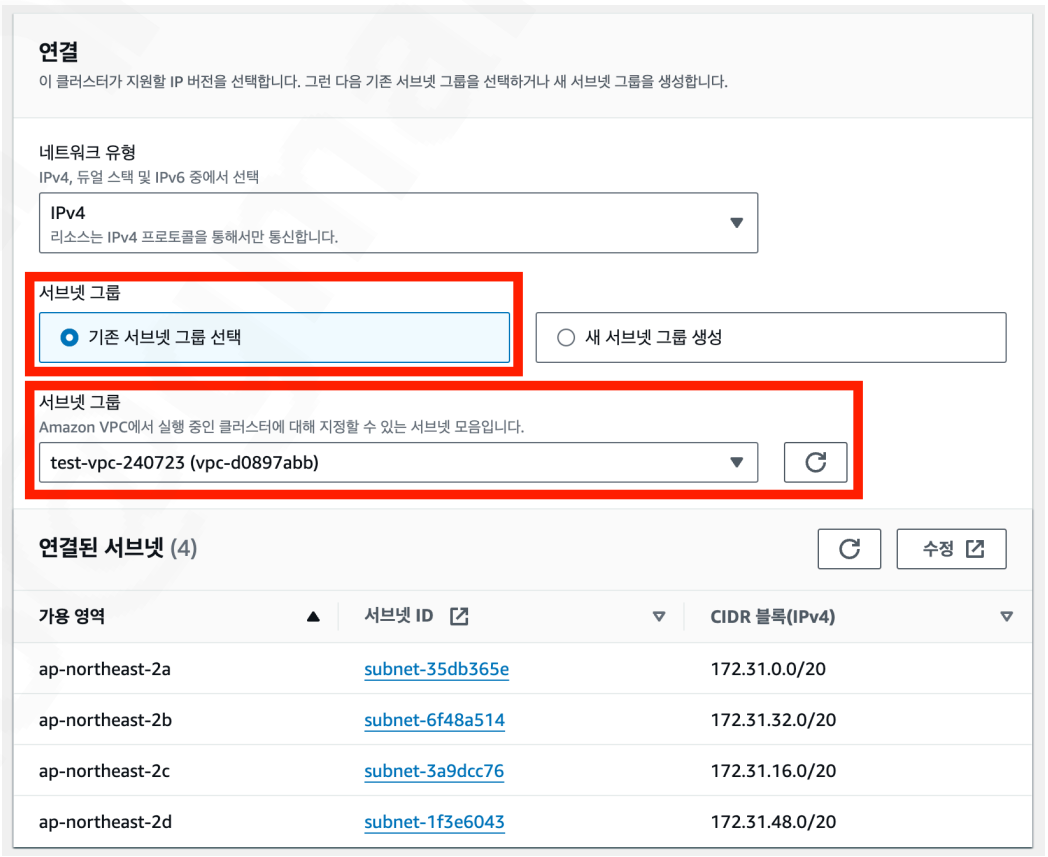
7. **'클러스터 설정' 셋팅하기**

khb302500@gmail.com

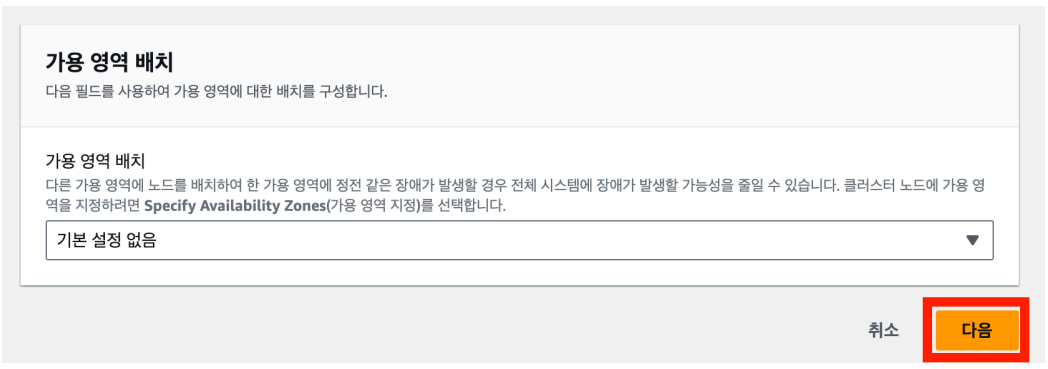


- 최소한의 비용으로 테스트하기 위해 복제본 개수는 0으로 하자.
  - 복제본 개수가 늘어날수록 노드(Node)가 늘어난다. ElastiCache는 노드(Node)당 가격을 매기고 있다.
  - 복제본 개수를 1 이상 써야만 Failover 기능을 활용할 수 있다. 이 때문에 프로덕션에서는 복제본 개수를 1개 이상 만드는 경우가 많다.

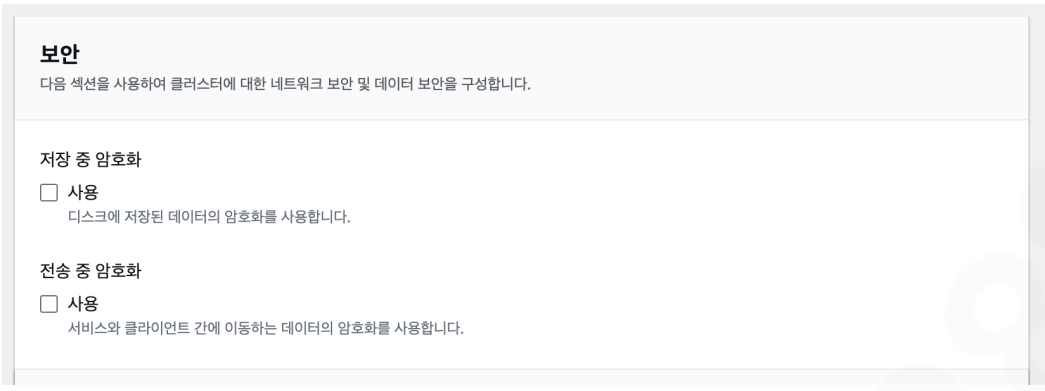
8. **기존 서브넷 그룹 선택하기**



9. **'가용 영역 배치'는 설정하지 않고 다음으로 넘어가기**

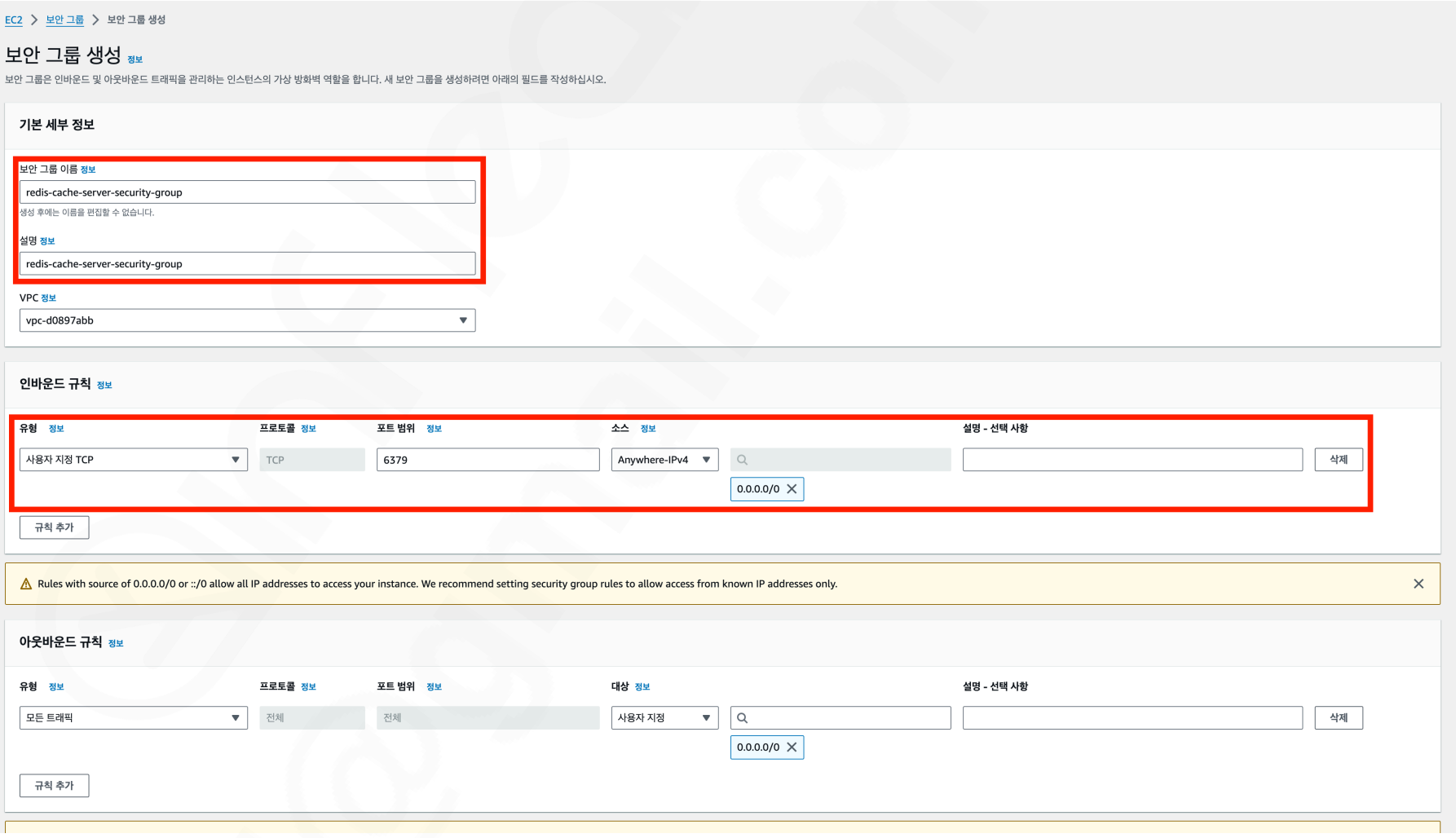


10. ‘보안’ 설정하지 않기



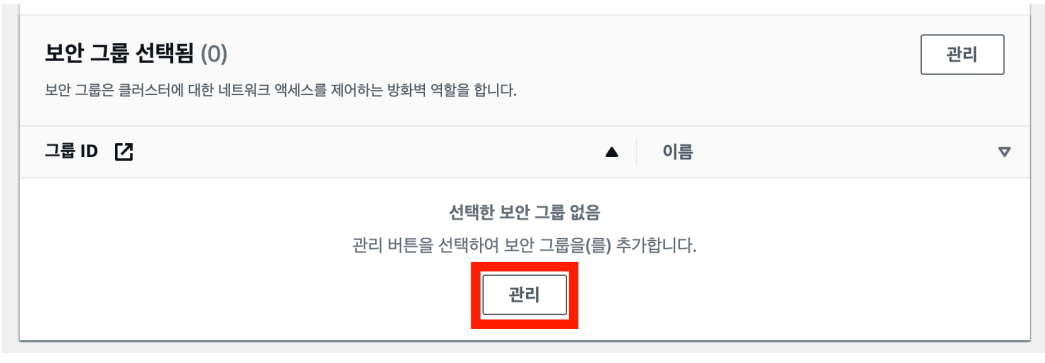
현재로썬 중요한 기능이 아니기 때문에 체크하지 않고 넘어가자. 이 옵션을 체크하지 않는다고 해서 보안적으로 치명적인 건 아니니 걱정하지 말자.

11. EC2 서비스로 가서 보안그룹 생성하기



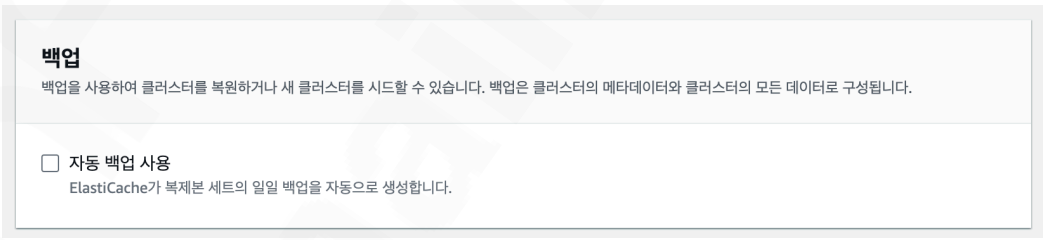
6379번 포트로 대해 모든 IP가 접근하게 허용하면 보안적으로 위험한 건 아닌가 걱정될 수 있다. 하지만 ElastiCache는 기본적으로 같은 VPC 내에서만 접근을 할 수 있고, 외부 IP에 대해 접근하는 게 막혀있다. 따라서 보안 그룹에서 모든 IP를 대상으로 허용한다고 하더라도 외부에서 접근할 수는 없으니 걱정하지 않아도 된다.

12. ‘보안그룹’ 셋팅하기





13. **'백업' 설정 해제하기**



임시로 데이터를 저장하는 캐싱 용도로 쓸 거기 때문에 백업 옵션은 해제하자.

14. **나머지 옵션은 그대로 두고 다음으로 넘어가기**

khb3025@gmail.com


유지 관리
클러스터에 대한 유지 관리 설정을 구성합니다.
유지 관리 기간
운영 체제 패치 적용, 드라이버 업데이트, 소프트웨어 또는 패치 설치와 같은 업데이트의 시간 범위(UTC)를 지정합니다.
기본 설정 없음
유지 관리 기간 지정
마이너 버전 자동 업그레이드
사용
최신 마이너 버전이 이용 가능하게 되면 클러스터 업그레이드를 자동으로 예약합니다. 클러스터 업그레이드는 유지 관리 기간 중에만 예약됩니다.
Amazon SNS 알림 주제
목록에서 SNS 주제를 선택하거나 기존 주제에 대한 Amazon 리소스 이름(ARN)을 입력합니다. 주제를 선택하지 않으면 알림이 전송되지 않습니다.
알림 사용 중지
로그
Redis 느린 로그 또는 엔진 로그를 제공할지 여부를 지정합니다.
느린 로그
사용
지정된 런타임을 초과하는 쿼리에 대해 Redis 느린 로그를 제공합니다.
엔진 로그
사용
지정된 런타임을 초과하는 쿼리에 대한 엔진 로그를 제공합니다.
태그
태그를 사용하여 클러스터을(를) 검색 및 필터링하거나 AWS 비용을 추적할 수 있습니다.
클러스터에 연결된 태그가 없습니다.
새 태그 추가
태그를 50개 더 추가할 수 있습니다.
취소
이전
다음

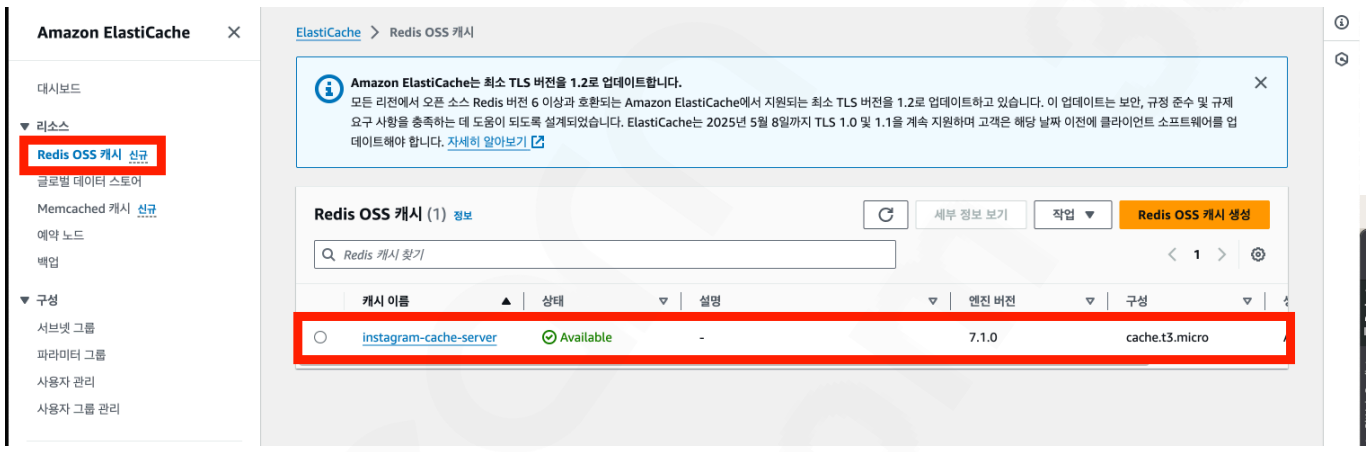
15. **'생성' 누르기**


자동 백업
비활성화됨
유지 관리
운영 체제 패치 적용, 드라이버 업데이트, 소프트웨어 또는 패치 설치와 같은 업데이트의 시간 범위(UTC)를 지정합니다.
유지 관리 기간
기본 설정 없음
마이너 버전 자동 업그레이드
활성화됨
Amazon SNS 알림 주제
SNS topic notifications disabled
로그
Redis 느린 로그 또는 엔진 로그를 제공할지 여부를 지정합니다.
느린 로그
비활성화됨
엔진 로그
비활성화됨
태그
태그를 사용하여 clusters을(를) 검색 및 필터링하거나 AWS 비용을 추적할 수 있습니다.
키
값
태그을(를) 찾을 수 없습니다.
생성한 태그이(가) 여기에 표시됩니다.
취소
이전
생성

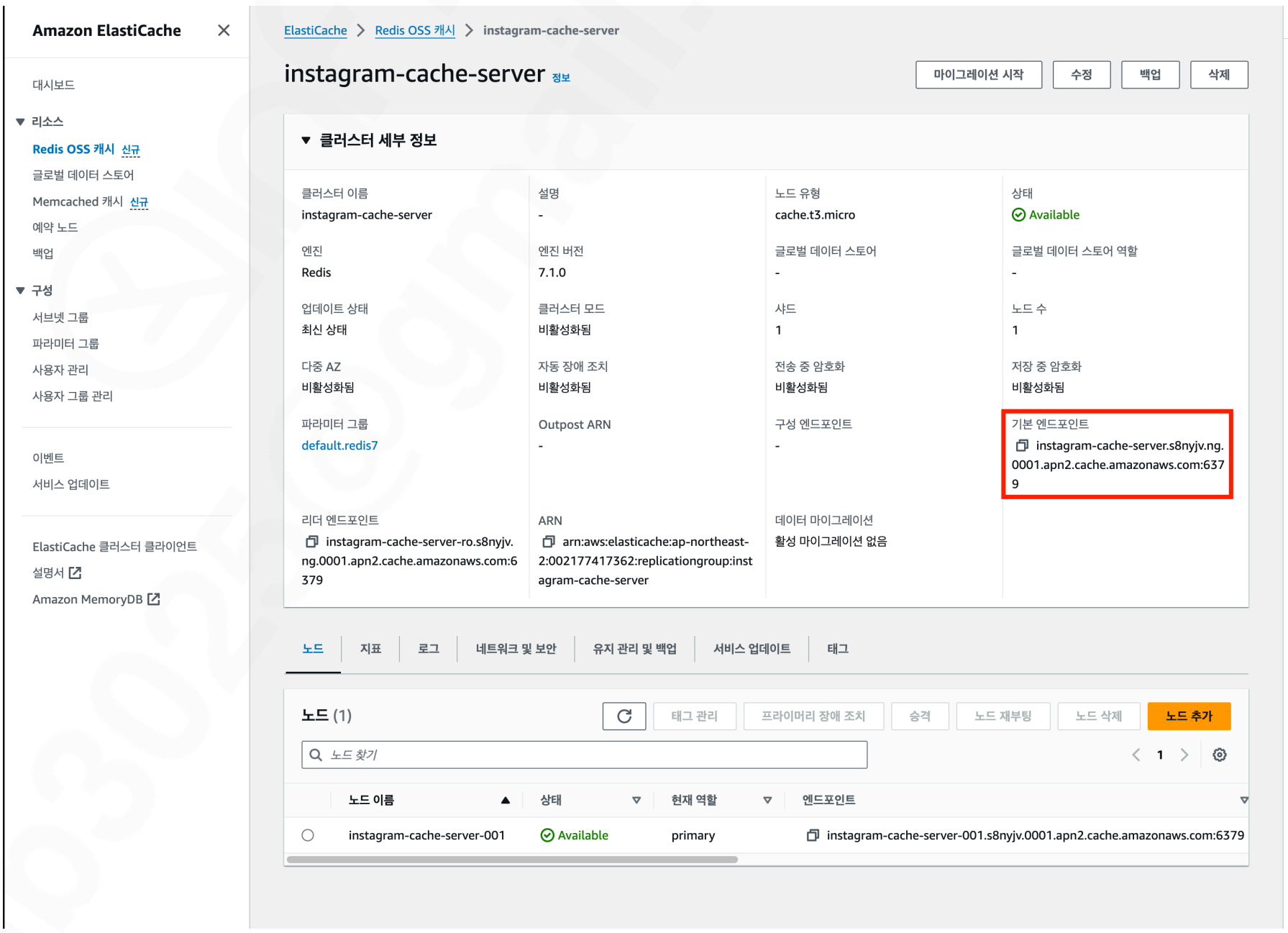
AWS ElastiCache가 정상적으로 잘 생성됐는 지 확인하기

✅ AWS ElastiCache가 정상적으로 잘 생성됐는 지 확인하기

1. **ElastiCache 대시보드 들어가기**



2. **'기본 엔드포인트' 주소에서 포트 번호 빼고 복사하기**



- **참고)** '리더(Reader) 엔드포인트'는 읽기 전용 주소이다.

3. **EC2에 들어가서 ElastiCache에 접속해보기**

```
# redis-cli -h {호스트 주소}
$ redis-cli -h {ElastiCache의 기본 엔드포인트}
```

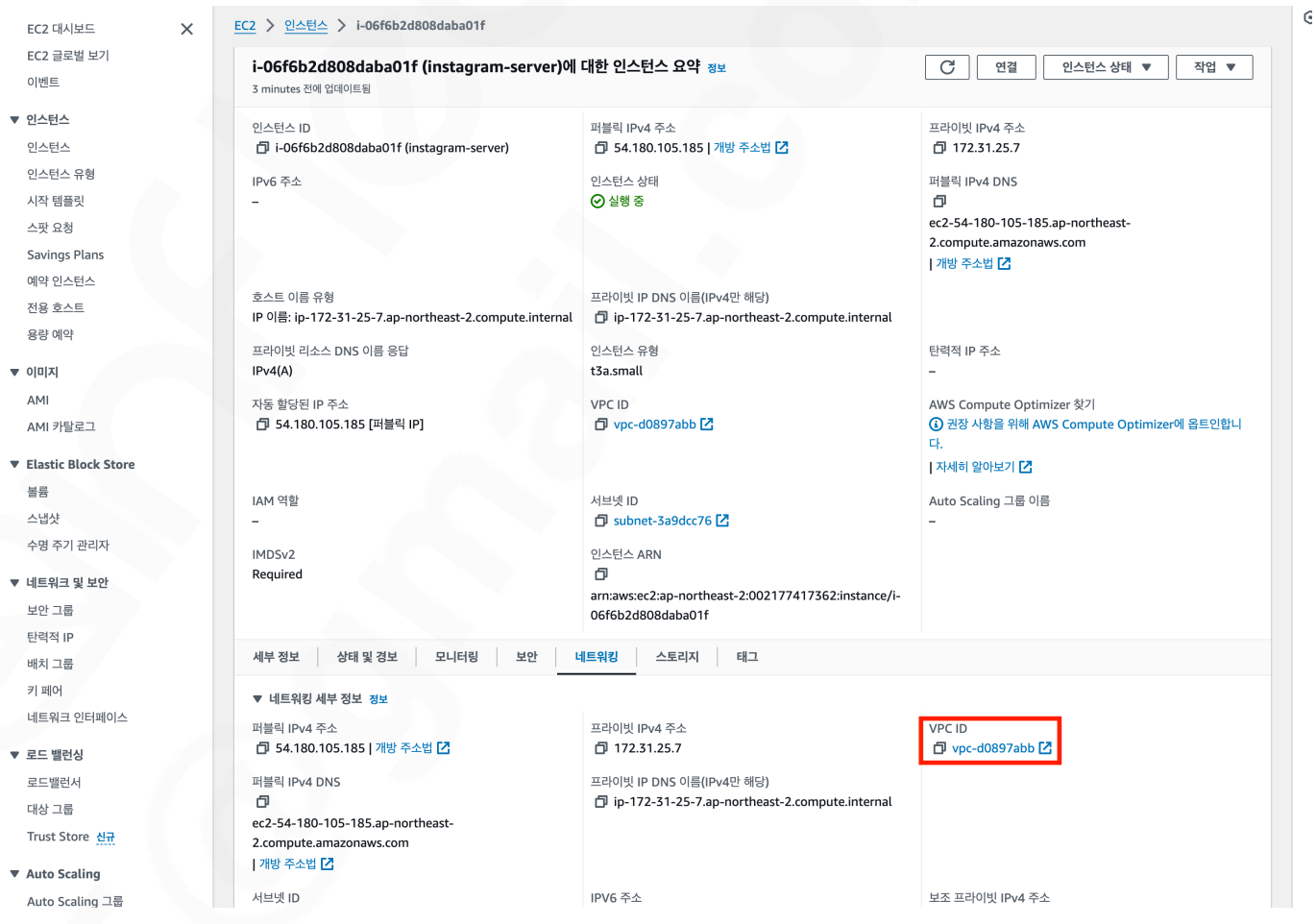
```
ubuntu@ip-172-31-25-7:~$ redis-cli -h instagram-cache-server.s8nyjv.ng.0001.apn2.cache.amazonaws.com
instagram-cache-server.s8nyjv.ng.0001.apn2.cache.amazonaws.com:6379> ping
PONG
instagram-cache-server.s8nyjv.ng.0001.apn2.cache.amazonaws.com:6379>
```

4. **로컬 환경에서 접속되는 지 테스트해보기**

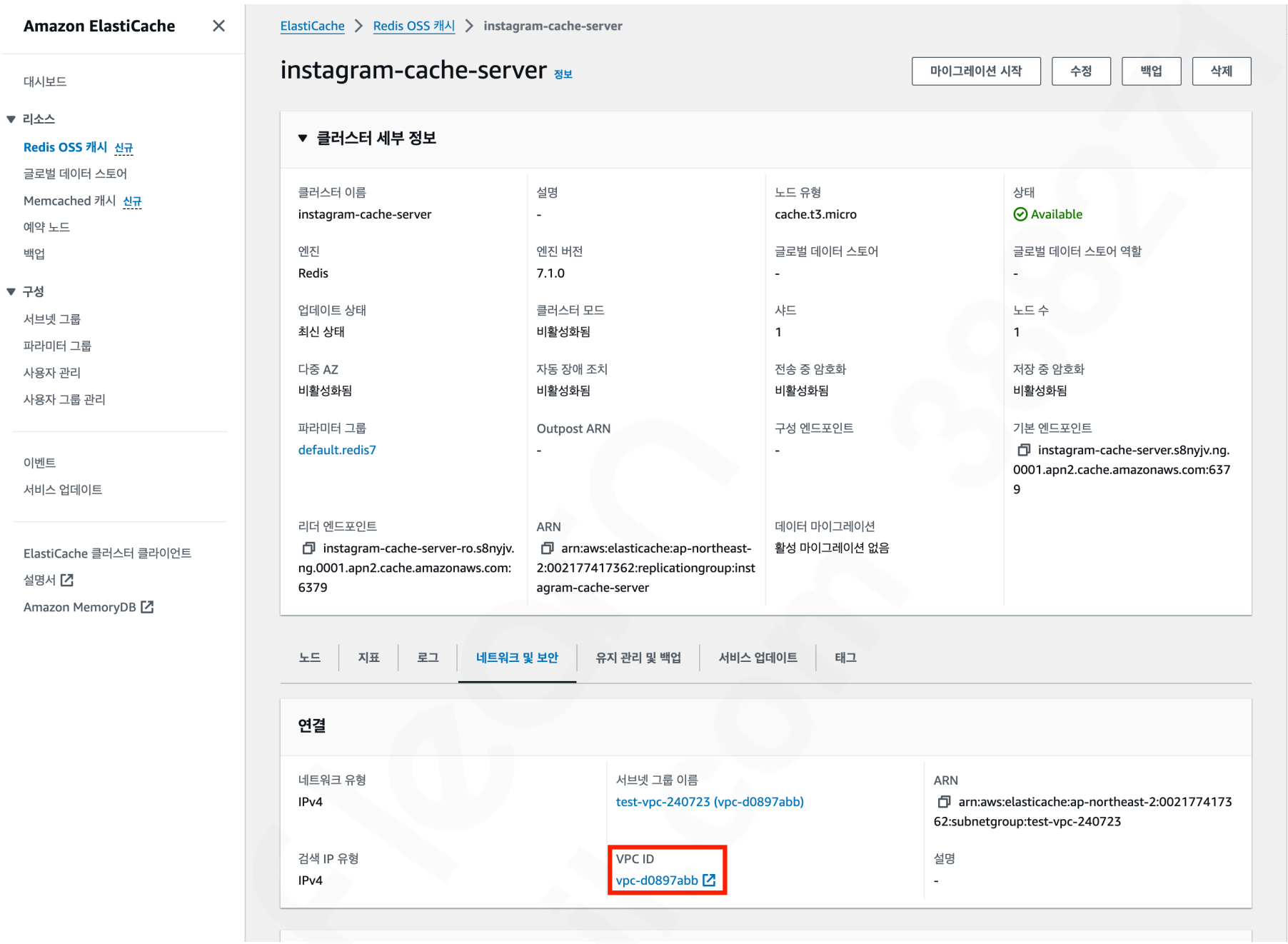
```
jaeseong ~ redis-cli -h instagram-cache-server.s8nyjv.ng.0001.apn2.cache.amazonaws.com
```

접속이 안 되는 걸 알 수 있다. 왜냐하면 ElastiCache는 같은 VPC일 때만 접속할 수 있기 때문이다. 그럼 EC2와 ElastiCache가 같은 VPC인지 확인해보자.

EC2의 VPC 정보



ElasitCache의 VPC 정보



kb302500@gmail.com

📖

Spring Boot에 ElastiCache 연결하기

✅ Spring Boot에 ElastiCache 연결하기

1. **application.yml 파일 수정하기**

   application.yml

```
# local 환경
spring:
  profiles:
    default: local
  datasource:
    url: jdbc:mysql://host.docker.internal:3306/mydb
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  data:
    redis:
      host: cache-server
      port: 6379

logging:
  level:
    org.springframework.cache: trace


---
# prod 환경
spring:
  config:
    activate:
      on-profile: prod
  datasource:
    url: jdbc:mysql://instagram-db.coseefawhrzc.ap-northeast-2.rds.amazonaws.com:3306/mydb
    username: admin
    password: password
  data:
    redis:
      host: instagram-cache-server.s8nyjv.ng.0001.apn2.cache.amazonaws.com
      port: 6379
```

2. **Github Repository에 Push하기**

3. **EC2에서 Git Pull 받기**

```
$ cd {프로젝트 경로 }
$ git pull origin main
```

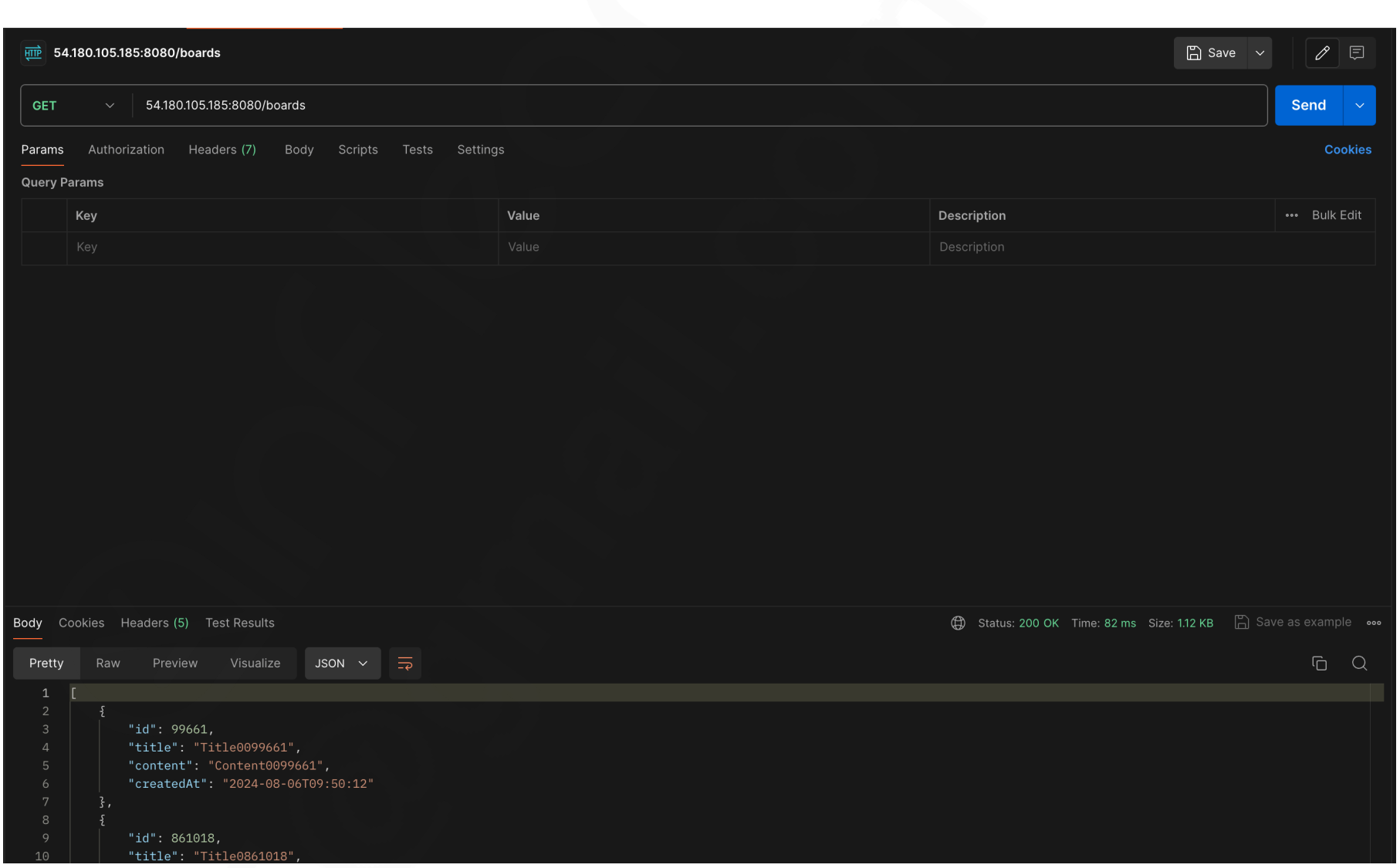
4. **기존 서버 종료시키기**

```
$ docker compose down # 이전 실습에서 실행시켰던 컨테이너 종료시키기
$ docker ps # 종료됐는 지 확인
```

5. **Spring Boot 프로젝트 실행시키기**

```
$ ./gradlew clean build -x test
$ cd build/libs
$ java -jar -Dspring.profiles.active=prod {빌드된 jar 파일명}
```

6. **Postman으로 테스트해보기**



7. **실제 ElastiCache에 캐시가 저장되고 있는 지 확인해보기**

   새로운 EC2 창 열어서 아래 명령어를 통해 ElastiCache에 접속하기

```
$ redis-cli -h {ElastiCache의 기본 엔드포인트}
$ keys *
$ get getBoards::boards:page:1:size:10
$ ttl getBoards::boards:page:1:size:10
```

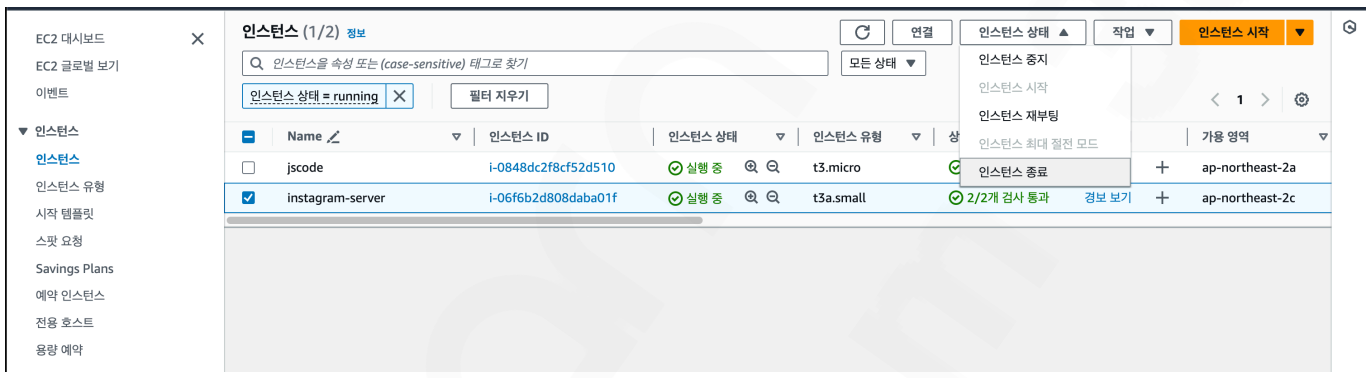
✅ 전체 흐름 다시 복습하기

📖 EC2, RDS, Spring Boot, ElastiCache를 활용한 아키텍처 구성

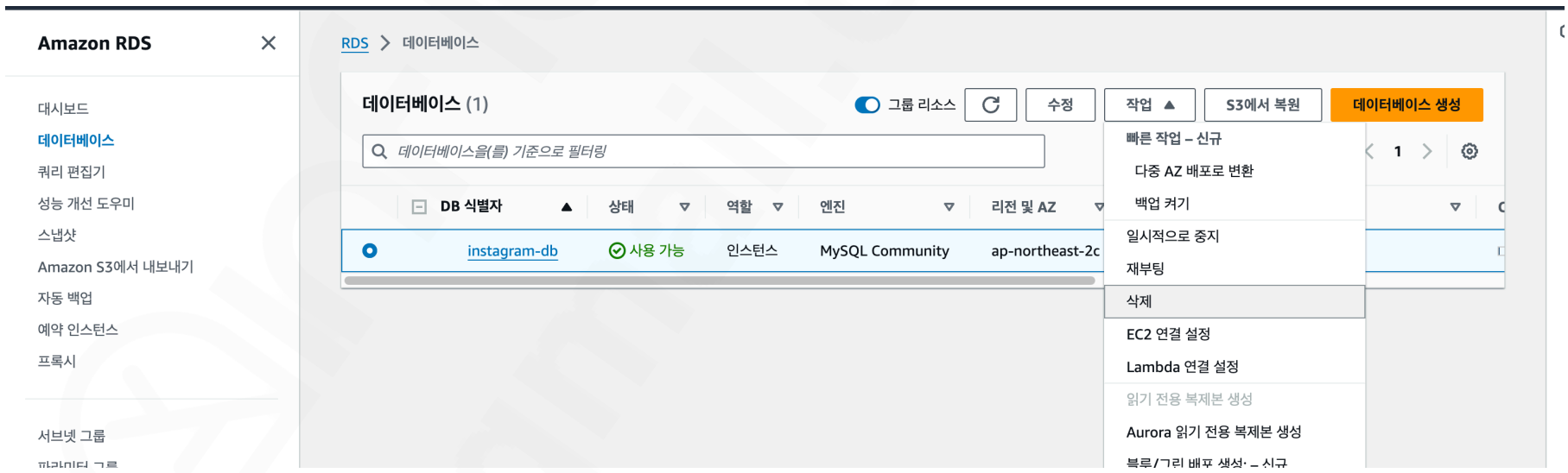
비용나가지 않게 지금까지 사용했던 AWS 리소스 종료하기

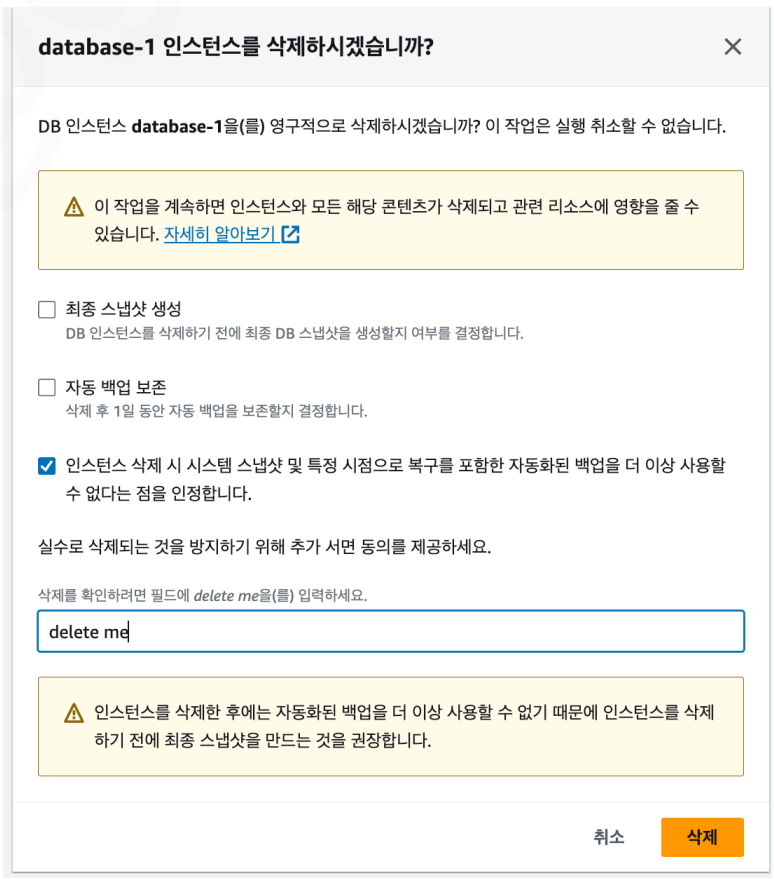
✅ 비용나가지 않게 지금까지 사용했던 AWS 리소스 종료하기

1. **EC2 인스턴스 종료하기**



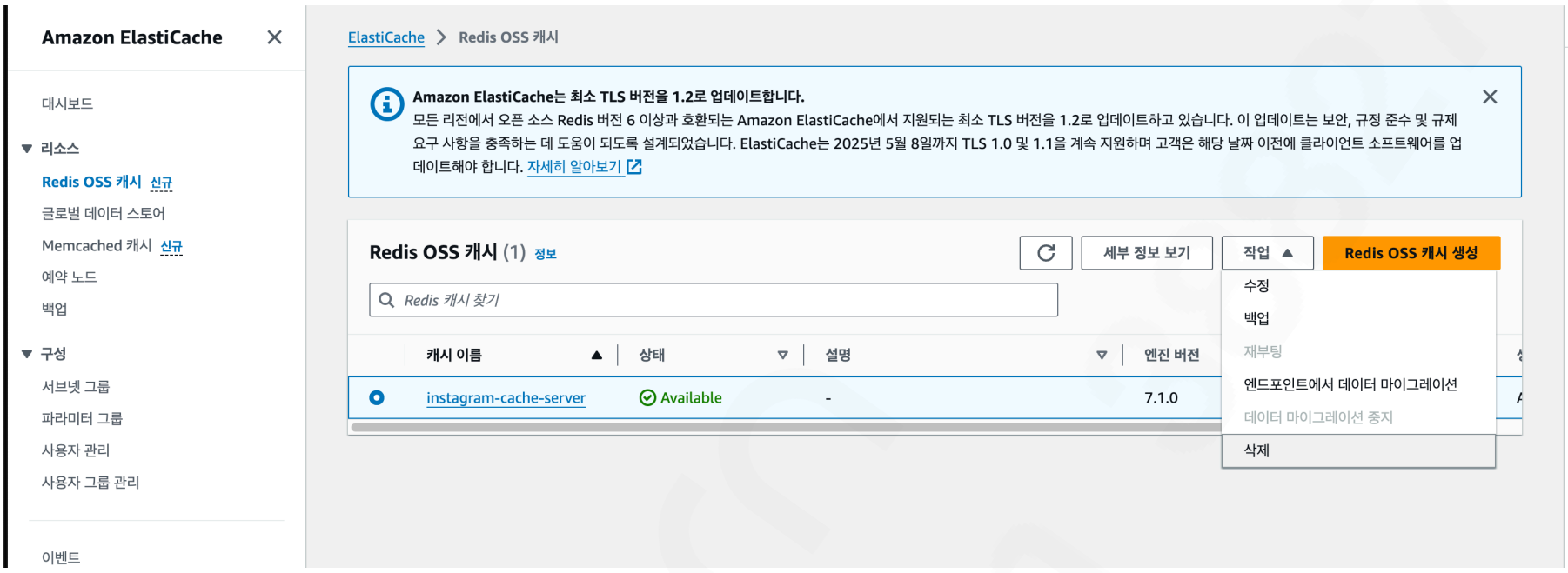
2. **RDS 인스턴스 종료하기**

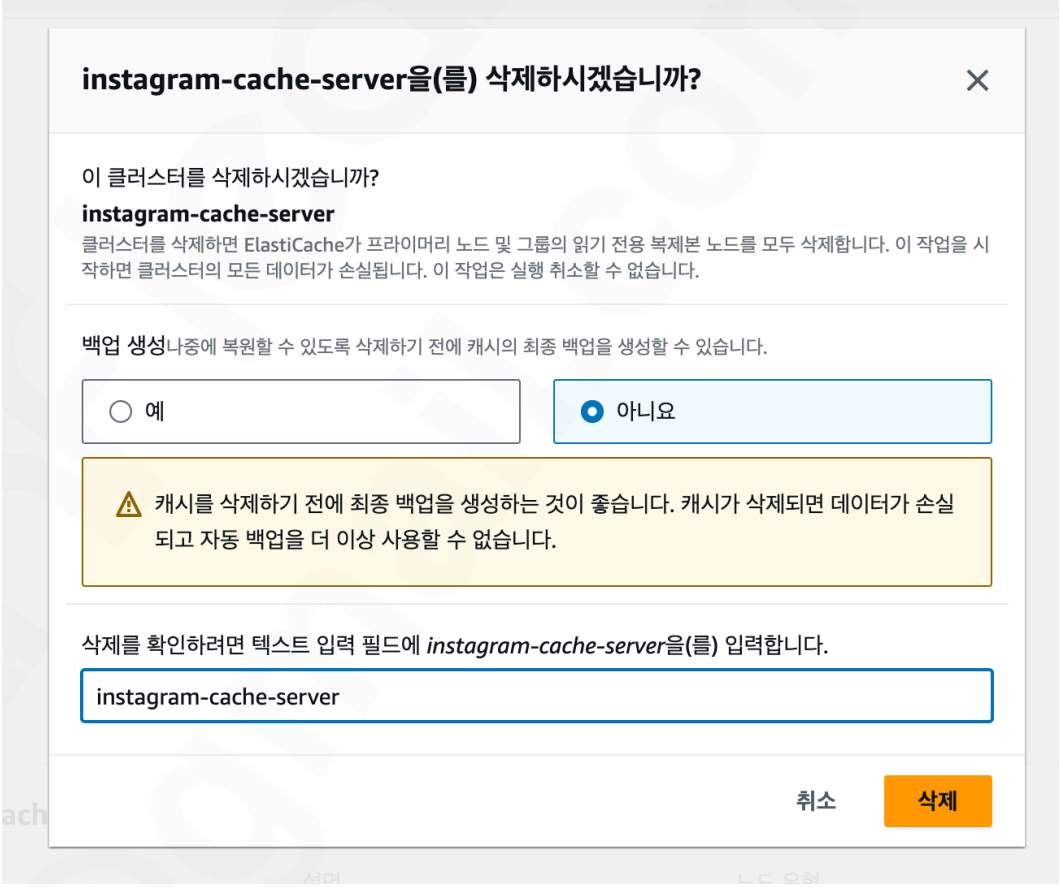




최종 스냅샷 생성과 자동 백업 보존을 체크하면 비용이 나간다. 따라서 실제 운영용 데이터베이스가 아니라면 체크를 해제하고 삭제를 하자.

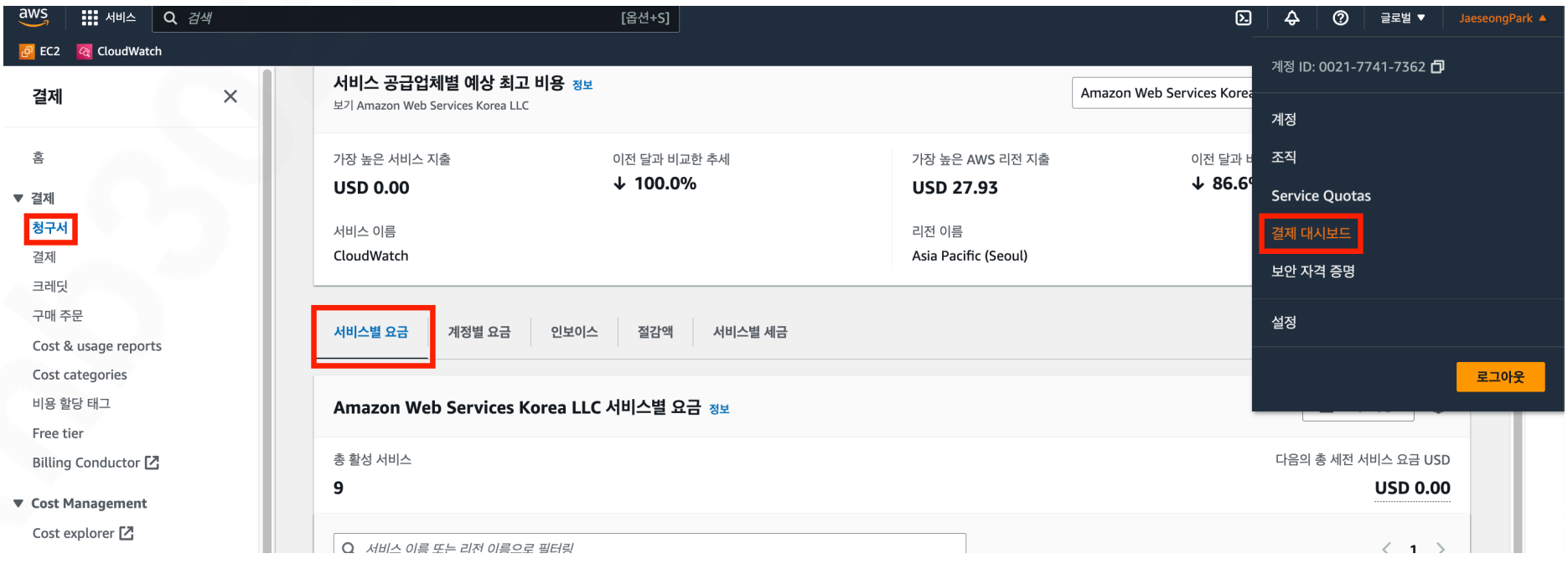
3. ElastiCache 종료하기





백업을 생성하면 비용이 나간다. 따라서 백업 생성에 `아니오`를 체크하고 삭제하자.

✅ 혹시나 비용이 많이 나가고 있는 건 아닌 지 체크하는 방법



Amazon Web Services Korea LLC 서비스별 요금 정보

모두 확장

총 활성 서비스

9

다음의 총 세전 서비스 요금 USD

USD 0.00

서비스 이름 또는 리전 이름으로 필터링

1

설명	사용량 수량	의 금액 USD
CloudWatch		USD 0.00
Data Transfer		USD 0.00
EC2 Container Registry (ECR)		USD 0.00
Elastic Compute Cloud		USD 0.00
Elastic Load Balancing		USD 0.00
Key Management Service		USD 0.00
Relational Database Service		USD 0.00
Simple Storage Service		USD 0.00
Virtual Private Cloud		USD 0.00
총 세금		**USD 0.00**

kb302500@gmail.com